คณะเทคโนโลยีสารสนเทศ
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

Chapter 13

Deep Learning for Video Processing

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง



หัวข้อ

- ข้อมูลแบบอนุกรมเวลา (Time Series)
- โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)
- หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)
- หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น
  (Convolution Long Short-Term Memory: ConvLSTM)
- วิธีการสร้าง Convolution LSTM ด้วย Pytorch
- โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ



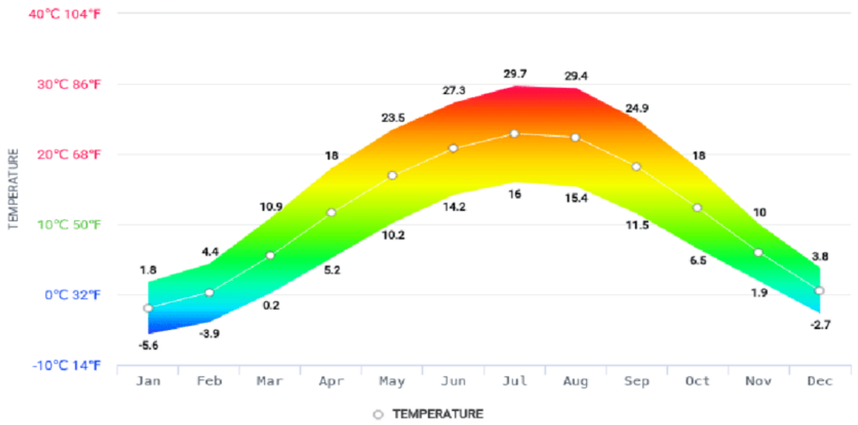
หัวข้อ

- ข้อมูลแบบอนุกรมเวลา (Time Series)
- โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)
- หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)
- หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น (Convolution Long Short-Term Memory: ConvLSTM)
- วิธีการสร้าง Convolution LSTM ด้วย Pytorch
- โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ

ข้อมูลแบบอนุกรมเวลา (Time Series)



https://www.nerdwallet.com/assets/blog/wp-content/uploads/2023/07/Image-7-14-23-at-4.17-PM-770x462.jpeg

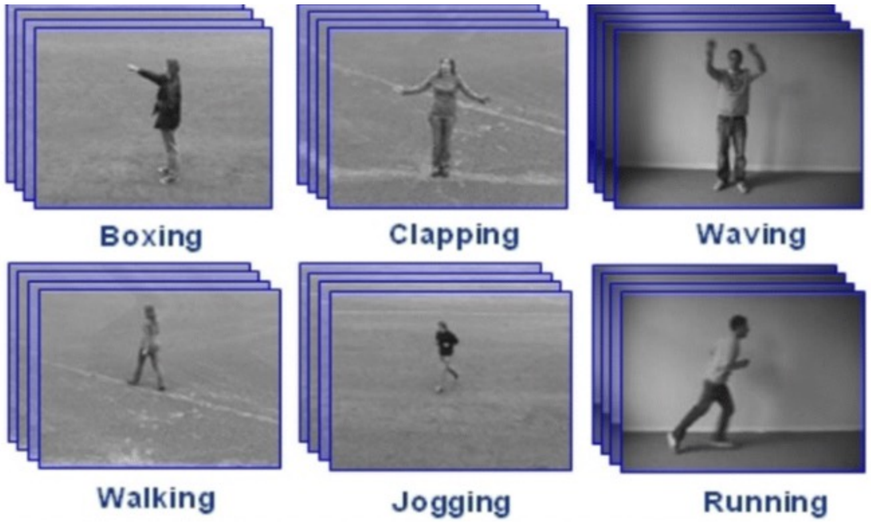


https://www.researchgate.net/publication/332383223/figure/fig1/AS:746928123936768@1555092950420/The-average-day-night-and-daily-temperature-in-Ruse-for-each-month-of-the-year.png

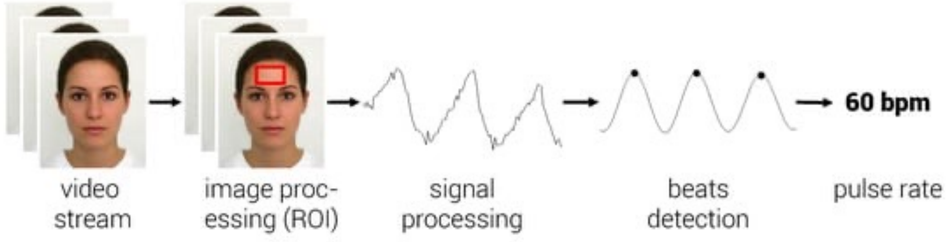
คือ ชุดข้อมูลที่บันทึกในช่วงเวลาต่อเนื่องกัน โดยข้อมูลแต่ละตัวจะถูกจัดเก็บตามลำดับเวลา ข้อมูลเหล่านี้มักจะใช้ในการวิเคราะห์แนวโน้ม (Trend Analysis) และรูปแบบที่เกิดขึ้นตามเวลา (Seasonal Analysis) รวมถึงการทำนายค่าในอนาคต (Forecasting) ตัวอย่างทั่วไปของข้อมูลแบบอนุกรมเวลา ได้แก่ ข้อมูลราคาหุ้น, ข้อมูลอุณหภูมิ, ข้อมูลการผลิตสินค้าในโรงงาน, และข้อมูลผู้เข้าชมเว็บไซต์ในแต่ละวัน

- **ราคาหุ้น (Stock Prices):** ข้อมูลราคาหุ้นจะถูกบันทึกในทุกๆ วันหรือทุกๆ นาที ขึ้นอยู่กับความถี่ในการซื้อขาย
- **อุณหภูมิรายวัน (Daily Temperature):** ข้อมูลอุณหภูมิที่วัดได้ในแต่ละวัน ตัวอย่างเช่น อุณหภูมิสูงสุดในกรุงเทพฯ ในแต่ละวัน
- **ปริมาณการขายสินค้า (Sales Volume):** ข้อมูลจำนวนสินค้าที่ขายได้ในแต่ละเดือน ตัวอย่างเช่น จำนวนเครื่องใช้ไฟฟ้าที่ขายได้ในแต่ละเดือน

ข้อมูล Time Series สำหรับวิดีโอ



https://media.springernature.com/lw685/springer-static/image/art%3A10.1007%2Fs11042-020-08806-9/MediaObjects/11042_2020_8806_Fig7_HTML.png



https://www.mdpi.com/applsci/applsci-09-04364/article_deploy/html/images/applsci-09-04364-g001-550.jpg

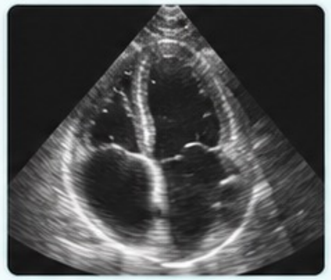
คือ การใช้ข้อมูลแบบอนุกรมเวลา (Time Series) สำหรับวิดีโอเป็นการวิเคราะห์ข้อมูลที่เกี่ยวข้องกับลำดับของเฟรมที่เกิดขึ้นตามเวลา ซึ่งเป็นข้อมูลที่มีความซับซ้อนมากกว่าข้อมูลแบบอนุกรมเวลาทั่วไป เนื่องจากประกอบด้วยทั้งข้อมูลเชิงพื้นที่ (spatial data) และข้อมูลเชิงเวลา (temporal data)

ตัวอย่างการใช้งาน

- การตรวจจับการกระทำ (Action Recognition): วิดีโอที่บันทึกการเล่นกีฬาจะมีลำดับของเฟรมที่แสดงการเคลื่อนไหวต่างๆ การวิเคราะห์ข้อมูลนี้สามารถใช้ในการตรวจจับการกระทำ เช่น การเตะบอล การกระโดด หรือการหมุนตัว
- การตรวจจับเหตุการณ์ (Event Detection): ใช้ในการตรวจจับเหตุการณ์เฉพาะในวิดีโอ เช่น การตรวจจับการชนในวิดีโอจากกล้องวงจรปิด หรือการตรวจจับการล้มของผู้สูงอายุ ซึ่งสามารถใช้งานโมเดล Convolutional Neural Networks (CNN) ร่วมกับ Recurrent Neural Networks (RNN) หรือ Long Short-Term Memory (LSTM)
- การวิเคราะห์พฤติกรรม (Behavior Analysis): ใช้ในการวิเคราะห์พฤติกรรมของบุคคลหรือวัตถุในวิดีโอ เช่น การวิเคราะห์พฤติกรรมการเดินของผู้คนในสถานีรถไฟ ซึ่งสามารถใช้งานโมเดล 3D Convolutional Neural Networks (3D CNN)

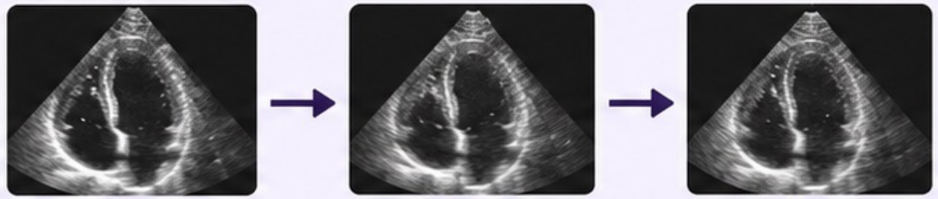
ความแตกต่างของภาพและวิดีโออยู่ที่มิติเวลา

ภาพนิ่ง (Image)



วิเคราะห์สิ่งที่อยู่ในภาพ

วิดีโอ (Video)



วิเคราะห์สิ่งที่เปลี่ยนไปตามเวลา

ประเด็น	ภาพนิ่ง	วิดีโอ
จำนวนเฟรม	1 เฟรม	หลายเฟรมต่อเนื่อง
มิติเวลา	ไม่มีลำดับเวลาภายในภาพ	มีความสัมพันธ์ระหว่างเฟรม
ลำดับเฟรม	ไม่เกี่ยวข้องเมื่อพิจารณาภาพเดี่ยว	สำคัญต่อการตีความการเคลื่อนไหว
บริบทที่ใช้	Spatial	Spatial + **Temporal**
ตัวอย่างงาน	แบ่งส่วนอวัยวะในภาพ	วิเคราะห์การเคลื่อนไหวของหัวใจ

Spatial = รูปร่าง ตำแหน่ง และความสัมพันธ์ในภาพ | **Temporal** = การเปลี่ยนแปลงและความสัมพันธ์ข้ามเวลา

Spatial และ Temporal: สองมุมมองของวิดีโอ

ข้อมูลจากแต่ละเฟรม + ความสัมพันธ์ข้ามเวลา

Spatial Features | ลักษณะเชิงพื้นที่

ในเฟรมนี้ มีอะไร อยู่ที่ไหน และมีลักษณะอย่างไร?



ความสัมพันธ์ของข้อมูลภายในเฟรมเดียว (x, y)

Temporal Features | ลักษณะเชิงเวลา

สิ่งนั้นเปลี่ยนไปอย่างไร เมื่อเวลาผ่านไป?



- ลำดับและทิศทาง
- ความเร็วและระยะเวลา
- จังหวะและการเปลี่ยนแปลง

ความสัมพันธ์ระหว่างหลายเฟรม (t)

ใช้ร่วมกันเป็น Spatiotemporal Features

Spatial Features
บอกลักษณะของสิ่งต่างๆ ในแต่ละเฟรม

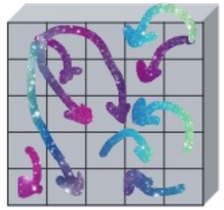
เข้าใจว่าใครหรืออะไร กำลังทำอะไร และเปลี่ยนไปอย่างไร

Temporal Features
บอกการเปลี่ยนแปลงของสิ่งนั้นในช่วงเวลา

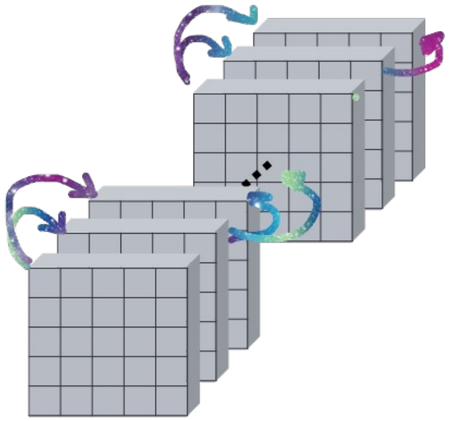
ทั้งสองเป็นคุณลักษณะที่สกัดจากวิดีโอ แต่พิจารณาความสัมพันธ์คนละมิติ

Temporal ไม่ได้มีแค่การเคลื่อนไหว: สี ความสว่าง และรูปร่างก็เปลี่ยนตามเวลาได้

ข้อมูล Time Series สำหรับวิดีโอ



คุณลักษณะเชิงตำแหน่ง (Spatial Feature)



ความสัมพันธ์ระหว่างเฟรมที่ต่อเนื่องกัน (Temporal Feature)

ในการประมวลผลข้อมูลประเภทวิดีโอ เราจำเป็นต้องพิจารณาลักษณะเชิงพื้นที่ (Spatial Feature) และลักษณะเชิงเวลา (Temporal Feature) พร้อมกัน เนื่องจากวิดีโอเป็นการรวมกันของภาพที่ต่อเนื่องกันซึ่งมีข้อมูลทั้งสองมิติอยู่

- **Spatial Feature** เป็นลักษณะเชิงพื้นที่คือข้อมูลที่เกี่ยวข้องกับรายละเอียดภายในแต่ละเฟรมของวิดีโอ ซึ่งเป็นข้อมูลที่บอกถึงโครงสร้างและลักษณะเฉพาะของวัตถุภายในภาพ เช่น ขอบเขต สี รูปร่าง และพื้นผิว ซึ่งสามารถสกัดด้วยเทคนิค Convolutional Neural Networks (CNNs) ซึ่งใช้ฟิลเตอร์คอนโวลูชันเพื่อดึงลักษณะต่างๆ จากภาพ
- **Temporal Feature** เป็นลักษณะเชิงเวลาคือข้อมูลที่เกี่ยวข้องกับการเปลี่ยนแปลงของภาพในแต่ละเฟรมเมื่อเทียบกับเฟรมก่อนหน้า เป็นการบ่งบอกถึงความสัมพันธ์และการเคลื่อนไหวของวัตถุภายในวิดีโอ ซึ่งสามารถสกัดลักษณะเชิงเวลามักจะใช้เทคนิค Recurrent Neural Networks (RNNs) หรือ Long Short-Term Memory (LSTM) ซึ่งเหมาะสมกับการจัดการข้อมูลที่มีลำดับเวลา

การรวมสกัดคุณลักษณะทางด้าน Spatial และ Temporal ร่วมกันจากวิดีโอที่ซับซ้อนมักจะใช้การผสมผสานระหว่าง CNNs และ LSTMs เพื่อสกัดลักษณะเชิงพื้นที่จากแต่ละเฟรม และใช้ LSTMs เพื่อจับความสัมพันธ์เชิงเวลาในเฟรมต่อเนื่อง นอกจากนี้ ยังมีเทคนิค 3D Convolutional Networks (3D CNNs) ที่ใช้ฟิลเตอร์สามมิติในการจับลักษณะเชิงพื้นที่และเชิงเวลาในครั้งเดียว

ความสำคัญของมิติเวลา

เฟรมเดิม แต่ลำดับต่างกัน ความหมายก็เปลี่ยน

ลำดับเวลา (Temporal order) บอกทิศทางของการกระทำ



ภาพเดี่ยวอาจบอกได้เพียงท่าทาง แต่ลำดับเฟรมบอกว่ากำลังทำอะไร

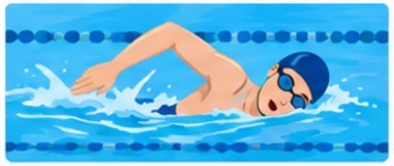
เฟรมเหมือนเดิม + ลำดับต่างกัน → การกระทำต่างกัน

ความเชื่อมโยงงานภาพในมุมมองงานวิดีโอ

ภาพ: มิติเชิงพื้นที่ (Spatial)

วิดีโอ: มิติเวลา (Temporal)

1 Image Classification
หนึ่งป้ายกำกับต่อภาพ



→ ว่ายน้ำ

Video Classification
หนึ่งป้ายกำกับต่อคลิป



→ ว่ายน้ำ

สรุปกิจกรรมของทั้งคลิป

2 Image Segmentation
ป้ายกำกับทุกพิกเซล



แบ่งพื้นที่ในภาพ

Temporal Action Segmentation
ป้ายกำกับทุกเฟรม



แบ่งลำดับกิจกรรมตลอดคลิป

3 Object Detection
ประเภทวัตถุ + กรอบตำแหน่ง



Temporal Action Localization
ประเภทการกระทำ + เวลาเริ่ม–จบ



ยิงประตู: 00:15–00:20

เทียบแนวคิดของผลลัพธ์: การแบ่งตามเวลา ≠ การแบ่งพิกเซลในวิดีโอ

1 Video Classification

จำแนกกิจกรรมของทั้งคลิป

Goal	จำแนกว่าคลิปวิดีโอเป็นคลาสอะไร
Input	วิดีโอหนึ่งคลิป มักตัดให้มีกิจกรรมหลักชัดเจน
Ground Truth	หนึ่งป้ายกำกับต่อคลิป เช่น ลุกขึ้นยืน เดิน **หรือนั่งลง**
Focus	พิจารณาข้อมูลจากหลายเฟรมเพื่อสรุปทั้งคลิป

Input: คลิปการเคลื่อนไหว



→ **Video Model** →

ผลลัพธ์ทั้งคลิป

ลุกขึ้นยืน

(1 class label)

ตัวอย่างนี้เป็น Single-label Classification: 1 คลิป → 1 คลาส

ตัวอย่างการใช้งาน



จำแนกท่ากายภาพบำบัด

ลุกขึ้นยืน / ยกแขน



จำแนกกิจกรรมกีฬา

วิ่ง / เล่นบาสเกตบอล



จำแนกเหตุการณ์ในคลิป

หกล้ม / กิจกรรมทั่วไป

2 Temporal Action Segmentation

ระบุการกระทำทุกเฟรม และแบ่งช่วงกิจกรรมเตามเวลา

Goal	จำแนกว่าทุกเฟรมของวิดีโอกำลังทำกิจกรรมอะไร
Input	วิดีโอยาวแบบ Untrimmed ที่มีหลายกิจกรรมต่อเนื่องกัน
Ground Truth	ป้ายกำกับกิจกรรมทุกเฟรมหรือทุก Time step
Focus	ทุกเฟรมตลอดวิดีโอ และรอยต่อระหว่างกิจกรรม

ตัวอย่าง: วิเคราะห์ขั้นตอนการทำอาหาร



ตัวอย่างการใช้งาน



วิเคราะห์
ขั้นตอนการผ่าตัด



วิเคราะห์
ขั้นตอนการทำอาหาร



วิเคราะห์
ขั้นตอนการประกอบสินค้า

3 Temporal Action Localization

ค้นหาว่าการกระทำเกิดขึ้นเมื่อไร และสิ้นสุดเมื่อไร

Goal
ระบุประเภทการกระทำ
พร้อมเวลาเริ่มต้นและสิ้นสุด

Input
วิดีโอยาวแบบ Untrimmed
ที่มีเหตุการณ์และช่วงที่ไม่ใช่เป้าหมาย

Ground Truth
รายการ (class, start_time, end_time) ของแต่ละเหตุการณ์

Focus
ช่วงเวลาของเหตุการณ์ที่สนใจ

ตัวอย่าง: ค้นหาเหตุการณ์ในวิดีโอฟุตบอล



ผลลัพธ์: ประเภท + ช่วงเวลา



การกระทำ	เวลาเริ่ม – จบ (นาที:วินาที)
ยิงประตู	01:15 – 01:20
ทำฟาวล์	02:42 – 02:48

เวลาสมมติสำหรับอธิบาย

ตัวอย่างการใช้งาน



ค้นหาช่วงยิงประตู
หรือทำฟาวล์



ค้นหาช่วงผู้สูงอายุหกล้ม



ค้นหาช่วงอุบัติเหตุ
ในวิดีโอจราจร

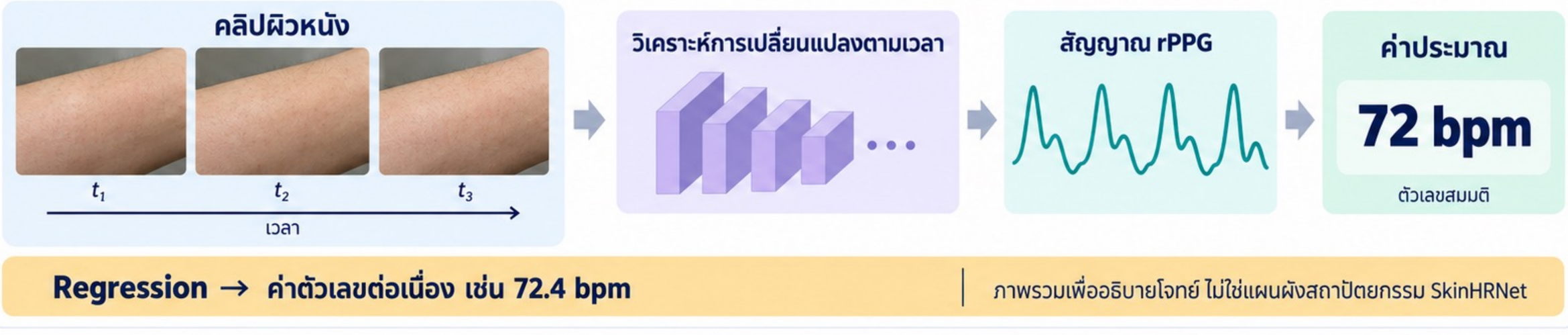
ผลลัพธ์เป็นช่วงเวลา ไม่จำเป็นต้องให้ป้ายกำกับทุกเฟรม

4 Video Regression

ทำนายค่าตัวเลขต่อเนื่องจากวิดีโอ

Goal	ประมาณค่าตัวเลขจากข้อมูลในหลายเฟรม
Input	คลิปวิดีโอที่มีข้อมูลสัมพันธ์กับค่าที่ต้องการทำนาย
Ground Truth	ค่าตัวเลขอ้างอิง เช่น อัตราการเต้นหัวใจเฉลี่ยของคลิป
Focus	ลักษณะเชิงพื้นที่และการเปลี่ยนแปลงตามเวลา

ตัวอย่างแนวคิดจากงาน SkinHRNet: ประมาณอัตราการเต้นหัวใจ



Regression → ค่าตัวเลขต่อเนื่อง เช่น 72.4 bpm

ภาพรวมเพื่ออธิบายโจทย์ ไม่ใช่แผนผังสถาปัตยกรรม SkinHRNet

ตัวอย่างการใช้งาน



ประมาณอัตราการเต้นหัวใจ
หน่วย: bpm

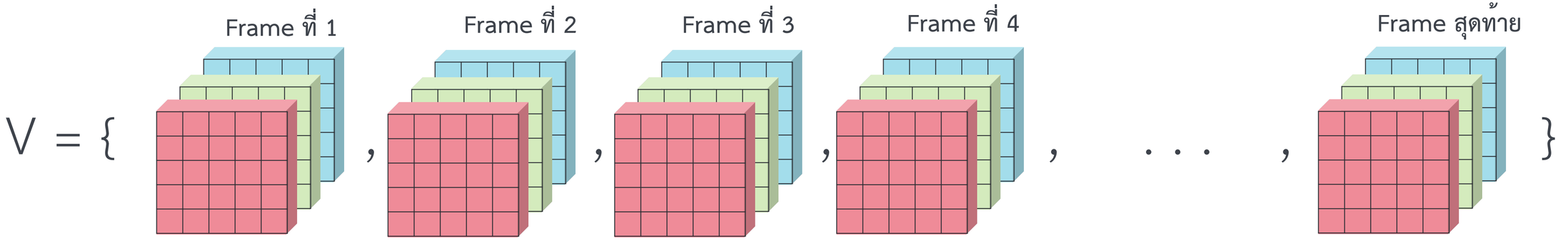


ประมาณอัตราการหายใจ
หน่วย: ครั้ง/นาที

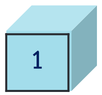


ประมาณมุมข้อต่อจากวิดีโอ
หน่วย: องศา

ข้อมูล Time Series สำหรับวิดีโอ

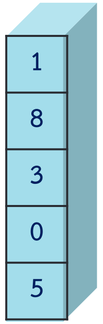


ซึ่งโครงสร้างข้อมูลในลักษณะข้างต้นมักนิยมเรียกว่า Tensor ที่มีดีกรี (Rank) อยู่ที่ 4 ซึ่งบางคนจะเรียกว่าข้อมูล 4 มิติ ก็ได้



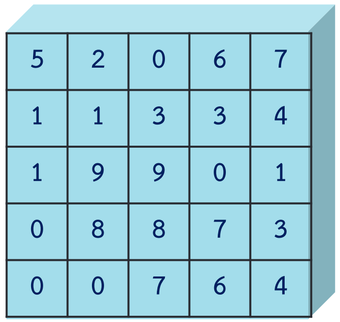
Tensor Rank 0

(Scalar)



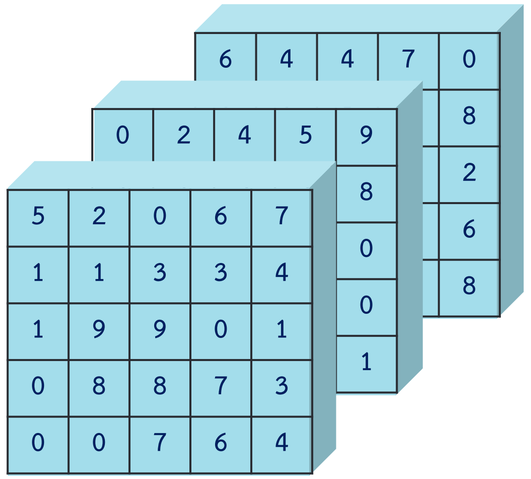
Tensor Rank 1

(Vector)



Tensor Rank 2

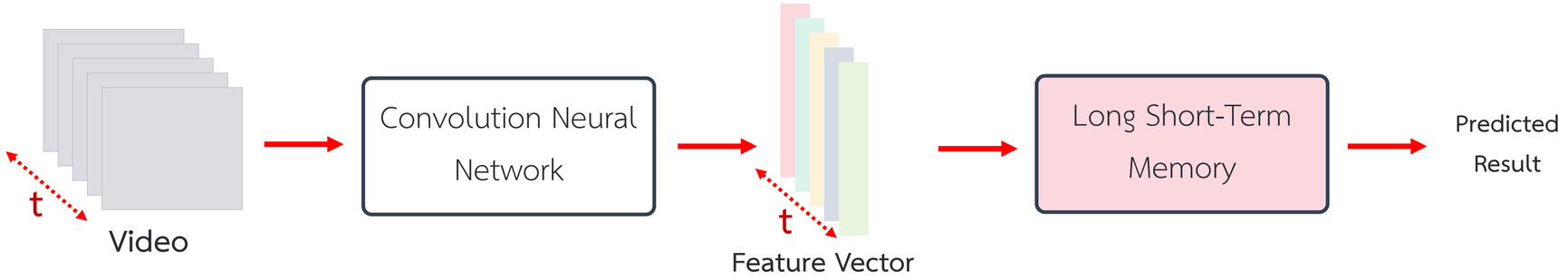
(Matrix)



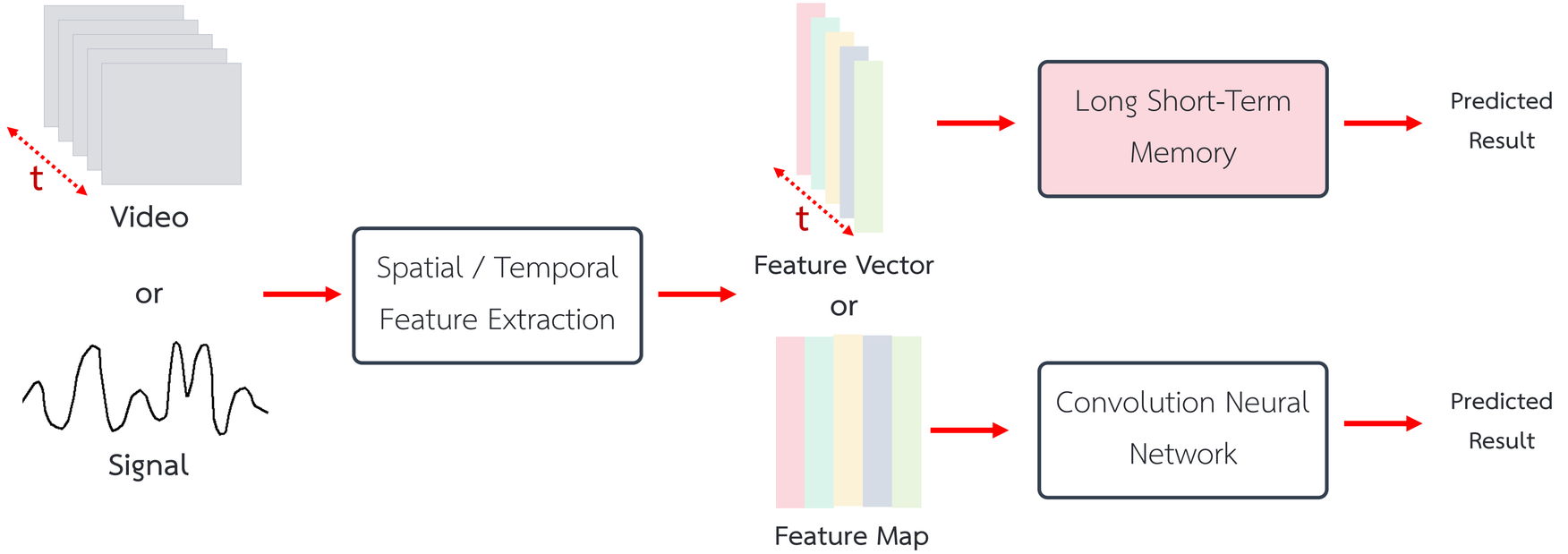
Tensor Rank 3 or higher (Tensor)

การใช้งาน Deep Learning สำหรับวีดิโอ

1



2

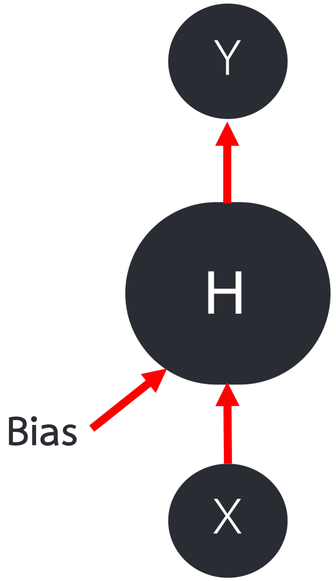




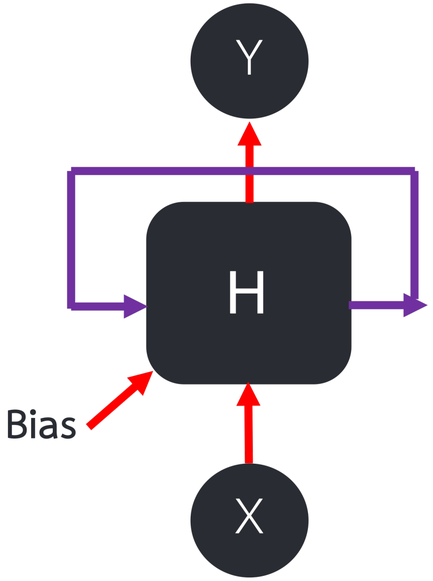
หัวข้อ

- ข้อมูลแบบอนุกรมเวลา (Time Series)
- **โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)**
- หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)
- หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น (Convolution Long Short-Term Memory: ConvLSTM)
- วิธีการสร้าง Convolution LSTM ด้วย Pytorch
- โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ

โครงข่ายประสาทแบบวนกลับ



(ก) สถาปัตยกรรม NN



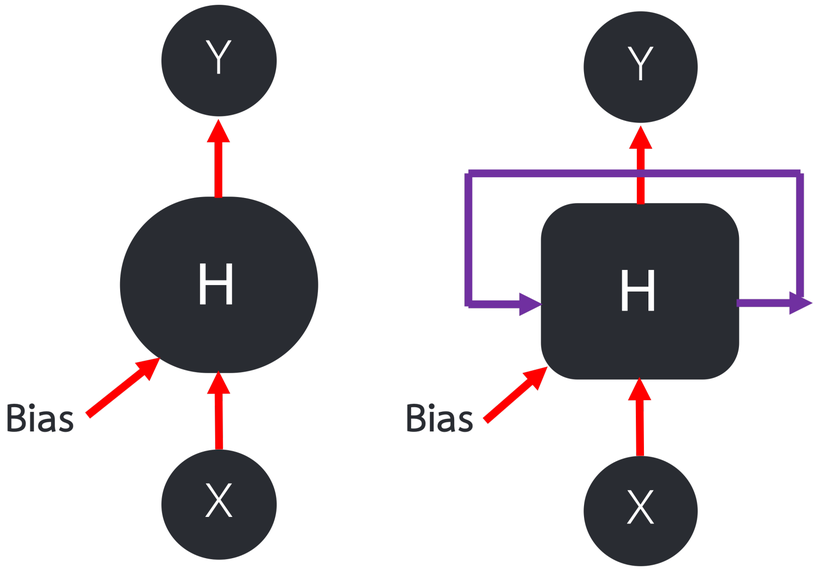
(ข) สถาปัตยกรรม RNN

สถาปัตยกรรม Neural Network (NN) และ Convolution Neural Network (CNN) ได้รับการออกแบบให้มีความเหมาะสมกับ

- อินพุตที่ใส่ค่าเข้ามาพร้อมกันทั้งหมด อาทิเช่น รับค่าพิกเซลเข้ามาทั้งภาพ หรือรับค่าของคุณลักษณะเข้ามาทั้งเวกเตอร์ เป็นต้น
- อินพุตต้องไม่มีความสัมพันธิเชิงลำดับ (Sequence) **ที่ส่งผลต่อการทำนายผลลัพธ์**
- อินพุตต้องมีขนาดที่เท่ากันและแน่นอน

ดังนั้น สถาปัตยกรรม NN และ CNN จึงไม่เหมาะสมกับข้อมูลที่มีความสัมพันธ์ในลำดับก่อนหลัง และไม่รองรับข้อมูลที่มีขนาดไม่เท่ากัน

โครงข่ายประสาทแบบวนกลับ



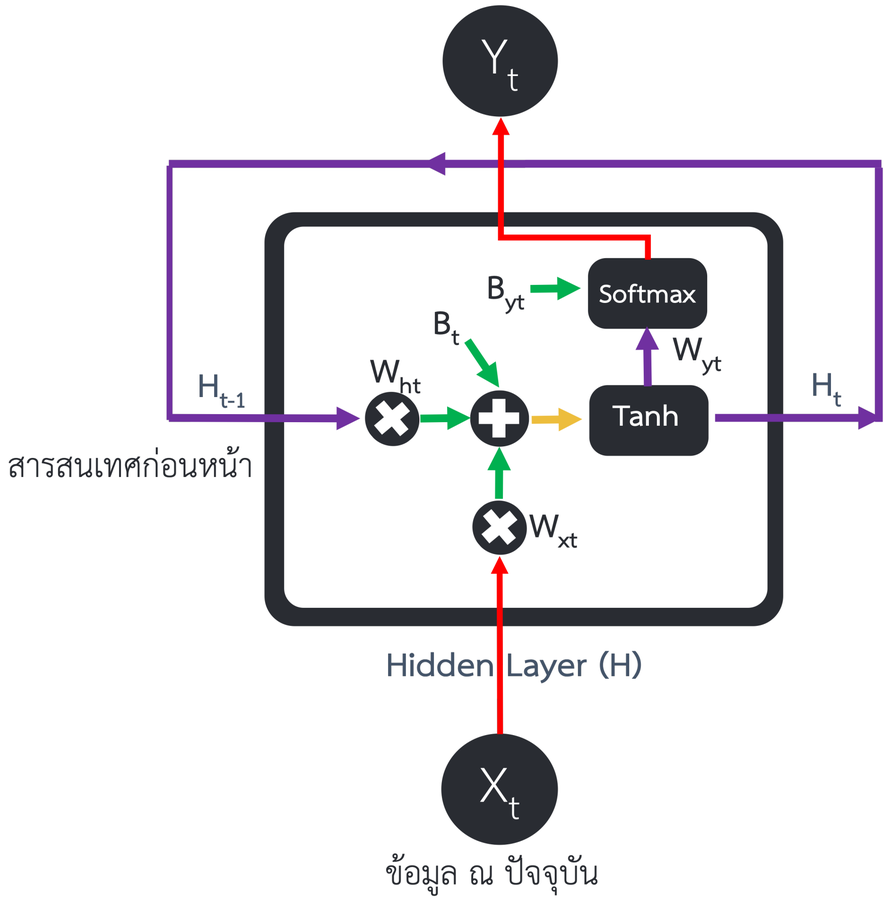
(ก) สถาปัตยกรรม NN

(ข) สถาปัตยกรรม RNN

เพื่อจะสามารถสร้างแบบจำลองที่รองรับข้อมูลที่*มีขนาดไม่เท่ากัน*และเหมาะสมกับ*ข้อมูลที่มีความสัมพันธ์เชิงลำดับ* (หรือเรียกว่า ข้อมูลอนุกรมเวลา) ดังนั้น จึงได้มีการนำเสนอสถาปัตยกรรมแบบโครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)

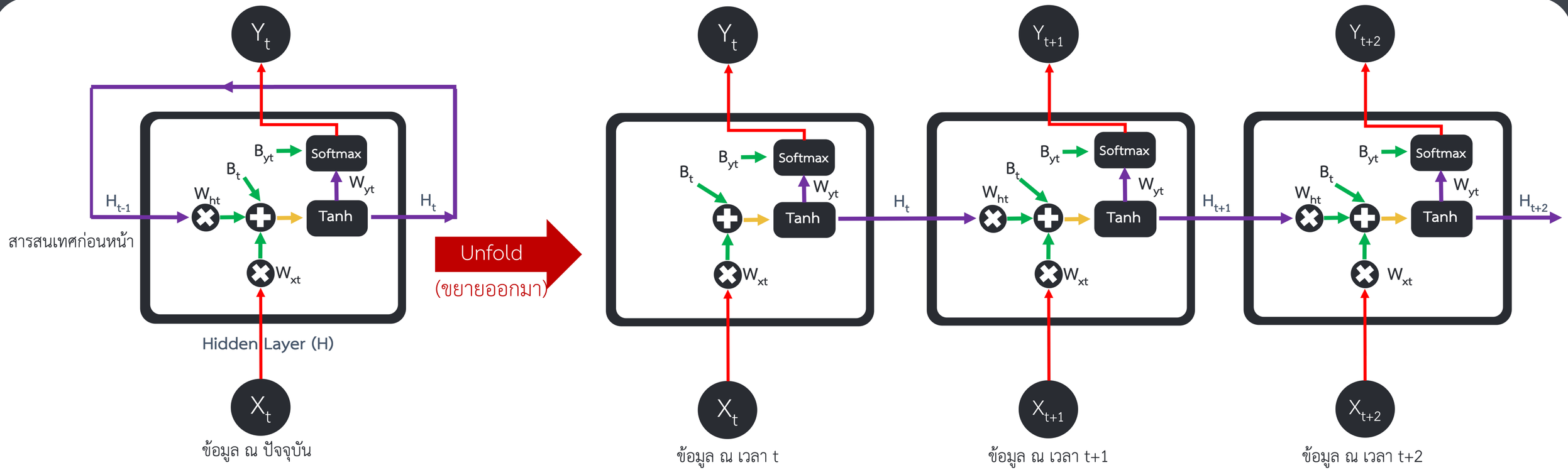
ข้อมูลที่มีความสัมพันธ์เชิงลำดับ (หรือ สถานะ/state/ความรู้ก่อนหน้า) มีความสำคัญต่อแบบจำลองเป็นอย่างมาก แต่จะพบว่าสถาปัตยกรรม NN ไม่มีการเก็บสารสนเทศก่อนหน้า (หรือ สถานะก่อนหน้า) เอาไว้ ขณะที่ สถาปัตยกรรม RNN มีการเก็บสารสนเทศของสารสนเทศก่อนหน้าไว้ (เส้นสีม่วง) และนำมาประมวลผลร่วมกับข้อมูลปัจจุบัน สิ่งเหล่านี้ทำให้ RNN เป็นสถาปัตยกรรมที่ได้ทำนายผลลัพธ์มาจากข้อมูล 2 แหล่ง ได้แก่ ข้อมูลปัจจุบันและสารสนเทศของสถานะก่อนหน้า

โครงข่ายประสาทแบบวนกลับ



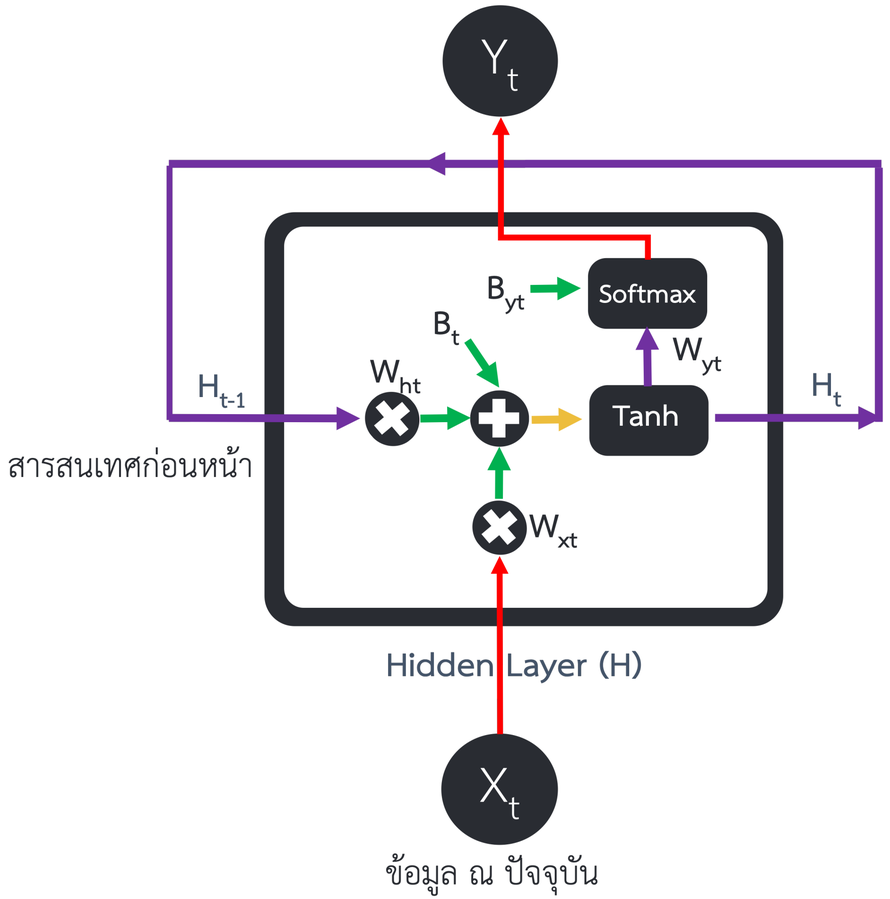
โครงข่ายประสาทแบบวนกลับเป็นสถาปัตยกรรมที่ได้รับการออกแบบมาเพื่อให้ใช้งานกับข้อมูลที่เป็นลำดับ (Sequence) ได้แก่ วีดิโอ (Video or Sequence of Images) หรือ ข้อความ (Text or Sequence of Words) เป็นต้น RNN มีลักษณะการทำงานคล้ายกับพฤติกรรมการอ่านหนังสือของมนุษย์ โดยที่ มนุษย์จะอ่านหนังสือทีละคำ เพื่อทำความเข้าใจเรื่องราวที่อ่าน ซึ่งมนุษย์สามารถเข้าใจเรื่องราวที่อ่านได้เนื่องจากมีข้อมูลของคำที่อ่านก่อนหน้ามาแล้วและผนวกกับคำที่เรากำลังอ่านอยู่ ณ ตำแหน่งปัจจุบัน สิ่งเหล่านี้ทำให้มนุษย์เข้าใจความหมายของสิ่งที่อ่านเข้าไป ดังนั้น RNN จึงได้รับการออกแบบให้มีสถาปัตยกรรมที่มีการพิจารณา (1) สารสนเทศก่อนหน้า (Previous Information) และ (2) ข้อมูล ณ ตำแหน่งปัจจุบัน เพื่อให้สามารถทำความเข้าใจข้อมูลและสิ่งที่ต้องการทำนายได้

โครงข่ายประสาทแบบวนกลับ



จากตัวอย่างข้างต้น นักศึกษาจะพบว่า Hidden Layer (หรือ เรียกได้ว่า RNN Cell) มีเพียงตัวเดียว แต่สามารถ Unfold ออกมาเพื่อให้เห็นการทำงานในแต่ละรอบได้ ซึ่งจากตัวอย่างได้แสดงให้เห็นการทำงาน 3 รอบ *“A recurrent neural network can be seen as the repeated use of a single RNN Cell.”*

โครงข่ายประสาทแบบวนกลับ



เมื่อพิจารณาภายใน RNN Cell (หรือ Hidden Layer: H) จะพบว่าคุณลักษณะหรือสารสนเทศที่สำคัญจะคำนวณมาจาก 2 ส่วน ได้แก่

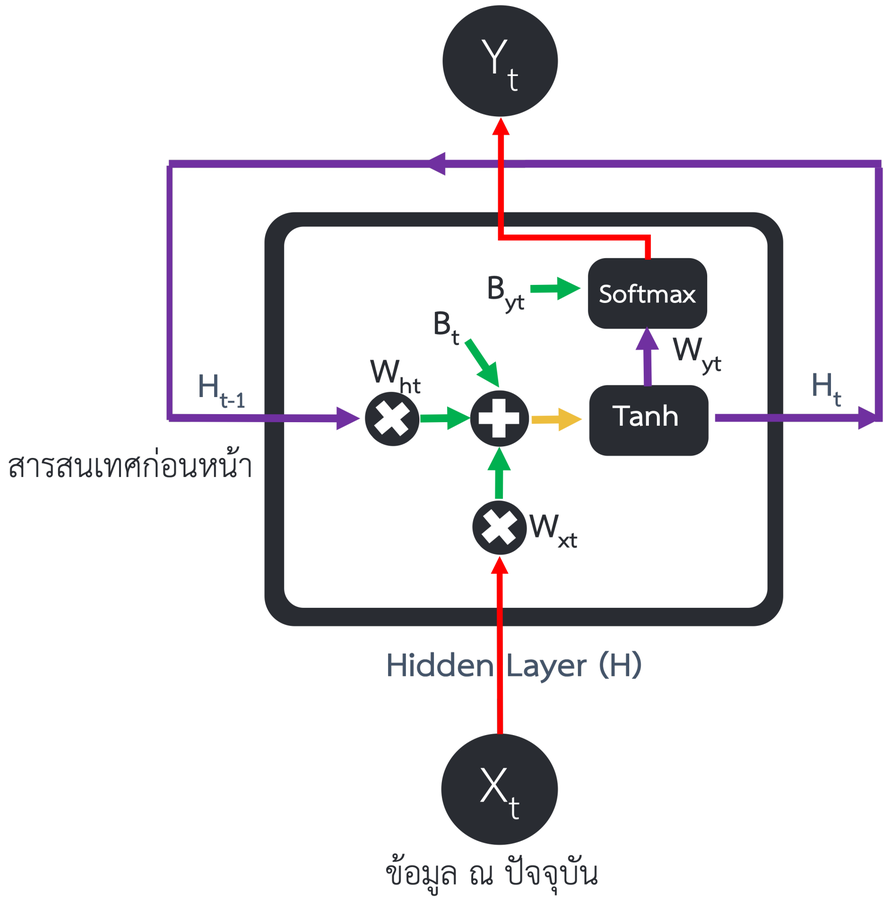
- ส่วนข้อมูล ณ เวลา ปัจจุบัน (X_t)
- ส่วนข้อมูลที่ได้รับการประมวลผลก่อนหน้า (H_{t-1})

โดยที่ X_t จะถูกควบคุมการไหลเข้าสู่ RNN Cell ด้วย W_{xt} ขณะที่ H_{t-1} จะถูกควบคุมการไหลเข้าสู่ RNN Cell ด้วย W_{ht} ซึ่งคุณลักษณะหรือสารสนเทศที่ได้รับจาก RNN Cell จะคำนวณได้จากสมการต่อไปนี้

ข้อมูลปัจจุบัน — สารสนเทศก่อนหน้า

$$H_t = \text{Tanh}(X_t \cdot W_{xt} + H_{t-1} \cdot W_{ht} + B_t)$$

โครงข่ายประสาทแบบวนกลับ

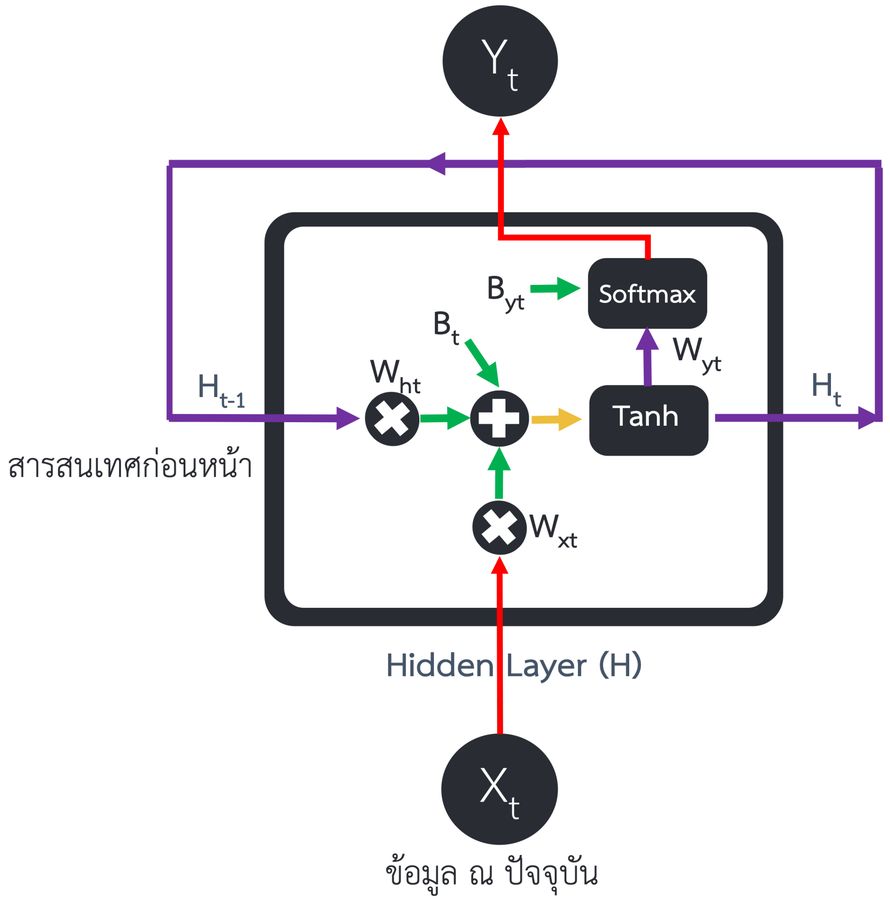


และสามารถคำนวณผลลัพธ์ของ RNN Cell ได้จากสมการต่อไปนี้

$$Y_t = \text{Softmax}(\ H_t \cdot W_{yt} + B_{yt}\ )$$

โดยที่ W_{yt} มีหน้าที่ควบคุมการไหลของสารสนเทศออกจาก RNN Cell ก่อนนำไปสร้างเป็นผลลัพธ์ของ RNN Cell

โครงข่ายประสาทแบบวนกลับ



สรุปแนวความคิดที่สำคัญ ดังนี้

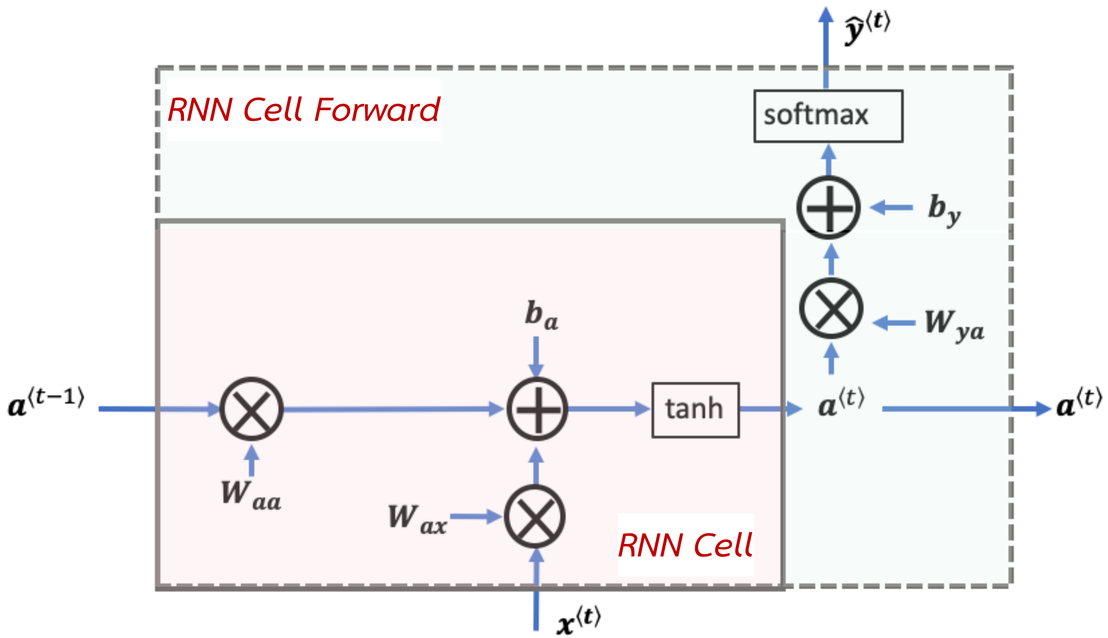
- ส่วนของสมการ H_t ของ RNN Cell ใช้สำหรับ***สกัดสารสนเทศหรือคุณลักษณะ***ที่สำคัญออกมาจากข้อมูลและสารสนเทศก่อนหน้า

$$H_t = \text{Tanh}(\ X_t \cdot W_{xt} + H_{t-1} \cdot W_{ht} + B_t\ )$$

- ส่วนของสมการ Y_t ของ RNN Cell ใช้สำหรับนำสารสนเทศหรือคุณลักษณะที่ได้มา***ตีความหรือนำไปคำนวณหาค่าผลลัพธ์ที่คาดหวัง***

$$Y_t = \text{Softmax}(\ H_t \cdot W_{yt} + B_{yt}\ )$$

โครงข่ายประสาทแบบวนกลับ

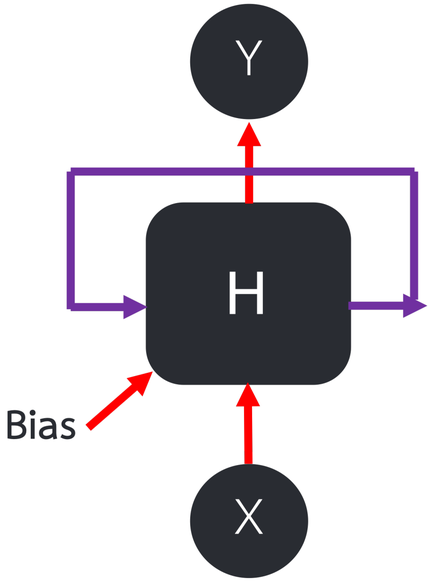


อ้างอิง https://durgaprasad-nagarkatte.github.io/implementing-forward-pass-for-rnn.html

อย่างไรก็ตาม บางครั้ง เราสามารถแบ่งภายใน Hidden Layer ของ RNN ได้เป็น 2 ส่วน ได้แก่ (1) ***RNN Cell*** และ (2) ***RNN Cell Forward*** ซึ่งนักศึกษาสามารถปรับเปลี่ยนส่วนของ RNN Cell Forward ได้ เนื่องจากส่วนดังกล่าวมีหน้าที่นำสารสนเทศหรือคุณลักษณะที่ได้มาตีความหรือนำไปคำนวณหาค่าผลลัพธ์ที่คาดหวัง ดังนั้น

- อาจจะปรับเปลี่ยนให้เกิดความเหมาะสมกับรูปแบบของผลลัพธ์ได้ตามความเหมาะสม
- บางครั้ง RNN Cell Forward อาจจะถูกมองว่าไม่ใช้ส่วนหนึ่งของ RNN Cell แต่เป็นเพียงส่วนต่อขยายเพื่อใช้สร้างผลลัพธ์ หมายความว่าถ้าไม่ต้องการสร้างผลลัพธ์ ก็ไม่จำเป็นต้องมีก็ได้

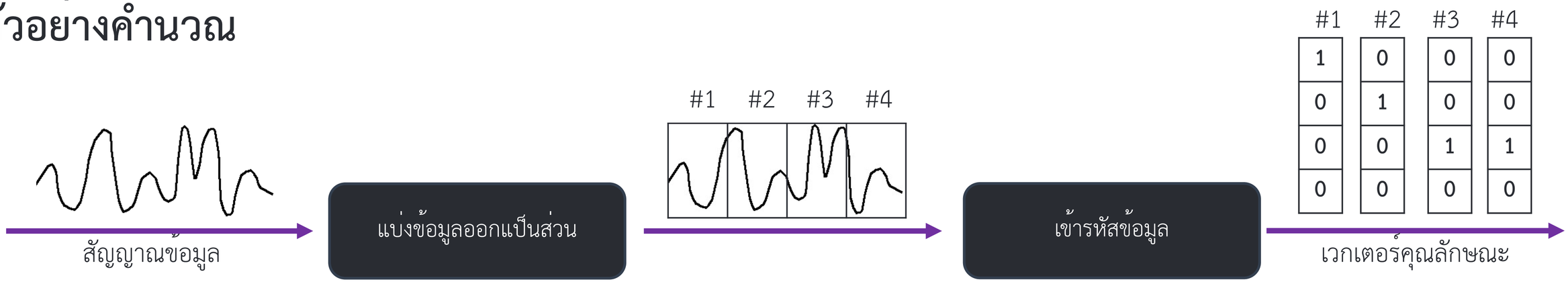
โครงข่ายประสาทแบบวนกลับ



อย่างไรก็ตาม สถาปัตยกรรมแบบ RNN ยิ่งเวลาผ่านไปและข้อมูลผ่านไปในแต่ละ RNN Cell หลายรอบเข้า สารสนเทศที่อยู่ใน RNN Cell ณ รอบแรก ๆ จะมีบทบาทลดลงไป ดังนั้น ถ้าข้อมูลอินพุตมีขนาดที่ยาวมากอาจส่งผลให้ RNN Cell ไม่สามารถรักษาสารสนเทศในรอบต้น ๆ ได้ ทำให้เกิดปัญหาตามมา

โครงข่ายประสาทแบบวนกลับ

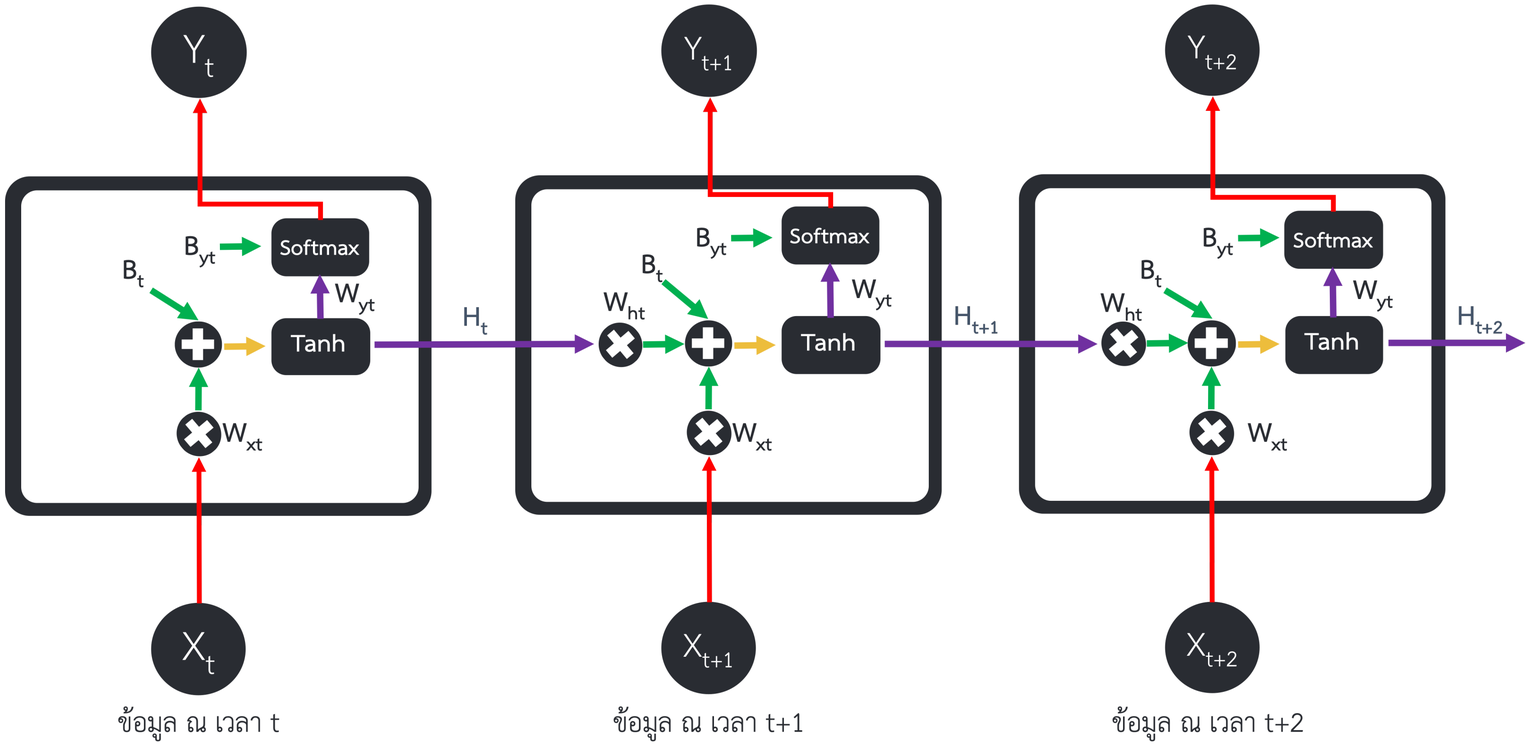
ตัวอย่างคำนวณ



ขั้นตอนที่ 1 การเตรียมและเข้ารหัสข้อมูล (Data Preparation and One-hot Encoder)

โครงข่ายประสาทแบบวนกลับ

ตัวอย่างคำนวณ



ขั้นตอนที่ 2 นำข้อมูลที่เข้ารหัสแล้วส่งต่อให้ RNN ประมวลผล



โครงข่ายประสาทแบบวนกลับ

ข้อดีและข้อเสียของ RNN

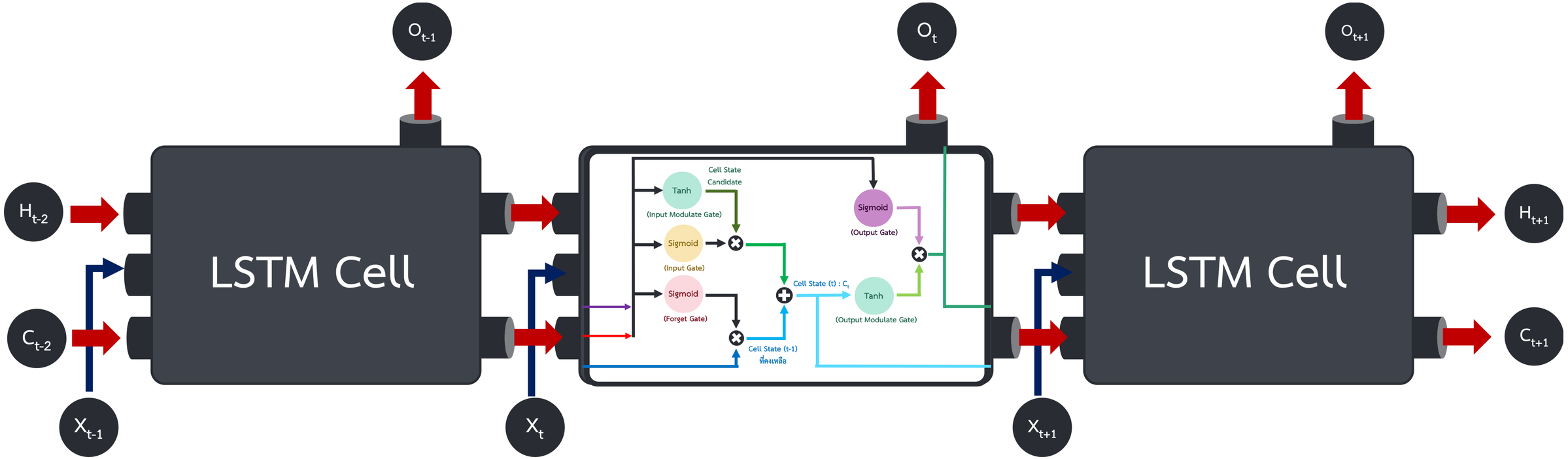
ข้อดี	ข้อเสีย
• ความสามารถในการประมวลผลอินพุตที่มีความยาวหรือขนาดแตกต่างกันได้ • ขนาดของโมเดลไม่เพิ่มขึ้นตามขนาดของอินพุต • การคำนวณจะพิจารณาข้อมูลก่อนหน้าที่ผ่านมา • การแชร์น้ำหนักข้ามช่วงเวลา	• การคำนวณช้า • ยากในการเข้าถึงข้อมูลจากเวลานานมาแล้ว • ไม่สามารถพิจารณาอินพุตในอนาคตสำหรับสถานะปัจจุบัน



หัวข้อ

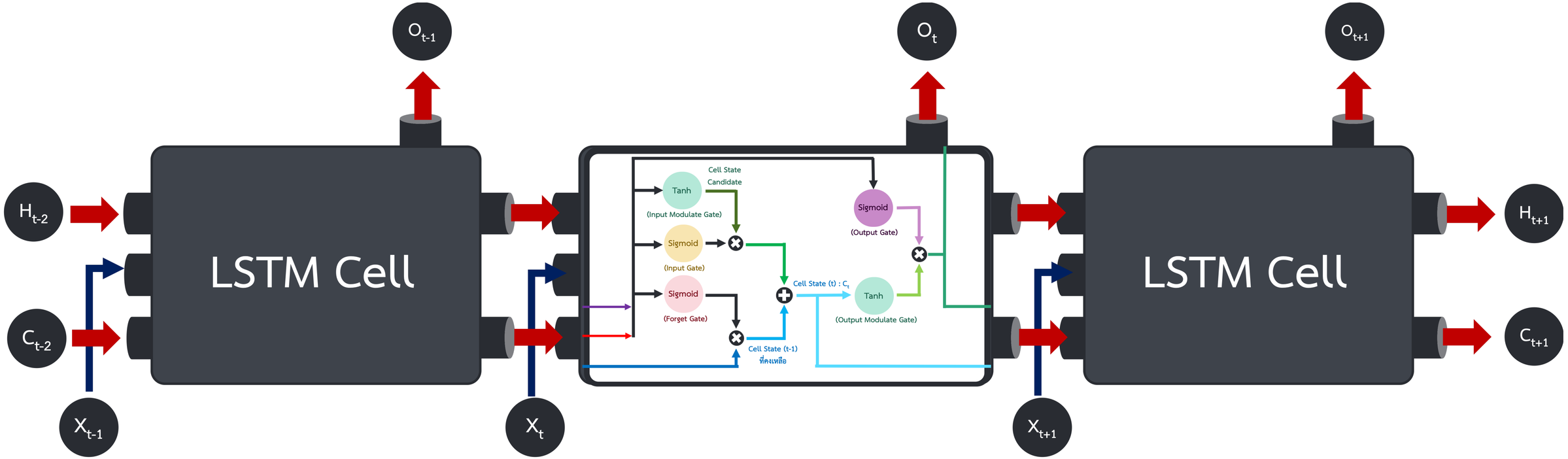
- ข้อมูลแบบอนุกรมเวลา (Time Series)
- โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)
- **หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)**
- หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น
  (Convolution Long Short-Term Memory: ConvLSTM)
- วิธีการสร้าง Convolution LSTM ด้วย Pytorch
- โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ

หน่วยความจำระยะสั้นแบบยาว

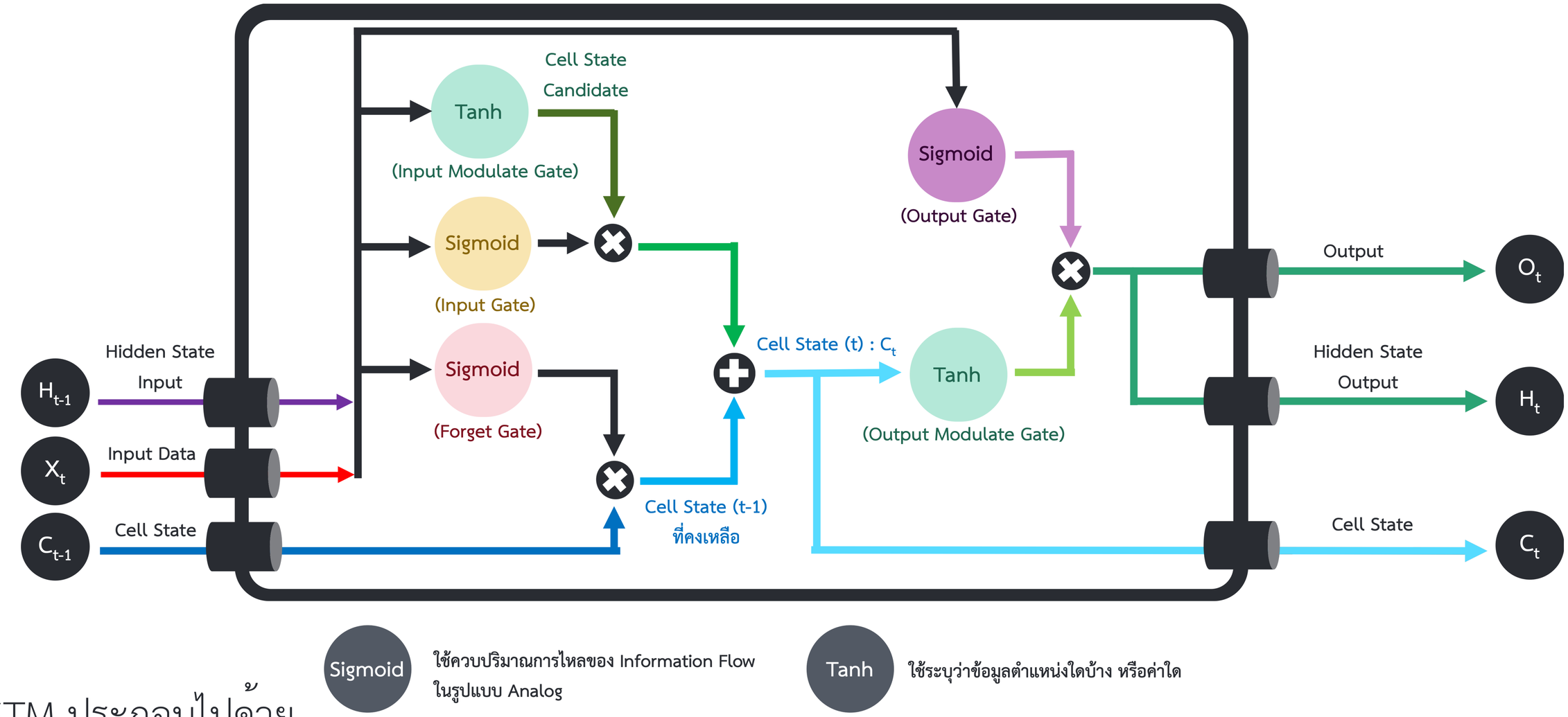


หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM) คือ โครงข่ายประสาทแบบวนกลับชนิดหนึ่งที่มีความสามารถเรียนรู้แบบการพึ่งพาแบบระยะยาว (long-term dependencies) ระหว่างข้อมูลในแต่ละลำดับเวลาได้ ซึ่งพัฒนาขึ้นโดย Hochreiter และคณะในปี 1997

หน่วยความจำระยะสั้นแบบยาว



LSTM ได้รับการออกแบบเพื่อหลีกเลี่ยงปัญหา “การพึ่งพาแบบระยะยาว (long-term dependency problem)” กล่าวคือ ในทางปฏิบัติแล้วการจดจำสารสนเทศเป็นระยะเวลานานไม่สามารถการันตีได้ว่าจะสามารถจดจำได้หมดหรือไม่ หรือจดจำได้ก็ไม่การันตีว่าจะสามารถส่งต่อไปครบถ้วนหรือไม่

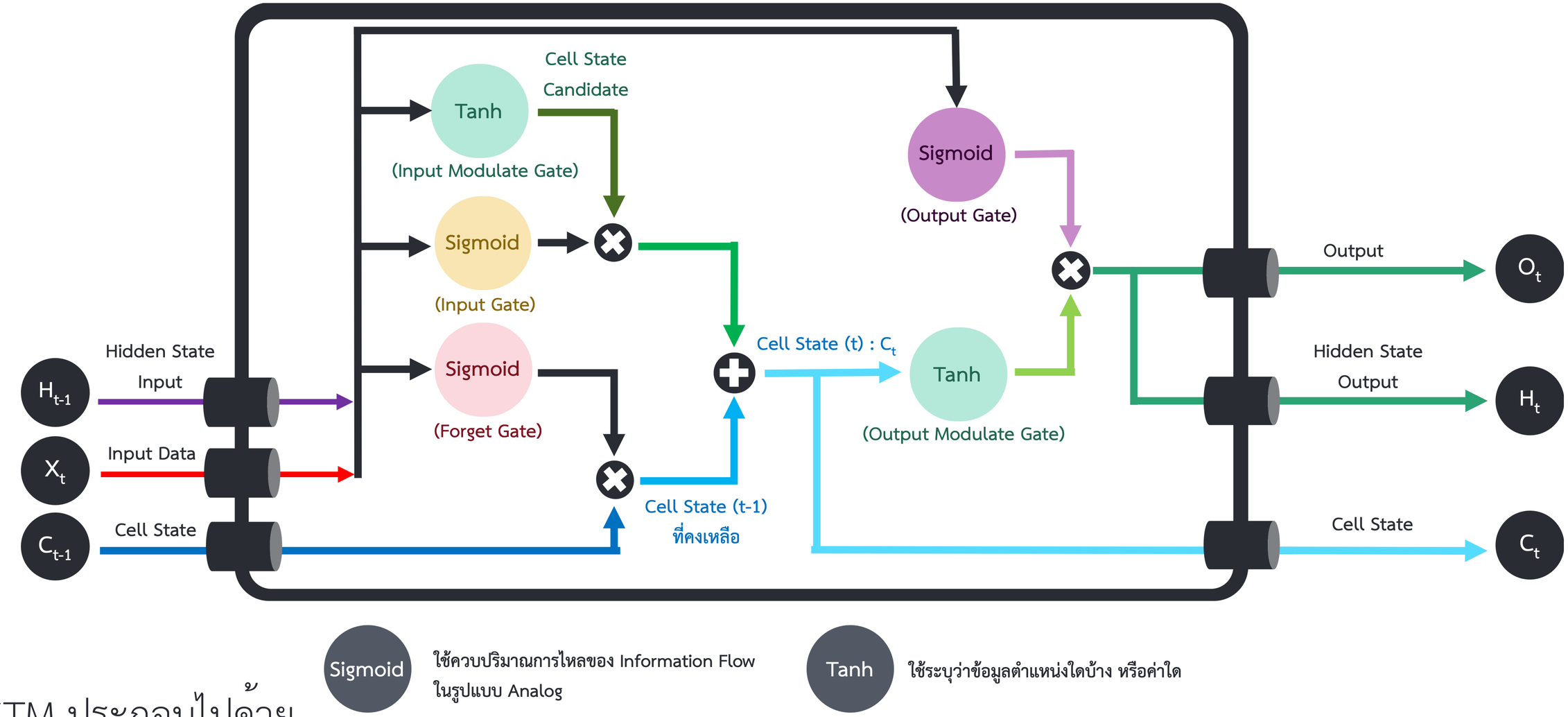


Sigmoid ใช้ควบปริมาณการไหลของ Information Flow ในรูปแบบ Analog

Tanh ใช้ระบุว่าข้อมูลตำแหน่งใดบ้าง หรือค่าใด

LSTM ประกอบไปด้วย

- 2 State ได้แก่ Cell State และ Hidden State
- 3 Gate ได้แก่ Input Gate, Forget or Remember Gate, และ Output Gate
- 2 Modulate ได้แก่ Input Modulate Gate และ Output Modulate Gate (ส่วนนี้ผมตั้งเอง**)



Sigmoid ใช้ควบปริมาณการไหลของ Information Flow ในรูปแบบ Analog

Tanh ใช้ระบุว่าข้อมูลตำแหน่งใดบ้าง หรือค่าใด

LSTM ประกอบไปด้วย

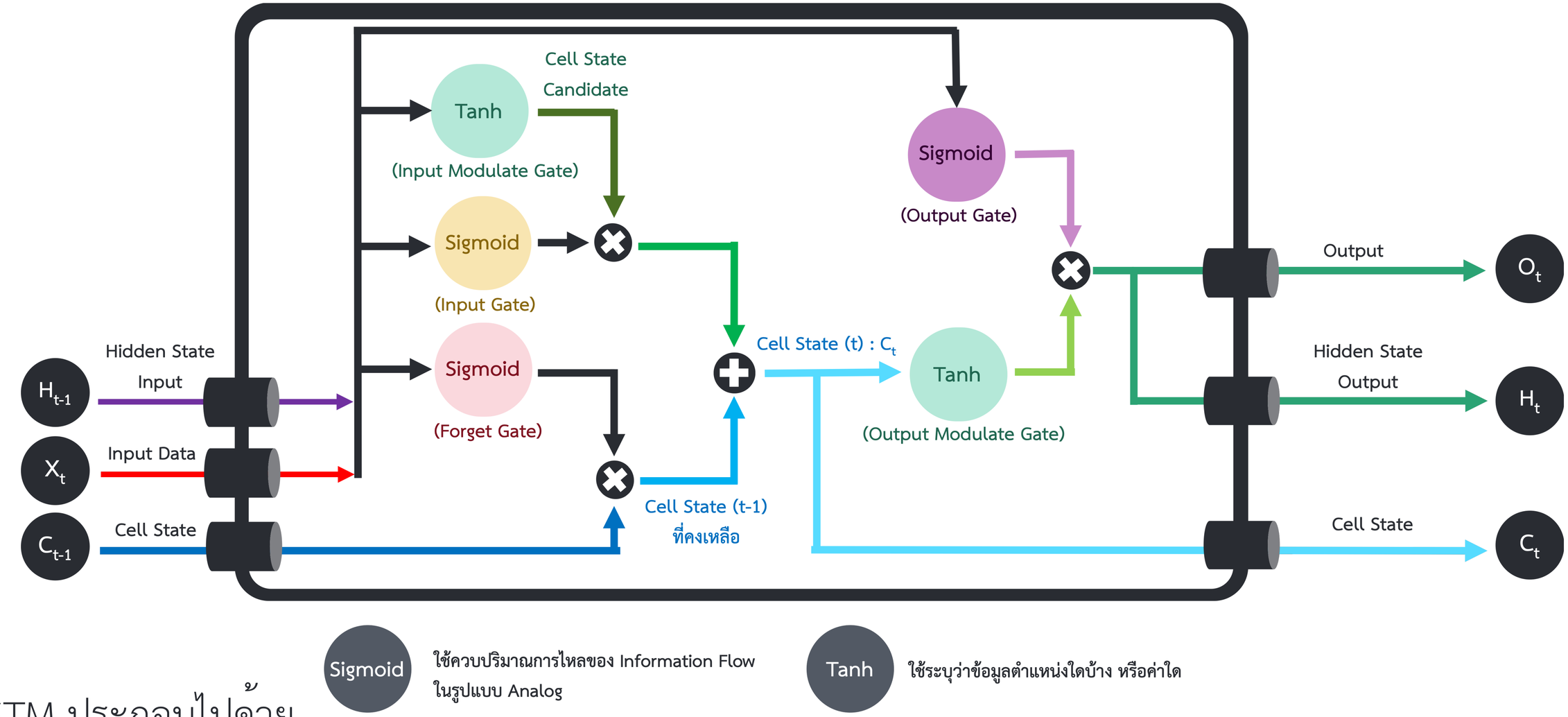
- 2 State ได้แก่ Cell State และ Hidden State
- 3 Gate ได้แก่ Input Gate, Forget or Remember Gate, และ Output Gate
- 2 Modulate ได้แก่ Input Modulate Gate และ Output Modulate Gate (ส่วนนี้ผมตั้งเอง**)



หน่วยความจำระยะสั้นแบบยาว

Cell State (Long Memory) คือ มีหน้าที่เข้ารหัสสารสนเทศ (ข้อมูลที่ได้รับการประมวลผลแล้ว ณ ตั้งแต่เริ่มจนถึงปัจจุบัน) ที่รวบรวมมาจากแต่ละเวลาก่อนหน้าทั้งหมด หรือสามารถกล่าวได้ว่า Cell State จะให้ความสนใจกับข้อมูลทั้งหมดมากกว่า

Hidden State (Short Memory) คือ มีหน้าที่เข้ารหัส “ลักษณะเฉพาะของข้อมูล” จากแต่ละเวลาก่อนหน้าทั้งหมด โดยที่ “ลักษณะเฉพาะของข้อมูล” เป็นนามธรรมที่ใช้เพื่อบ่งบอกถึงสิ่งที่ Hidden State เก็บไว้เพื่อใช้สำหรับทำนายต่อไป ซึ่งส่วนใหญ่สิ่งที่เก็บจะเกี่ยวข้องกับ state ณ เวลาล่าสุด แต่อย่างไรก็ตาม ผลลัพธ์ที่ได้รับ Hidden State ไม่ใช้ output หรือผลของการทำนายโดยตรง ซึ่งก็เหมือน RNN Cell Forward หรือสามารถกล่าวได้ว่า ผลลัพธ์ที่ได้รับ Hidden State รองรับการนำไปประมวลผลต่อเพื่อรับข้อมูลที่มีความหมายมากขึ้นหรือทำนายผลลัพธ์ก็ได้



Sigmoid ใช้ควบปริมาณการไหลของ Information Flow ในรูปแบบ Analog

Tanh ใช้ระบุว่าข้อมูลตำแหน่งใดบ้าง หรือค่าใด

LSTM ประกอบไปด้วย

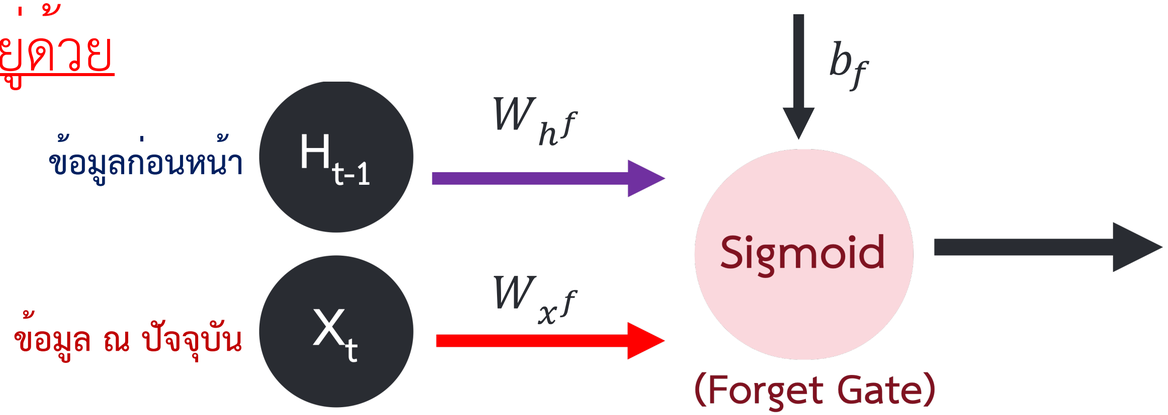
- 2 State ได้แก่ Cell State และ Hidden State
- 3 Gate ได้แก่ Input Gate, Forget or Remember Gate, และ Output Gate
- 2 Modulate ได้แก่ Input Modulate Gate และ Output Modulate Gate (ส่วนนี้ผมตั้งเอง**)

Forget or Remember Gate (Delete) คือ มีหน้าควบคุมการไหลของข้อมูลใน Cell State ก่อนหน้ามา Cell State ปัจจุบัน ซึ่งเปรียบเทียบได้กับการกำหนดว่า Cell State สามารถจำหรือลืมข้อมูลอะไรไปบ้าง โดยค่าของ Forget Gate จะอยู่ในช่วง 0 ถึง 1 โดยที่ 0 หมายความว่า ข้อมูลใน Cell State จะถูกลบหมด แต่ขณะที่ 1 หมายความว่าข้อมูลจะอยู่ครบทั้งหมด ค่าของ Forget Gate จากสมการต่อไปนี้

$$f_t = f_{sigmoid}\left(W_{xf}x_t + W_{hf}h_{t-1} + b_f\right)$$

ข้อมูล ณ ปัจจุบัน — ข้อมูลก่อนหน้า

โดยที่ ค่าของ Forget Gate จะคำนวณมาจาก (1) ค่าของ Hidden Node ณ State ก่อนหน้า (2) ค่า Input ณ State ปัจจุบัน และ (3) ค่า Bias ของ Forget Gate ซึ่งจะพบว่า ค่า (1) และ ค่า (2) จะมีค่าถ่วงน้ำหนักเฉพาะ ณ Forget Gate กำกับอยู่ด้วย



ซึ่ง Sigmoid Function จะรับหน้าที่เป็นตัวนำข้อมูลข้างต้นมาตัดสินใจในรูปแบบ Analog ที่ให้ผลลัพธ์อยู่ในช่วง 0 ถึง 1

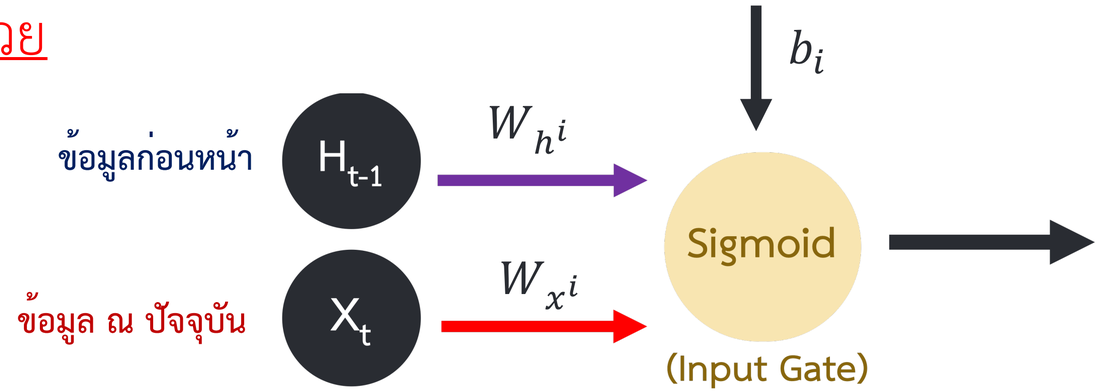
Input Gate (Write) คือ มีหน้าที่ควบคุมปริมาณการไหลของข้อมูลใหม่เข้าสู่ Cell State ปัจจุบัน ซึ่งเปรียบเทียบได้กับการกำหนดว่า Cell State จะอัพเดตข้อมูลปัจจุบันหรือไม่นั้นเอง โดยค่าของ Input Gate จะอยู่ในช่วง 0 ถึง 1 โดยที่ 0 หมายความว่า ข้อมูลใหม่จะไม่ไหลเข้าสู่ Cell State เลย แต่ขณะที่ 1 หมายความว่าข้อมูลใหม่จะไหลเข้าสู่ Cell State ทั้งหมด

$$i_t = f_{sigmoid}\left(W_{x^i}x_t + W_{h^i}h_{t-1} + b_i\right)$$

ข้อมูล ณ ปัจจุบัน

ข้อมูลก่อนหน้า

โดยที่ ค่าของ Input Gate จะคำนวณมาจาก (1) ค่าของ Hidden Node ณ State ก่อนหน้า (2) ค่า Input ณ State ปัจจุบัน และ (3) ค่า Bias ของ Input Gate ซึ่งจะพบว่า ค่า (1) และ ค่า (2) จะมีค่าถ่วงน้ำหนักเฉพาะ ณ Input Gate กำกับอยู่ด้วย



ซึ่ง Sigmoid Function จะรับหน้าที่เป็นตัวนำข้อมูลข้างต้นมาตัดสินใจในรูปแบบ Analog ที่ให้ผลลัพธ์อยู่ในช่วง 0 ถึง 1

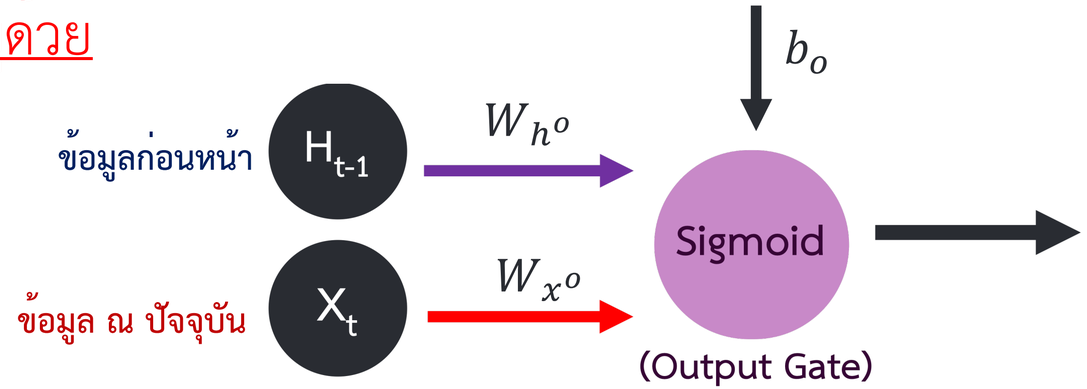
Output Gate (Read) คือ มีหน้าที่กำหนดการไหลของข้อมูลใน Cell State ปัจจุบัน ออกให้คนนอกนำไปใช้งานได้ ที่เรียกว่า Read เพราะว่า ข้อมูลทั้งหมดใน Cell State จะเปิดเผยให้ผู้อื่นมาอ่านไปใช้งานได้มากน้อยเพียงใดก็ขึ้นอยู่กับ Output Gate เป็นตัวตัดสินใจ โดยค่าของ Output Gate จะอยู่ในช่วง 0 ถึง 1 โดยที่ 0 หมายความว่าจะไม่เปิดเผยข้อมูลใน Cell State ต่อภายนอกเลย แต่ขณะที่ 1 หมายความว่าเปิดเผยข้อมูลใน Cell State ทั้งหมด

ข้อมูลก่อนหน้า

$$o_t = f_{sigmoid}(W_{x^o} x_t + W_{h^o} h_{t-1} + b_o)$$

ข้อมูล ณ ปัจจุบัน

โดยที่ ค่าของ Output Gate จะคำนวณมาจาก (1) ค่าของ Hidden Node ณ State ก่อนหน้า (2) ค่า Input ณ State ปัจจุบัน และ (3) ค่า Bias ของ Output Gate ซึ่งจะพบว่า ค่า (1) และ ค่า (2) จะมีค่าถ่วงน้ำหนักเฉพาะ ณ Output Gate กำกับอยู่ด้วย



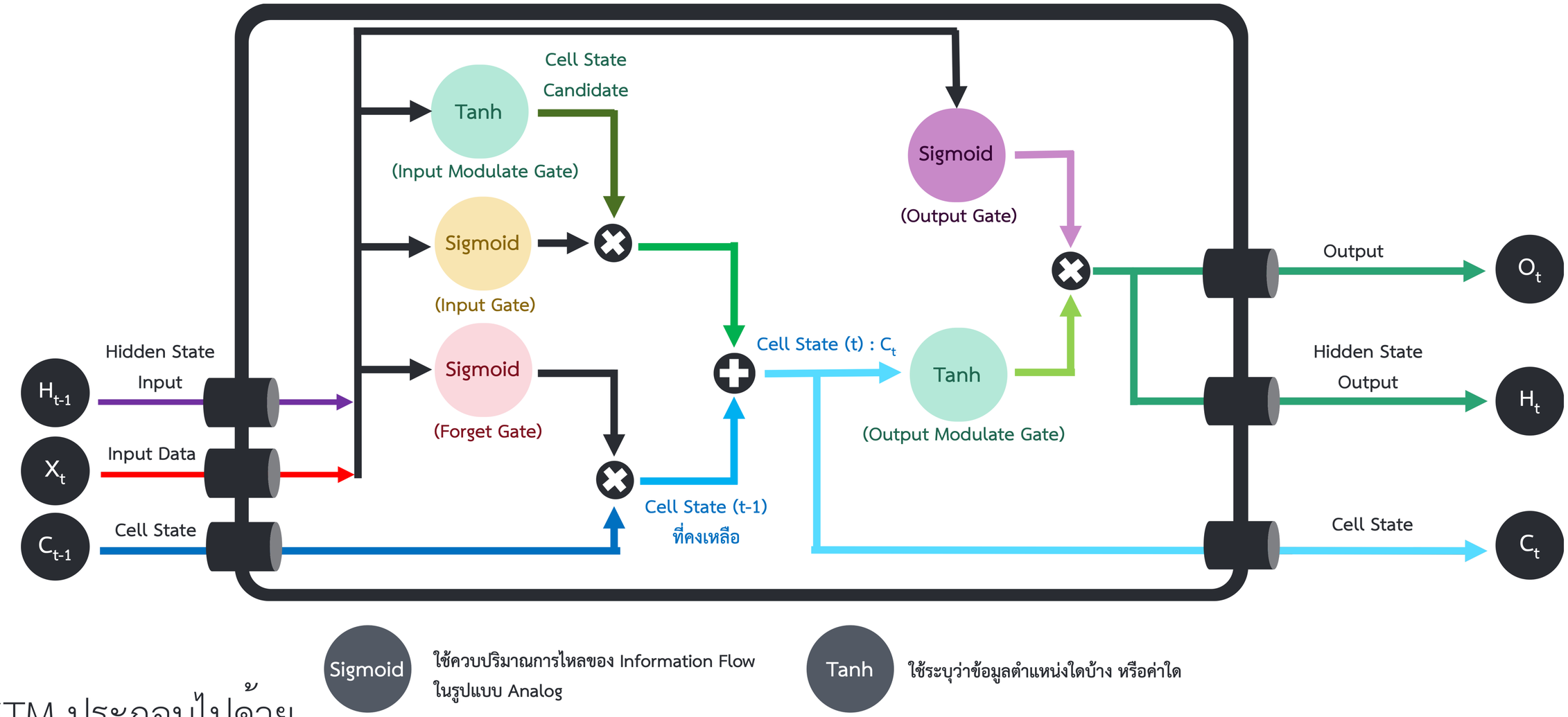
ซึ่ง Sigmoid Function จะรับหน้าที่เป็นตัวนำข้อมูลข้างต้นมาตัดสินใจในรูปแบบ Analog ที่ให้ผลลัพธ์อยู่ในช่วง 0 ถึง 1



หน่วยความจำระยะสั้นแบบยาว

Forget or Remember Gate (Delete) คือ มีหน้าควบคุมการไหลของข้อมูลใน Cell State ก่อนหน้ามา Cell State ปัจจุบันว่า ซึ่งเปรียบเทียบได้กับการกำหนดว่า Cell State สามารถจำหรือลืมข้อมูลอะไรไปบ้าง โดยค่าของ Forget Gate จะอยู่ในช่วง 0 ถึง 1 โดยที่ 0 หมายความว่า ข้อมูลใน Cell State จะถูกลบหมด แต่ขณะที่ 1 หมายความว่าข้อมูลจะอยู่ครบทั้งหมด

Input Gate (Write) คือ มีหน้าที่เข้ารหัส “ลักษณะเฉพาะของข้อมูล” จากแต่ละเวลาก่อนหน้าทั้งหมด โดยที่ “ลักษณะเฉพาะของข้อมูล” เป็นนามธรรมที่ใช้เพื่อบ่งบอกถึงสิ่งที่ Hidden State เก็บไว้เพื่อใช้ในการทำนายต่อไป ซึ่งส่วนใหญ่สิ่งที่เก็บจะเกี่ยวข้องกับ state ณ เวลาล่าสุด



Sigmoid ใช้ควบปริมาณการไหลของ Information Flow ในรูปแบบ Analog

Tanh ใช้ระบุว่าข้อมูลตำแหน่งใดบ้าง หรือค่าใด

LSTM ประกอบไปด้วย

- 2 State ได้แก่ Cell State และ Hidden State
- 3 Gate ได้แก่ Input Gate, Forget or Remember Gate, และ Output Gate
- 2 Modulate ได้แก่ Input Modulate Gate และ Output Modulate Gate (ส่วนนี้ผมตั้งเอง**)

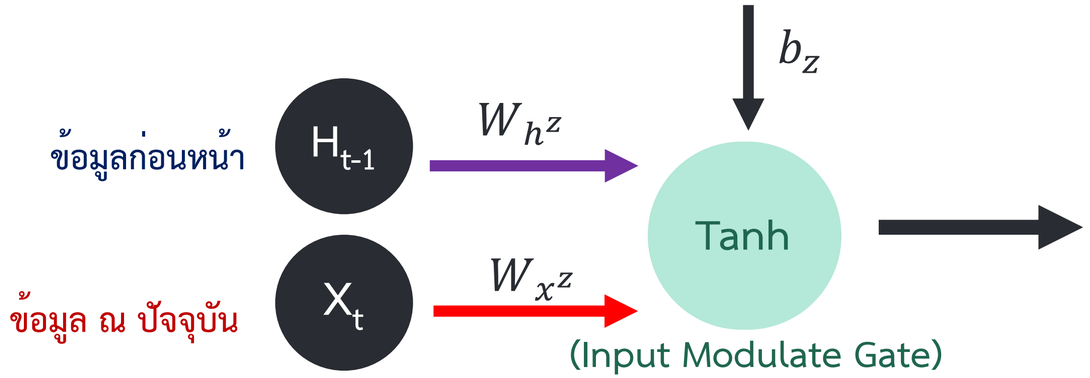
Input Modulate Gate คือ มีหน้าใช้ในการปรับปรุงและควบคุมข้อมูลใหม่ที่ต้องการเพิ่มเข้าไปใน cell state (สถานะของเซลล์) หรือหน่วยความจำของ LSTM ในแต่ละ timestep (จังหวะเวลา) โดยอาศัย input (ข้อมูลที่ป้อนเข้ามา) และ hidden state (สถานะที่ซ่อนอยู่) จาก timestep ก่อนหน้า มาผ่าน Tanh function เพื่อคำนวณค่าที่จะใช้ปรับปรุง cell state

ข้อมูลก่อนหน้า

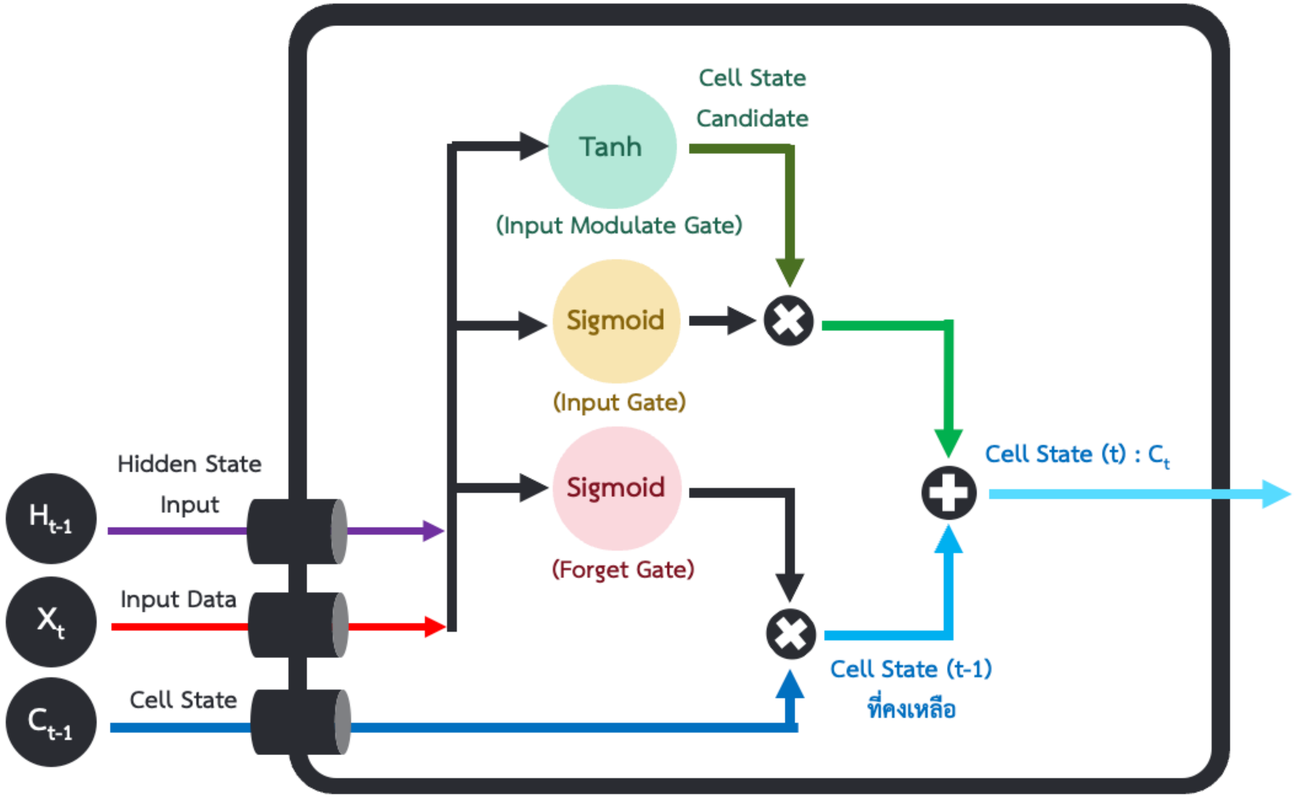
$$z_t = f_{tanh}(W_{x^z}x_t + W_{h^z}h_{t-1} + b_z)$$

ข้อมูล ณ ปัจจุบัน

โดยอาศัย input (ข้อมูลที่ป้อนเข้ามา) และ hidden state (สถานะที่ซ่อนอยู่) จาก timestep ก่อนหน้า มาผ่าน tanh activation function และ (3) ค่า Bias ของ Input Modulate Gate เพื่อคำนวณค่าที่จะใช้ปรับปรุง cell state ซึ่งจะพบว่า ค่า (1) - (3) จะมีค่าถ่วงน้ำหนักเฉพาะ ณ Input Modulate Gate กำกับอยู่ด้วย



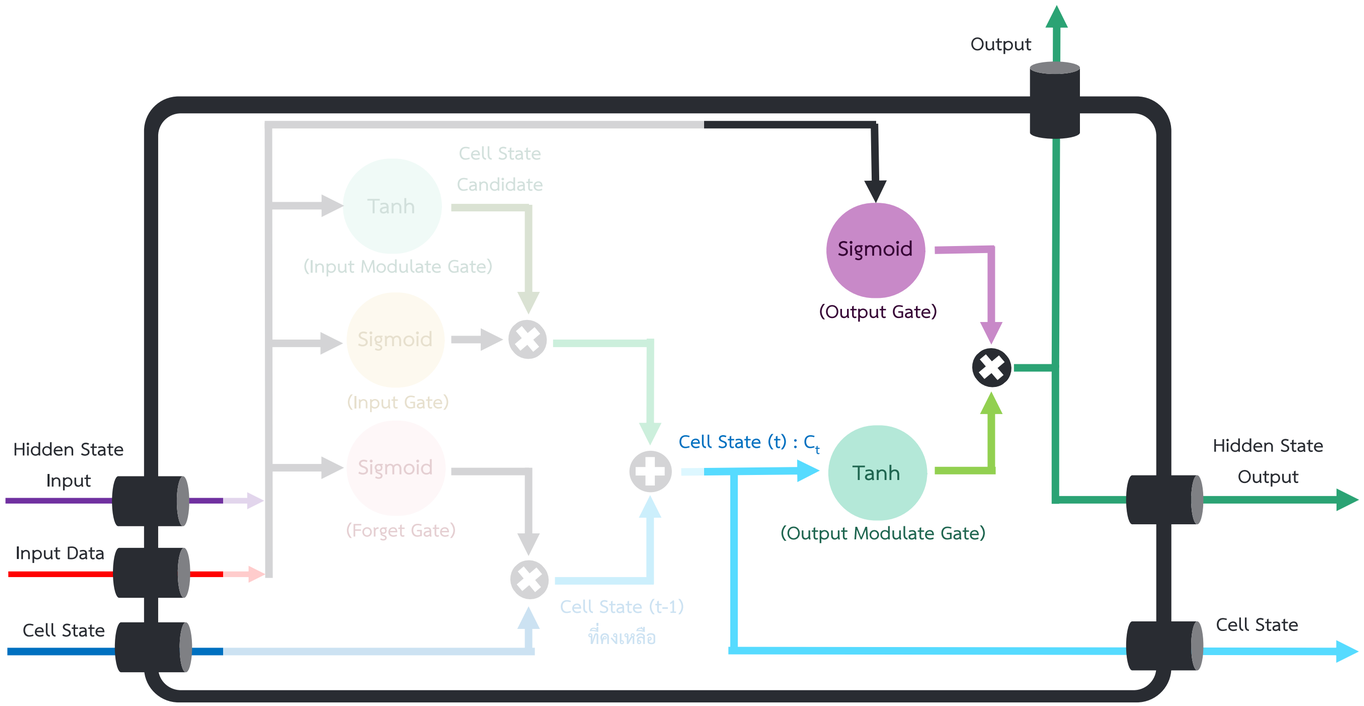
ซึ่ง Tanh function จะรับหน้าที่เป็นตัวนำข้อมูลข้างต้นมาตัดสินใจในรูปแบบ Analog ที่ให้ผลลัพธ์อยู่ในช่วง -1 ถึง 1



จากนั้น ดำเนินการปรับปรุงและควบคุมข้อมูลใหม่ที่ต้องการเพิ่มเข้าไปใน cell state (สถานะของเซลล์) ณ ปัจจุบัน โดยผ่านกระบวนการ Combination with Input Gate กล่าวคือนำค่า z_t มาคูณกับค่า i_t มาคำนวณ เพื่อควบคุมปริมาณข้อมูลที่เพิ่มเข้าไปใน cell state

$$C_t = f_t C_{t-1} + i_t * z_t$$

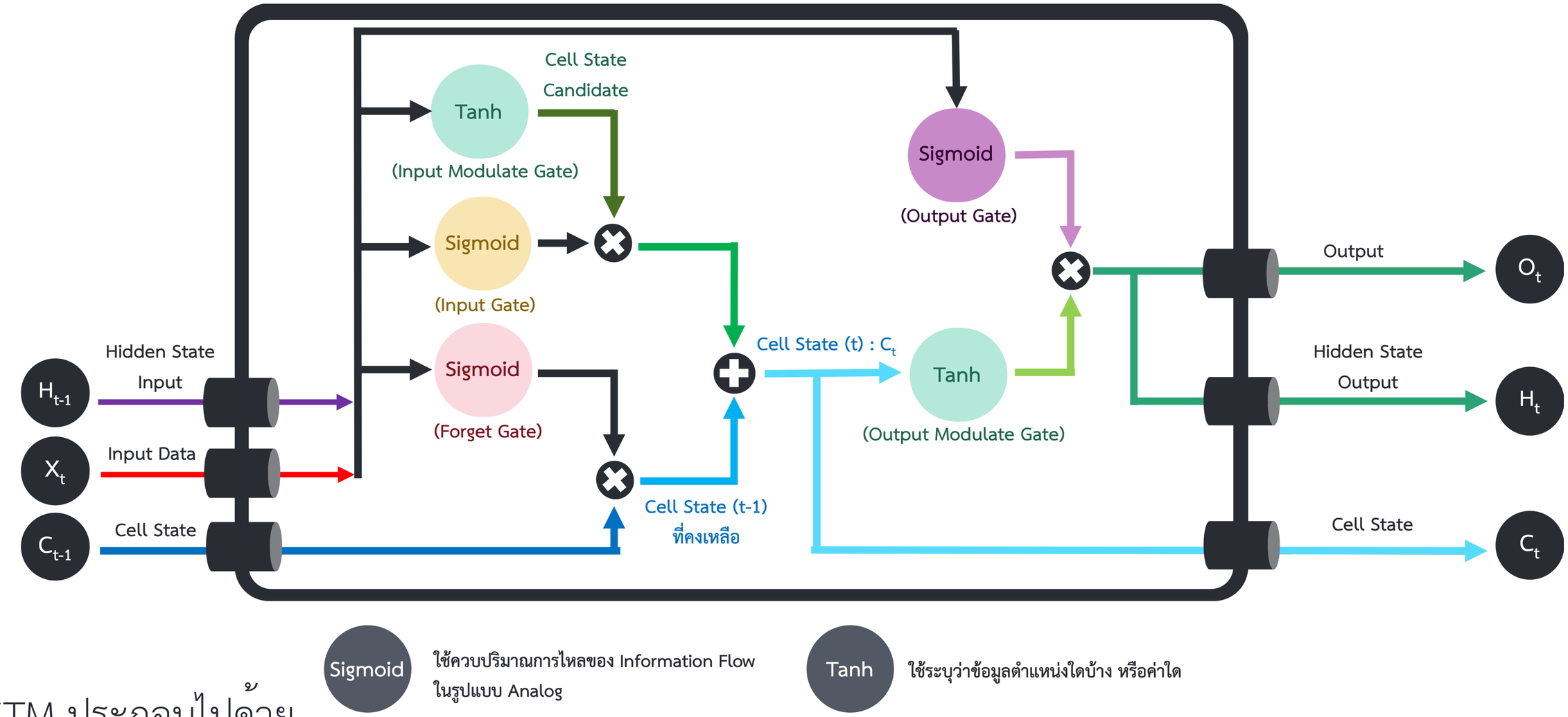
ข้อมูลก่อนหน้า ข้อมูลที่เพิ่ม



Output Modulate Gate คือ มีหน้าปรับปรุง Hidden State โดยที่ค่า h_t คำนวณจากค่า o_t คูณกับค่า C_t ที่มาผ่าน Tanh function

$$h_t = o_t * f_{tanh}(C_t)$$

Output Modulate Gate ใน LSTM มีบทบาทในการควบคุมข้อมูลที่ถูกส่งออกจาก cell state เพื่อใช้ในการคำนวณ hidden state ซึ่งจะถูกส่งต่อไปยัง timestep ถัดไป การทำงานของ Output Gate ทำให้ LSTM สามารถเลือกได้ว่าข้อมูลใดจาก cell state จะถูกนำไปใช้ในการคำนวณและส่งต่อไปยัง hidden state ถัดไป ทำให้สามารถปรับปรุงการจัดการข้อมูลในลำดับเวลาได้ดีขึ้น



Sigmoid ใช้ควบปริมาณการไหลของ Information Flow ในรูปแบบ Analog

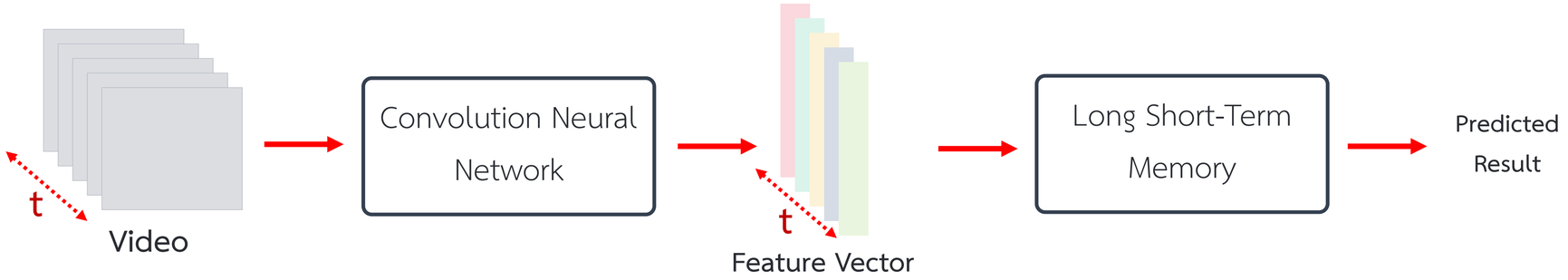
Tanh ใช้ระบุว่าข้อมูลตำแหน่งใดบ้าง หรือค่าใด

LSTM ประกอบไปด้วย

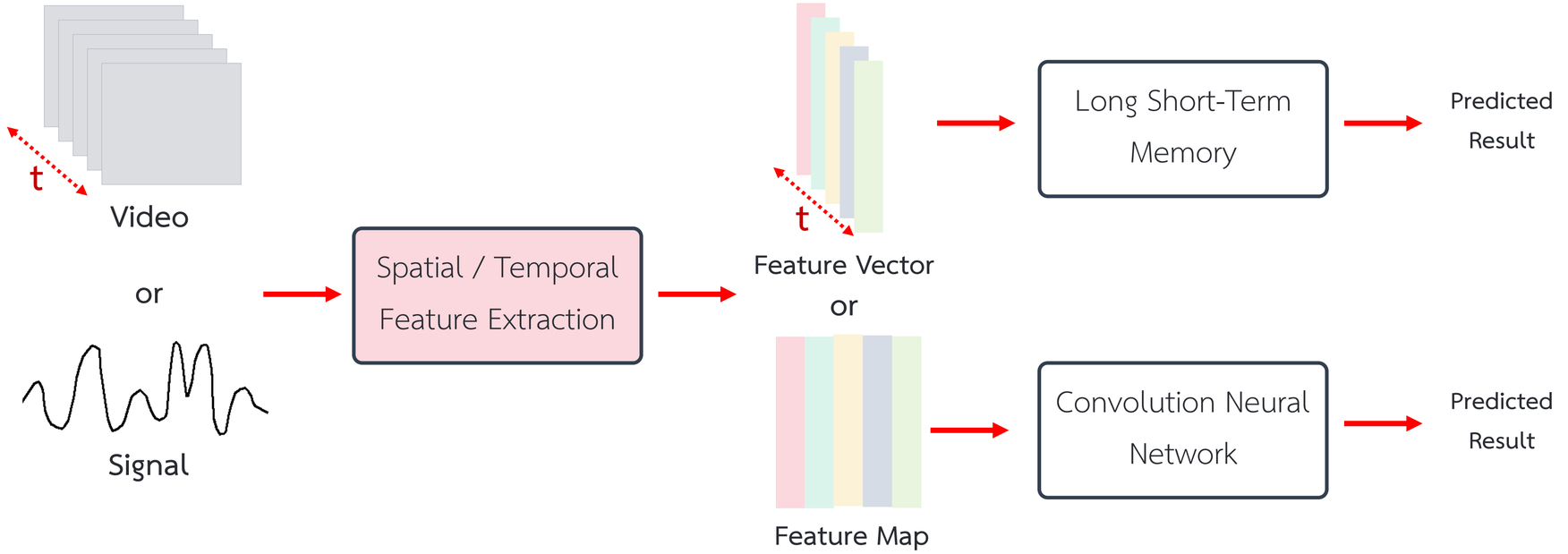
- 2 State ได้แก่ Cell State และ Hidden State
- 3 Gate ได้แก่ Input Gate, Forget or Remember Gate, และ Output Gate
- 2 Modulate ได้แก่ Input Modulate Gate และ Output Modulate Gate (ส่วนนี้ผมตั้งเอง**)

การใช้งาน Deep Learning สำหรับวีดิโอ

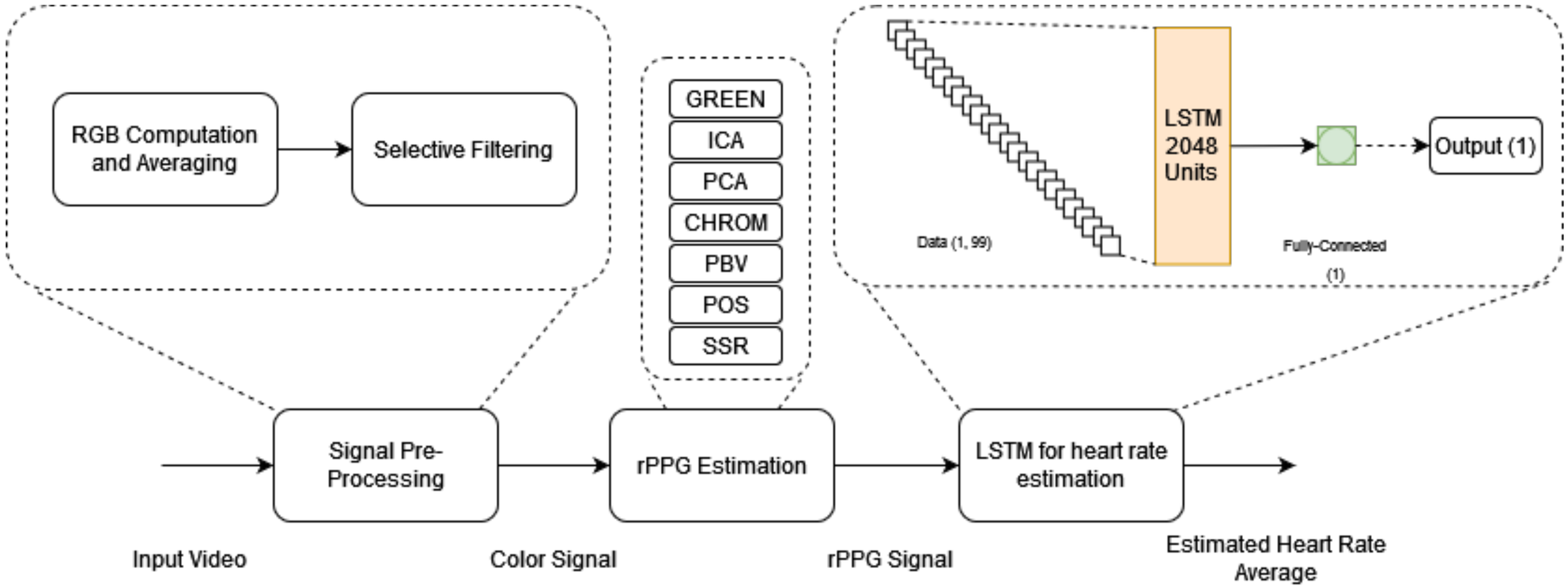
1



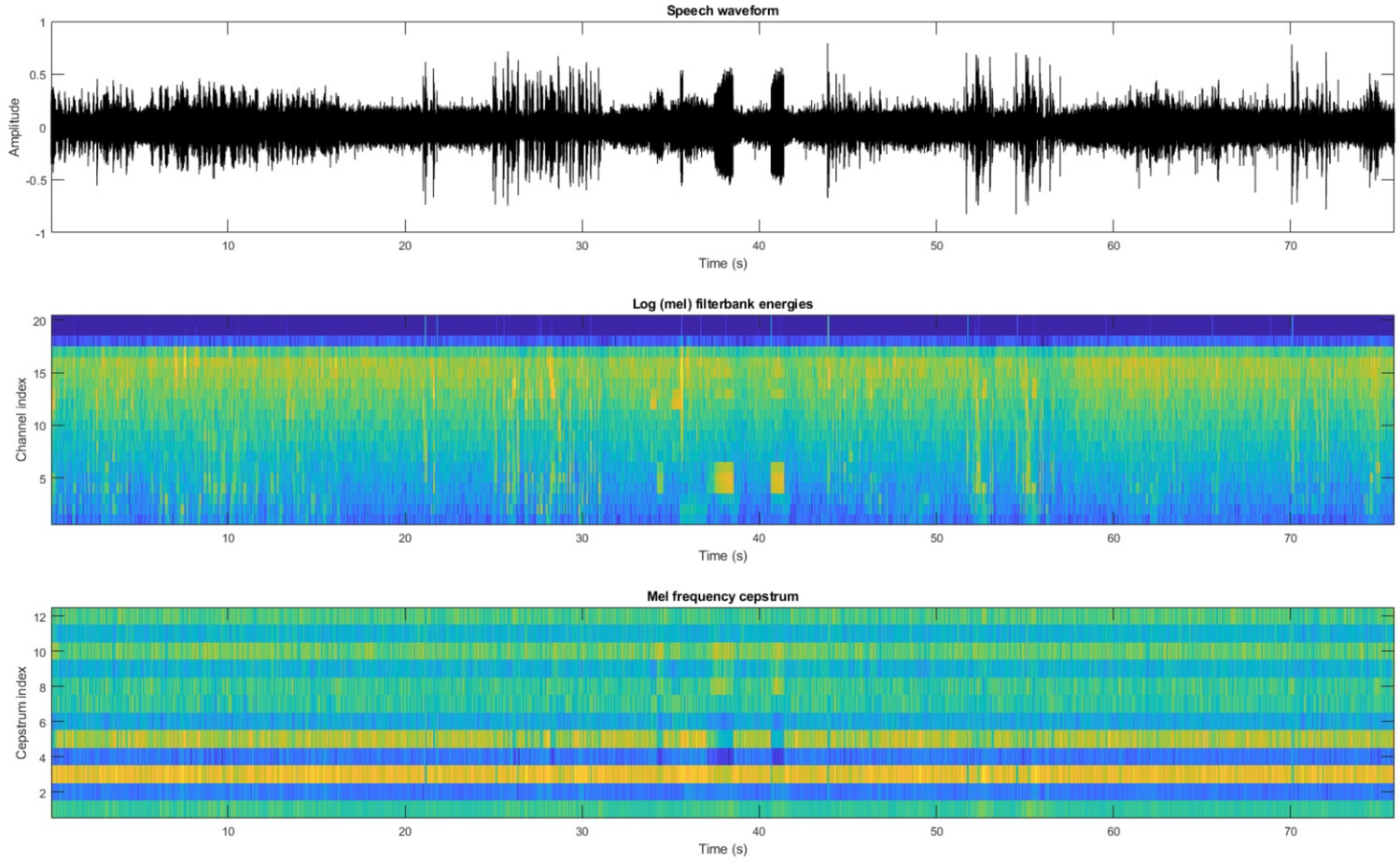
2



แนวทางการประยุกต์ใช้งาน

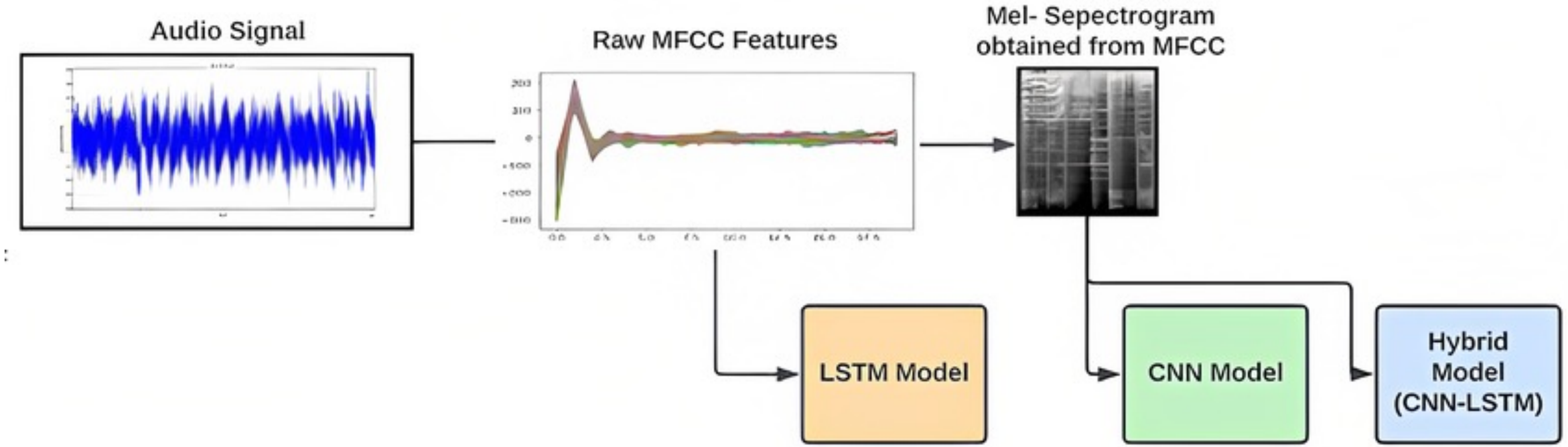


แนวทางการประยุกต์ใช้งาน



อ้างอิง https://stackoverflow.com/questions/56758148/how-to-train-a-neural-network-lstm-with-mfcc-coefficients-in-matlab

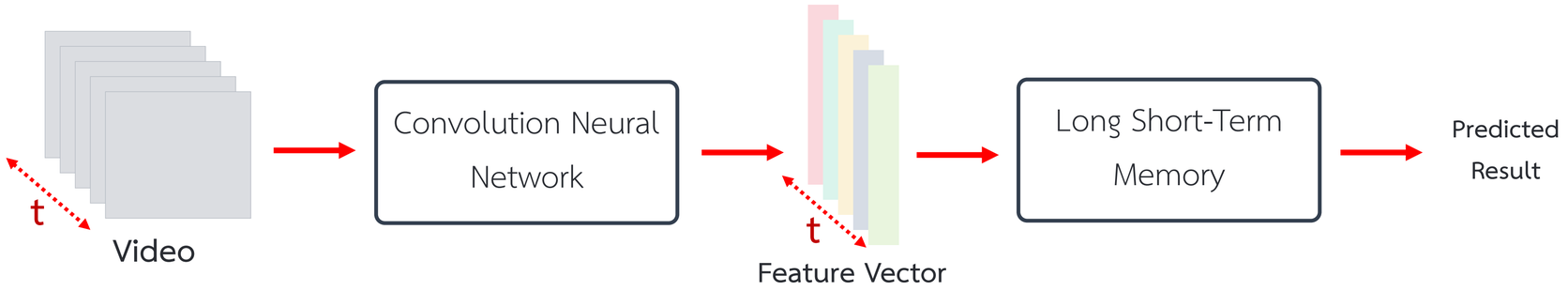
แนวทางการประยุกต์ใช้งาน



อ้างอิง DOI: 10.3390/technologies11040091

การใช้งาน Deep Learning สำหรับวีดิโอ

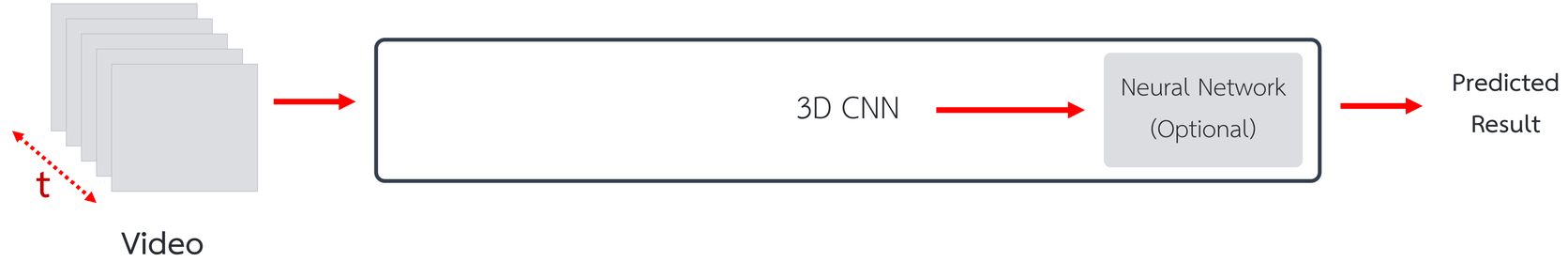
1



3

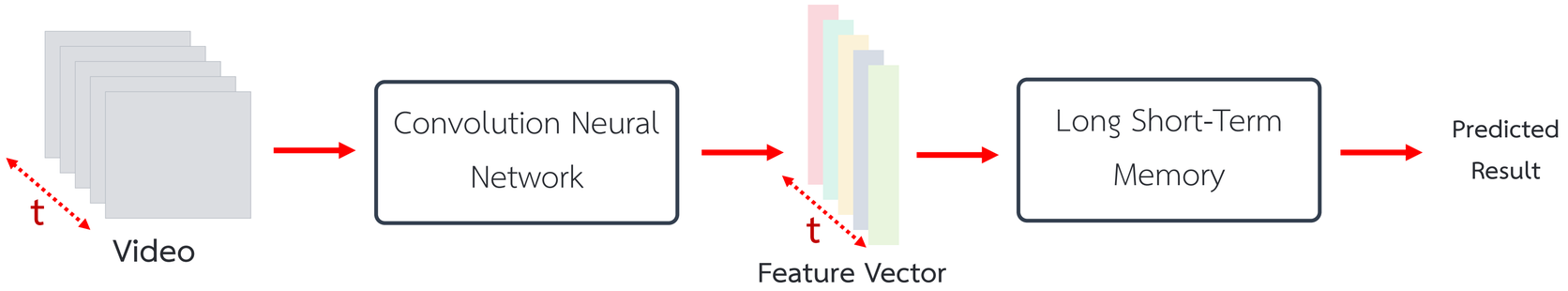


4

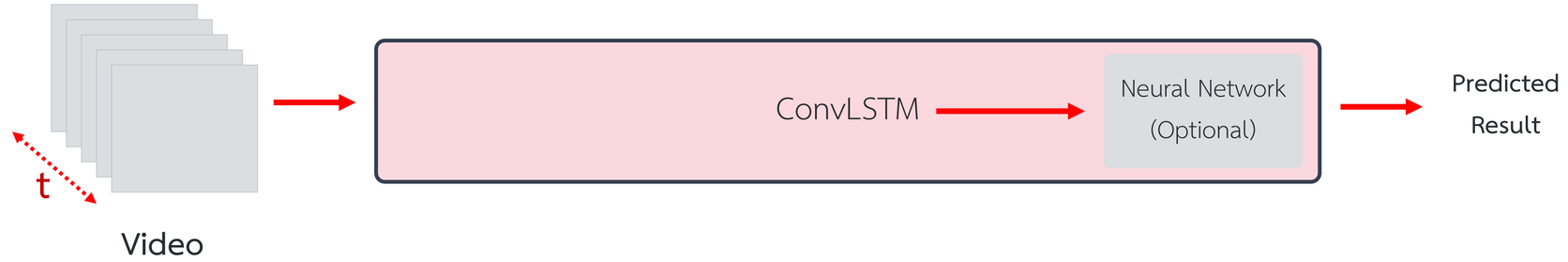


การใช้งาน Deep Learning สำหรับวีดิโอ

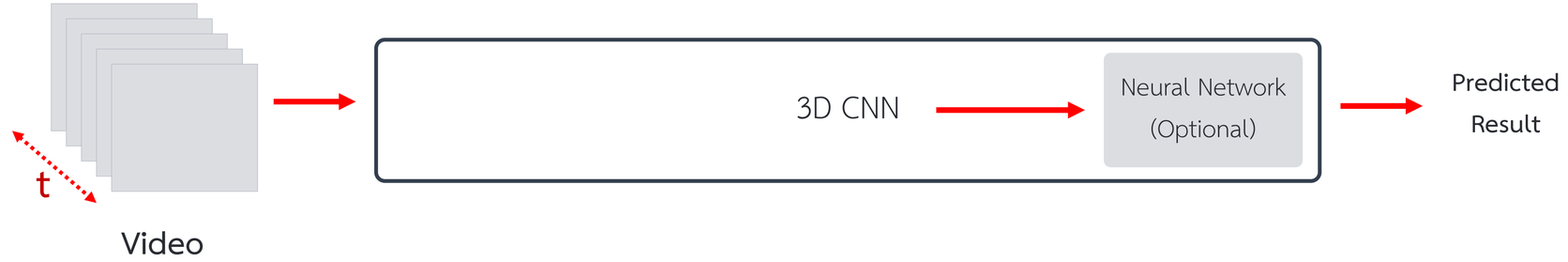
1



3



4

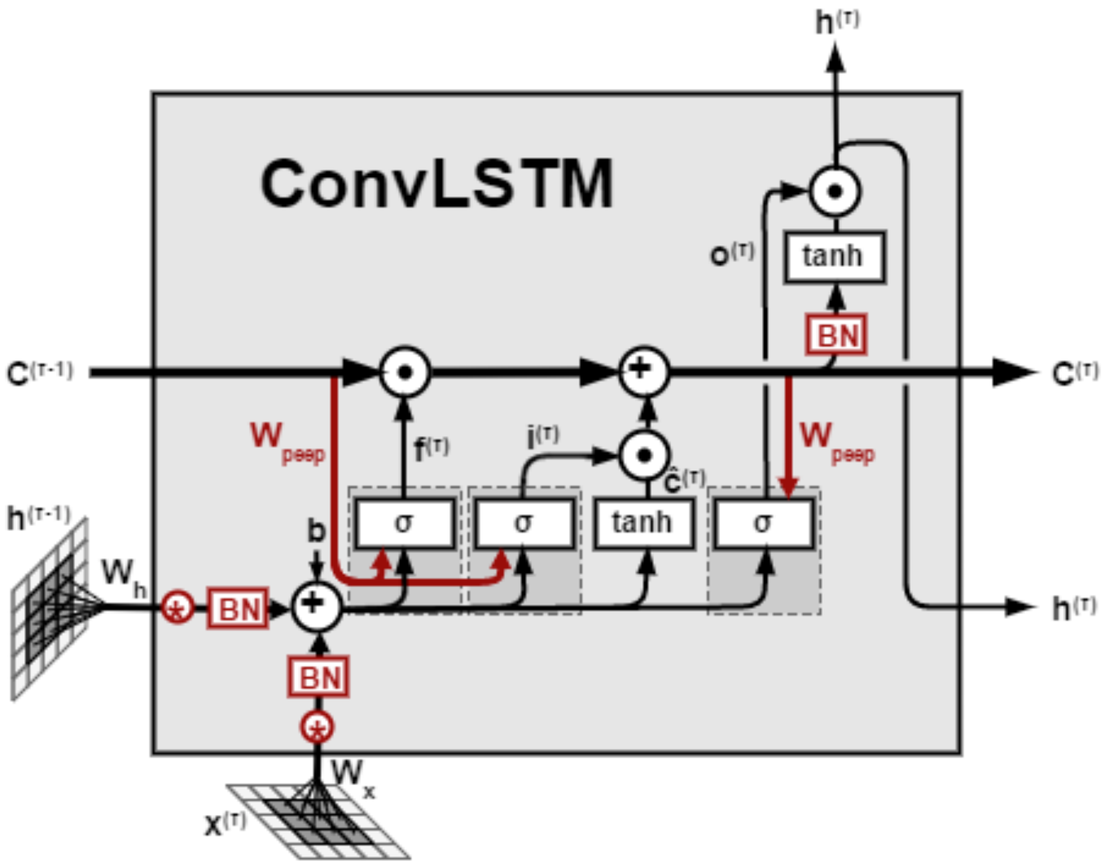




หัวข้อ

- ข้อมูลแบบอนุกรมเวลา (Time Series)
- โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)
- หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)
- **หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น (Convolution Long Short-Term Memory: ConvLSTM)**
- วิธีการสร้าง Convolution LSTM ด้วย Pytorch
- โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ

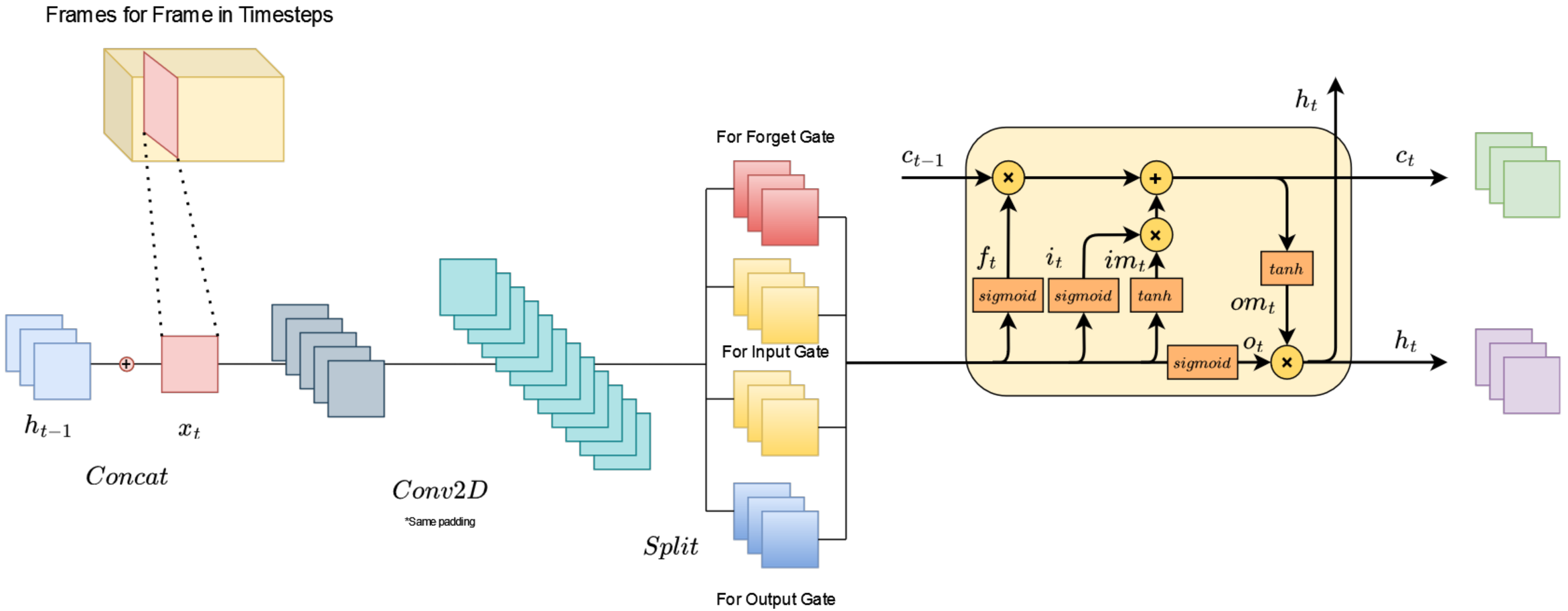
หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น



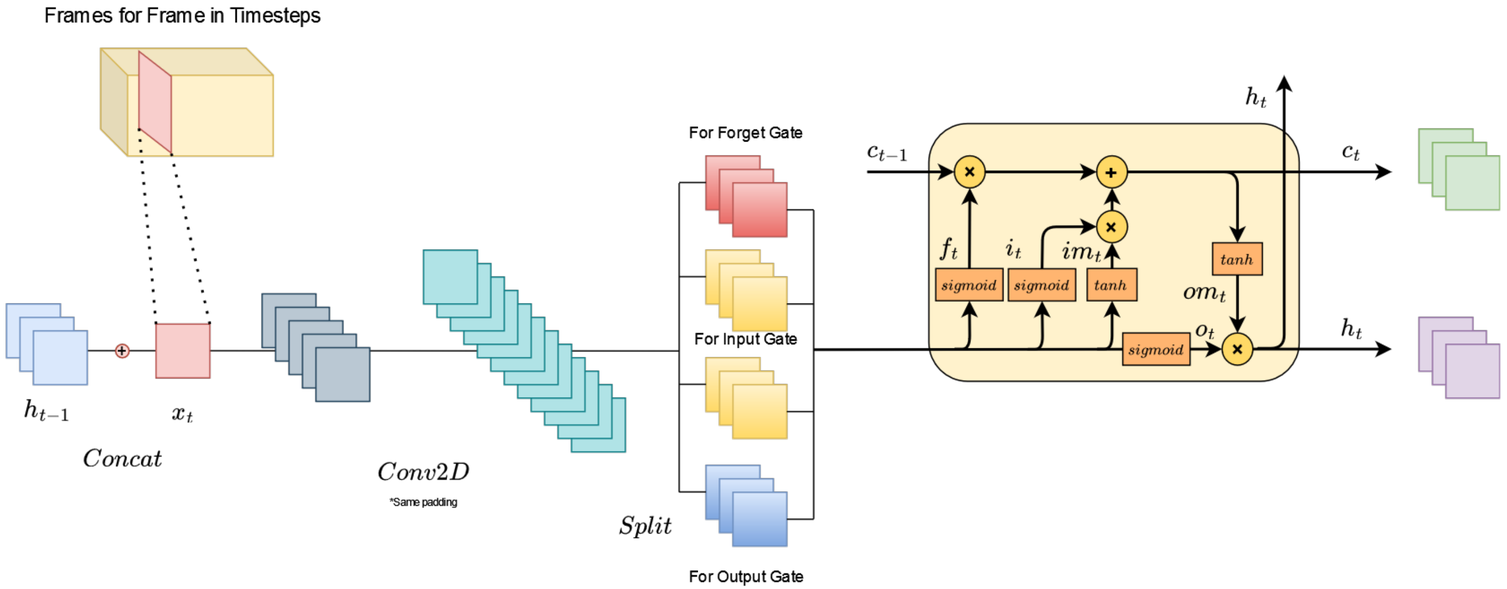
ในปัจจุบันนี้ การค้นหาข้อมูลในรูปแบบของลำดับภาพ (a sequence of images) กำลังได้รับความนิยมเป็นอย่างมาก อาทิ เช่น Movies and trailers, Satellites pictures, และ Security cameras โดย Convolutional LSTM Network ได้รับการคิดค้นขึ้นในปี ค.ศ. 2015 โดย Xingjian Shi และคณะ เป็น recurrent neural network ชนิดหนึ่งสำหรับใช้ทำนายข้อมูลในมุมมองเชิงภูมิศาสตร์และด้านเวลาควบคู่กัน ซึ่งจะประกอบด้วย 2 ส่วนหลัก ได้แก่ Convolution Layer และ LSTM

อ้างอิง https://medium.com/neuronio/an-introduction-to-convlstm-55c9025563a7

หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น



หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น



คือ การผสมผสานระหว่าง Convolutional Neural Networks (CNNs) และ Long Short-Term Memory (LSTM) ซึ่งถูกออกแบบมาเพื่อประมวลผลข้อมูลที่มีโครงสร้างเชิงพื้นที่และเชิงเวลา เช่น วิดีโอหรือข้อมูลทางภูมิศาสตร์ ConvLSTM ถูกสร้างขึ้นเพื่อจัดการกับปัญหาที่ LSTM แบบดั้งเดิมไม่สามารถจัดการได้ดี ซึ่งรวมถึงการจัดการกับข้อมูลที่มีมิติสูงและมีการเชื่อมโยงเชิงพื้นที่ ซึ่งลักษณะสำคัญของ ConvLSTM คือ การใช้การคอนโวลูชันในส่วนของการคำนวณสถานะซ่อน (hidden state) และอินพุต (Input) ทำให้สามารถจับลักษณะเชิงพื้นที่ได้ดีขึ้น การเปลี่ยนแปลงหลักจาก LSTM ดั้งเดิม ประกอบด้วย 2 ส่วน ได้แก่

- เปลี่ยนแปลง element-wise multiplication ด้วยการคอนโวลูชัน (convolution operation) ในการคำนวณสถานะซ่อนและอินพุต
- การใช้ตัวกรองคอนโวลูชันเพื่อคำนวณการอัพเดตสถานะซ่อน แทนการคูณเมทริกซ์แบบเดิม

หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น

หลักการคำนวณของแต่ละ Gate ใน ConvLSTM จะเขียนได้ดังสมการต่อไปนี้

$$
\begin{aligned}
\mathbf{i_t} &= \text{Sigmoid}\left(\text{Conv}\left(\mathbf{x_t}; \mathbf{w_{xi}}\right) + \text{Conv}\left(\mathbf{h_{t-1}}; \mathbf{w_{hi}}\right) + \mathbf{b_i}\right) \\
\mathbf{f_t} &= \text{Sigmoid}\left(\text{Conv}\left(\mathbf{x_t}; \mathbf{w_{xf}}\right) + \text{Conv}\left(\mathbf{h_{t-1}}; \mathbf{w_{hf}}\right) + \mathbf{b_f}\right) \\
\mathbf{o_t} &= \text{Sigmoid}\left(\text{Conv}\left(\mathbf{x_t}; \mathbf{w_{xo}}\right) + \text{Conv}\left(\mathbf{h_{t-1}}; \mathbf{w_{ho}}\right) + \mathbf{b_o}\right) \\
\mathbf{g_t} &= \text{Tanh} \quad \left(\text{Conv}\left(\mathbf{x_t}; \mathbf{w_{xg}}\right) + \text{Conv}\left(\mathbf{h_{t-1}}; \mathbf{w_{hg}}\right) + \mathbf{b_g}\right) \quad , \\
\mathbf{c_t} &= \mathbf{f_t} \odot \mathbf{c_{t-1}} + \mathbf{i_t} \odot \mathbf{g_t} \\
\mathbf{h_t} &= \mathbf{o_t} \odot \text{Tanh}(\mathbf{c_t})
\end{aligned}
$$

อ้างอิง https://medium.com/neuronio/an-introduction-to-convlstm-55c9025563a7



หัวข้อ

- ข้อมูลแบบอนุกรมเวลา (Time Series)
- โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)
- หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)
- หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น
  (Convolution Long Short-Term Memory: ConvLSTM)
- **วิธีการสร้าง Convolution LSTM ด้วย Pytorch**
- โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

การสร้างโมเดล Convolution LSTM ยังคงสามารถใช้ลำดับขั้นตอนของการสร้างโมเดลแบบพื้นฐานคือ

ข้อ	คำอธิบาย
1	กำหนดโครงสร้าง Dataset ที่จะนำมาใช้ในการฝึกสอนโมเดล
2	กำหนด Convolution LSTM สำหรับการฝึกสอนโมเดล
3	เลือก Loss Function และ Optimizer ที่เหมาะสมกับงานของเรา
4	การฝึกสอนโมเดล
5	การทดสอบและวัดผลโมเดล

ซึ่งส่วนที่แตกต่างไปคือรายละเอียดในแต่ละขั้นตอนเท่านั้น โดยจะถูกเปลี่ยนไปให้เหมาะสมกับลักษณะงานที่จะไปประยุกต์ใช้หรือวิธีการที่นำมาประยุกต์ใช้งาน

อ้างอิง : https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
class VideoDataset(Dataset):


    def __init__(self, root_dir, labels, transform=None, max_frames=0,
resize=(224, 224)):
        self.root_dir = root_dir
        self.labels = labels
        self.transform = transform
        self.max_frames = max_frames
        self.resize = resize
        self.video_paths = []
        self.video_labels = []

        for label in labels:
            label_dir = os.path.join(root_dir, label)
            for filename in os.listdir(label_dir):
                if filename.endswith(('.mp4', '.avi', '.mov')):
                    self.video_paths.append(os.path.join(label_dir, filename))
                    self.video_labels.append(label)

    def __len__(self):
        return len(self.video_paths)
```

1. กำหนดโครงสร้าง Dataset ที่จะนำมาใช้ในการฝึกสอนโมเดล

ในการทำ Video Classification จะมีความลักษณะพิเศษคือรับข้อมูลนำเข้าเป็นข้อมูล 5 มิติ (Batch, Frame, Width, Height, Channel) ซึ่งไม่มีชุดข้อมูลสำเร็จรูปสำหรับข้อมูลประเภทนี้ เราจึงจำเป็นต้องกำหนดโครงสร้าง Dataset โดยใช้ชุดคำสั่งแบบกำหนดเองที่ชุดคำสั่ง Pytorch กำหนดไว้ให้ โดยในส่วนของฟังก์ชัน __init__ จะจัดเก็บข้อมูลที่จำเป็นสำหรับการเรียกใช้ (Load Video) ขณะฝึกสอนแบบจำลอง เช่น ชื่อวิดีโอ ป้ายเฉลย ขนาดวิดีโอ



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
class VideoDataset(Dataset):

    def __getitem__(self, idx):
        video_path = self.video_paths[idx]
        label = self.video_labels[idx]
        video_frames = self.load_video(video_path) #numpy format

        if self.transform:
            video_frames = [self.transform(frame) for frame in video_frames]

        # Stack frames to create a tensor
        video_tensor = torch.stack(video_frames)

        # Assuming labels are provided as strings
        label_idx = self.labels.index(label)

        return video_tensor, label_idx
```

ถัดมา ขั้นตอนการเรียกใช้ข้อมูลจะทำการเรียกขึ้นเป็นลักษณะของภาพเรียงต่อกัน (รายละเอียดจะอยู่ในสไลด์ถัดไป) ในรูปแบบของ numpy.array() จากนั้นนำมาผ่าน transform ที่ละภาพหรือเฟรม (frame) ก่อนจะนำมาต่อกันให้อยู่ในรูปแบบของ tensor ก่อนส่งออกไปประมวลผลต่อไป



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
class VideoDataset(Dataset):
    def load_video(self, path):
        cap = cv2.VideoCapture(path)
        frames = []
        try:
            while True:
                ret, frame = cap.read()
                 # Stop condition when failing to read
                if not ret:
                    break
                 # Preprocess Option
                if self.resize:
                    frame = cv2.resize(frame, self.resize)
                frames.append(frame)
                 # Stop condition when the number of maximum frames equals
                 # the number of frames read from the video
                if self.max_frames and (len(frames) >= self.max_frames):
                    break
        finally:
            cap.release()
        return np.array(frames)

         # Zero is False and Positive and Negative Number is True.
```

ในส่วนของการอ่านวิดีโอ ภาษาไพทอนมีชุดข้อมูลที่นิยมนำมาใช้คือ OpenCV โดยจะทำการอ่านที่ละเฟรมไปจนสิ้นสุดความยาวของวิดีโอ ซึ่งสามารถกำหนดรายละเอียด หรือทำ Data Processing จากส่วนนี้ได้เลย ก่อนที่จะเก็บใส่ list และแปลงเป็น numpy.array โดยที่ ret จะคืนค่าเป็น True เมื่อการอ่านเฟรมมาสำเร็จและจะคืนค่าเป็น False เมื่ออ่านเฟรมไม่สำเร็จ



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
# Parameters
root_dir = './action_classification_dataset/train'
classes = ['Biking', 'Diving', 'HorseRiding']
transform = transforms.Compose([
    transforms.ToImage(),
    transforms.ToDtype(torch.float32),
    transforms.Normalize(mean=(0.5, 0.5, 0.5), std=(0.5, 0.5, 0.5))

])

batch_size = 4
max_frames = 30
resize = (112, 112)

# Create dataset
train_dataset = VideoDataset(root_dir=root_dir, labels=classes,
transform=transform, max_frames=max_frames, resize=resize)

train_dataloader = DataLoader(train_dataset, batch_size=batch_size,
shuffle=True, num_workers=0)
```

เมื่อสร้าง Dataset เสร็จเรียบร้อยแล้ว ให้นักศึกษานำข้อมูลเข้าไปเพื่อเตรียมการแปลงข้อมูลให้อยู่ในรูปแบบที่เหมาะสม รวมถึงกำหนดพารามิเตอร์ที่ต้องการ เช่น ขนาดแบตช์ (batch size) สำหรับพารามิเตอร์ num_workers เมื่อมีค่าเท่ากับ 0 หรือ 1 โปรแกรมจะสร้าง Instance เดียวในการโหลดข้อมูลผ่าน CPU ในขณะที่ num_workers มากกว่า 1 โปรแกรมจะสร้างจำนวน Instance เท่ากับจำนวนที่กำหนด เพื่อช่วยกันโหลดข้อมูลผ่าน CPU โดยจำนวน num_workers ไม่ควรเกินจากจำนวนคอร์ (core) ของ CPU

หมายเหตุ num_workers มีการทำงานเบื้องต้นแบบ synchronize



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
from network.convlstm import ConvLSTM
import torch.nn as nn

class ConvLSTMModel(nn.Module):
    def __init__(self, num_classes, image_size, batch_size=1, hidden_dim=[64, 64]):

        self.batch_size = batch_size
        self.convlstm = ConvLSTM(input_dim=3, hidden_dim=hidden_dim, kernel_size=(3, 3), num_layers=2, batch_first=True)
        self.fc = nn.Linear(image_size[0] * image_size[1] * hidden_dim[-1], num_classes)

    def forward(self, x):
        _, last_state_list = self.convlstm(x)                          # dada shape (batch_size, time, c, h, w)
        x = last_state_list[-1][0].view(self.batch_size, -1)           # reshape information from last hidden state

        x = self.fc(x)
        return x


num_classes = len(classes)
model = ConvLSTMModel(num_classes, image_size=(112, 112), batch_size=batch_size)
```

2. กำหนด Convolution LSTM Network สำหรับการฝึกสอนโมเดล

เนื่องจากในปัจจุบันนี้ชุดคำสั่งสำเร็จรูปสำหรับ Convolution LSTM ในชุดคำสั่ง Pytorch ยังไม่มีให้บริการ ดังนั้น นักศึกษาต้องกำหนดขึ้นมาเอง โดยลักษณะโครงสร้างสามารถดูได้จากโฟลเดอร์ network ในไฟล์ชื่อ convlstm.py จากนั้น นำลักษณะโครงสร้างที่กำหนดมาต่อเข้ากับ Fully Connected Layer ในชั้นสุดท้ายเพื่อ Classification โดยสืบทอดคลาส nn.Module เพื่อบอกให้ Pytorch ทราบว่านี่คือโมเดล

```
class ConvLSTMCell(nn.Module):

    def __init__(self, input_dim, hidden_dim, kernel_size, bias):
        super(ConvLSTMCell, self).__init__()

        self.input_dim = input_dim
        self.hidden_dim = hidden_dim
        self.kernel_size = kernel_size
        self.padding = kernel_size[0] // 2, kernel_size[1] // 2
        self.bias = bias

        self.conv = nn.Conv2d(in_channels=self.input_dim + self.hidden_dim,  out_channels=4 * self.hidden_dim,
                              kernel_size=self.kernel_size,   padding=self.padding,   bias=self.bias)

    def forward(self, input_tensor, cur_state):
        h_cur, c_cur = cur_state
        combined = torch.cat([input_tensor, h_cur], dim=1)  # concatenate along channel axis
        combined_conv = self.conv(combined)

        cc_i, cc_f, cc_o, cc_g = torch.split(combined_conv, self.hidden_dim, dim=1)
        i = torch.sigmoid(cc_i)
        f = torch.sigmoid(cc_f)
        o = torch.sigmoid(cc_o)
        g = torch.tanh(cc_g)

        c_next = f * c_cur + i * g
        h_next = o * torch.tanh(c_next)

        return h_next, c_next
```

รายละเอียดเพิ่มเติม https://github.com/ndrplz/ConvLSTM_pytorch/blob/master/convlstm.py



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
# define a Loss Function and an optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

3. เลือก Loss Function และ Optimizer

นักศึกษาต้องกำหนด Loss Function และ Optimizer ให้เหมาะสมและสอดคล้องกับงานที่ต้องการทำ โดยพิจารณาว่าเป็นงาน Regression, Multiclass, Multilabel, หรือ Multitask เป็นต้น



วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
num_epochs = 10
model.train()

for epoch in range(num_epochs):
    running_loss = 0.0

    for batch_idx, (videos, labels) in enumerate(dataloader):

        optimizer.zero_grad()
        outputs = model(videos)

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        if batch_idx % 10 == 9:  # print every 10 batches
          print(f'Epoch [{epoch + 1}/{num_epochs}], Batch [{batch_idx
+ 1}/{len(train_dataloader)}], Loss: {running_loss / 10:.4f}')
            running_loss = 0.0
print('Finished Training’)

# Save model
torch.save(model.state_dict(), 'video_classification_model.pt')
```

4. การฝึกสอนโมเดล

ตัวอย่างโปรแกรมฝึกสอนแบบจำลองจากโครงสร้างแบบจำลอง model ที่กำหนดโดยอาศัยข้อมูลมาจาก dataloader ที่นักศึกษาได้ custom ขึ้นมา โดยมีการเรียกใช้งาน Loss Function ผ่านตัวแปร criterion และ Optimizer เมื่อปรับปรุง Weight ของแบบจำลองผ่านกระบวนการฝึกสอน

หลังจากฝึกฝนแบบจำลองถึงจำนวนรอบที่กำหนดผ่านตัวแปร epoch แล้ว โปรแกรมจะบันทึกแบบจำลองไว้ใช้งานในภายหลัง

วิธีการสร้าง Convolution LSTM ด้วย Pytorch

```
from sklearn.metrics import accuracy_score

model.eval()
correct = 0
total = 0

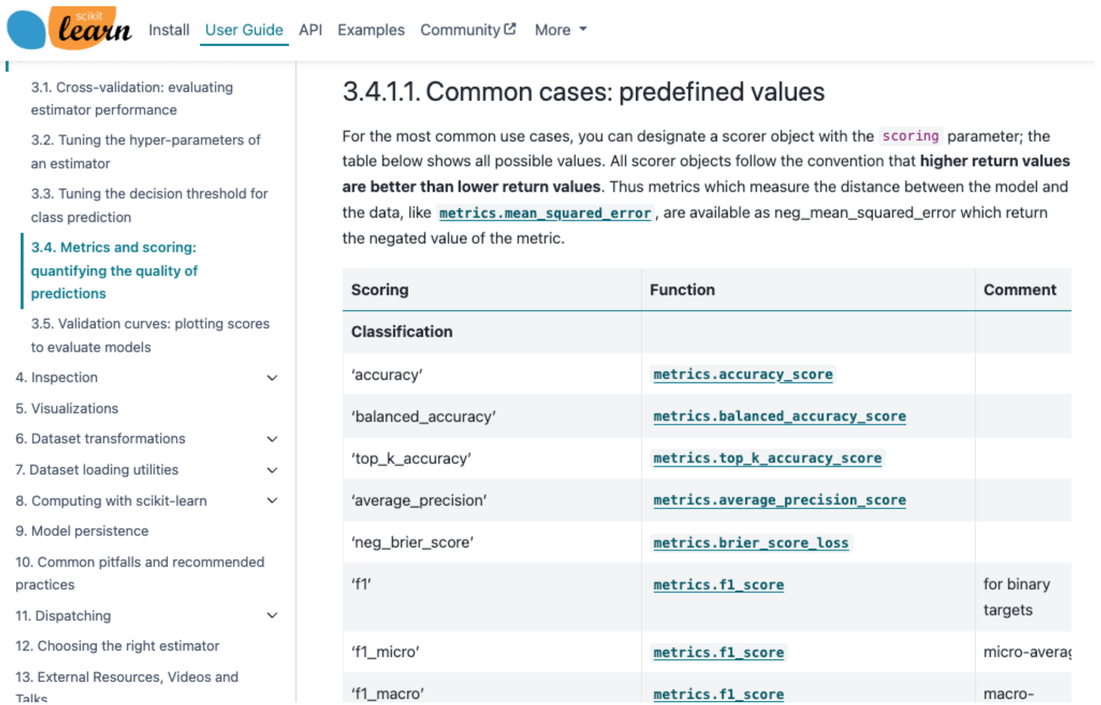
# since we're not training, we don't need to calculate the
gradients for our outputs
with torch.no_grad():
    for inputs, labels in test_dataloader:
        images, labels = inputs, labels

        # calculate outputs by running images through the network
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)

        correct += accuracy_score(labels, predicted)
        total += 1

print(f'Accuracy of the network on the test video: {100 * correct /
total:.02f} %')
```

5. การทดสอบและวัดผลโมเดล



นักศึกษาสามารถเลือกใช้ตัวชี้วัดผ่านชุดคำสั่ง sklearn.metrics ได้



ตัวอย่างโปรแกรมเรียกใช้งานแบบจำลอง

```
import numpy as np
import cv2

def load_video(path, resize, max_frames):
        cap = cv2.VideoCapture(path)
        frames = []
        try:
            while True:
                ret, frame = cap.read()
                if not ret:
                    break
                if resize:
                    frame = cv2.resize(frame, resize)
                frames.append(frame)
                if max_frames and len(frames) >= max_frames:
                    break
        finally:
            cap.release()
        return np.array(frames)
```



ตัวอย่างโปรแกรมเรียกใช้งานแบบจำลอง

```
max_frames = 30
resize = (112, 112)

video_path = 'action_classification_dataset/train/Biking/v_Biking_g01_c01.avi'
video_frames = load_video(video_path, resize, max_frames)
video_frames = [transform(frame) for frame in video_frames]

# Stack frames to create a tensor
video = torch.stack(video_frames)

model.eval()

video = video.to(device)
video = video.unsqueeze(0)

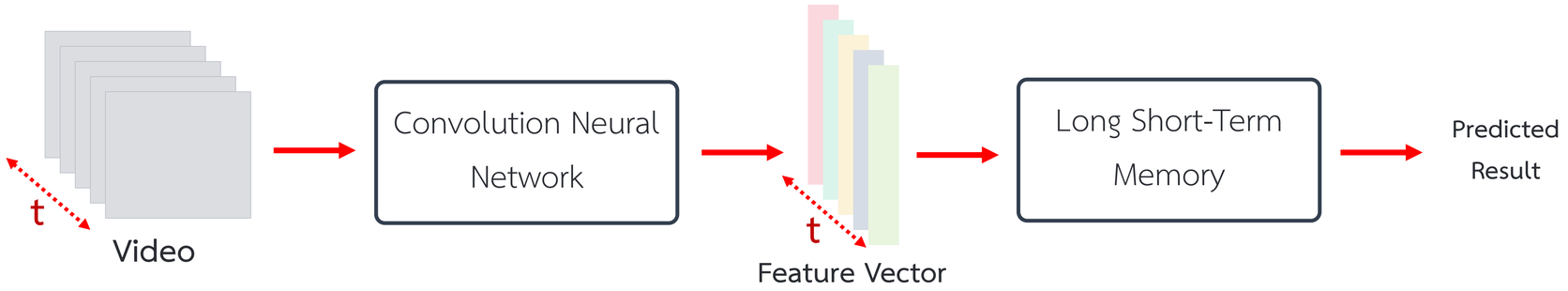
outputs = model(video)
outputs = outputs.squeeze(0)

score = torch.softmax(outputs, 0)
_, predicted = torch.max(score, 0)

print(f'Predicted: {classes[predicted]} {score[predicted] * 100:.02f}%')
```

การใช้งาน Deep Learning สำหรับวีดิโอ

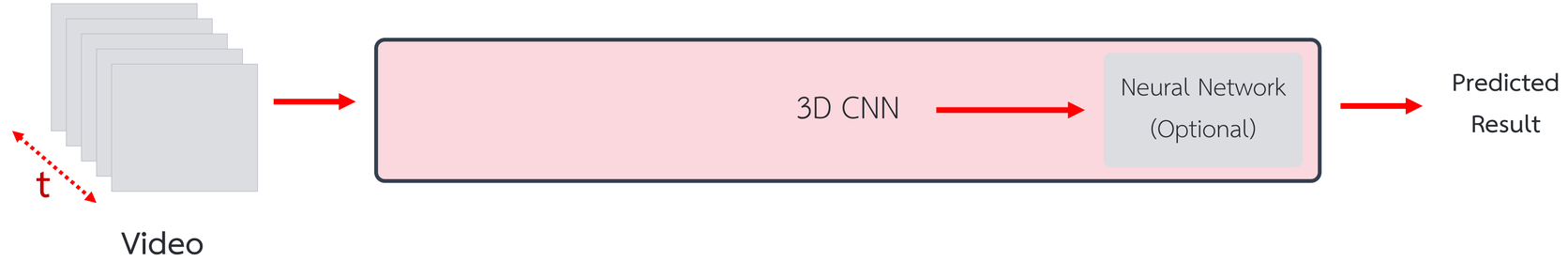
1



3



4

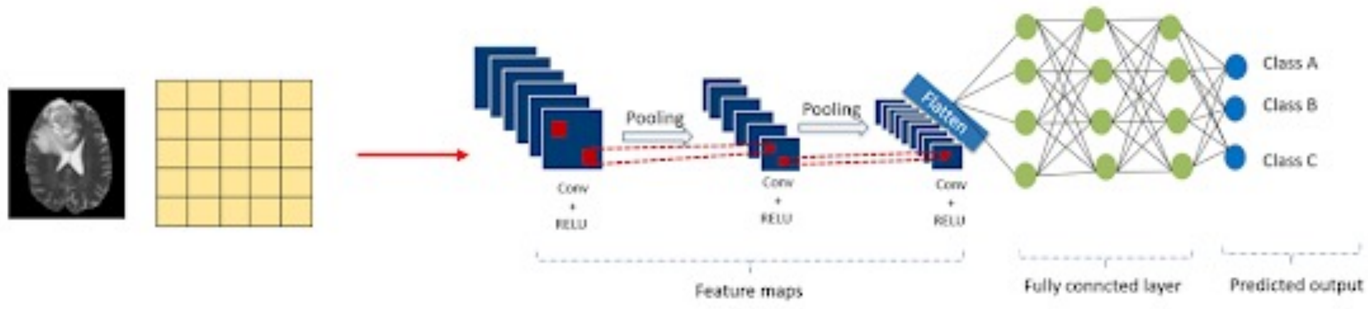




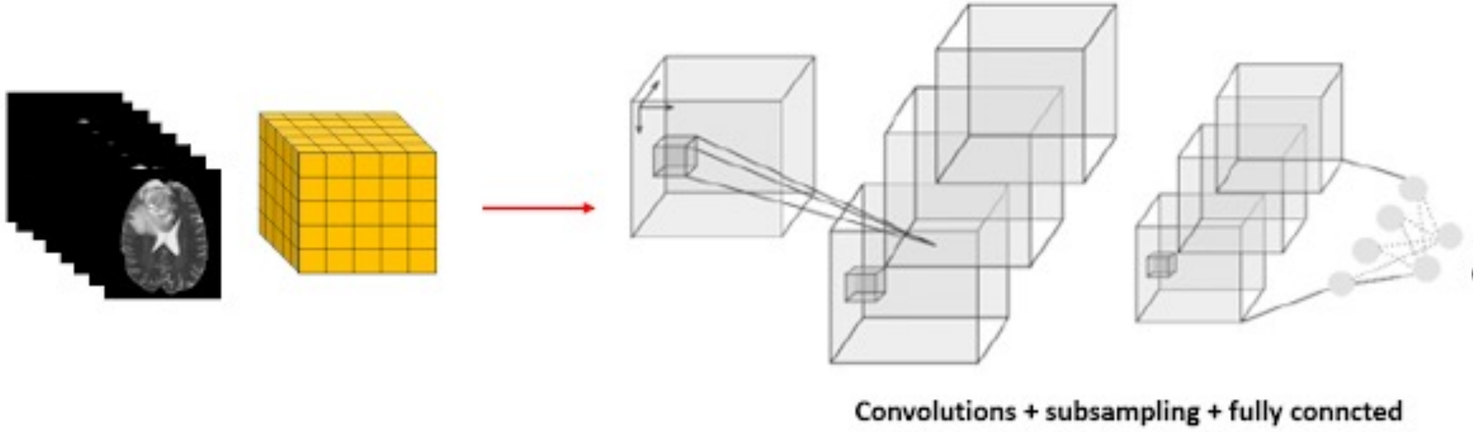
หัวข้อ

- ข้อมูลแบบอนุกรมเวลา (Time Series)
- โครงข่ายประสาทแบบวนกลับ (Recurrent Neural Network: RNN)
- หน่วยความจำระยะสั้นแบบยาว (Long Short-Term Memory: LSTM)
- หน่วยความจำระยะสั้นแบบยาวด้วยคอนโวลูชั่น (Convolution Long Short-Term Memory: ConvLSTM)
- วิธีการสร้าง Convolution LSTM ด้วย Pytorch
- **โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ**

โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ



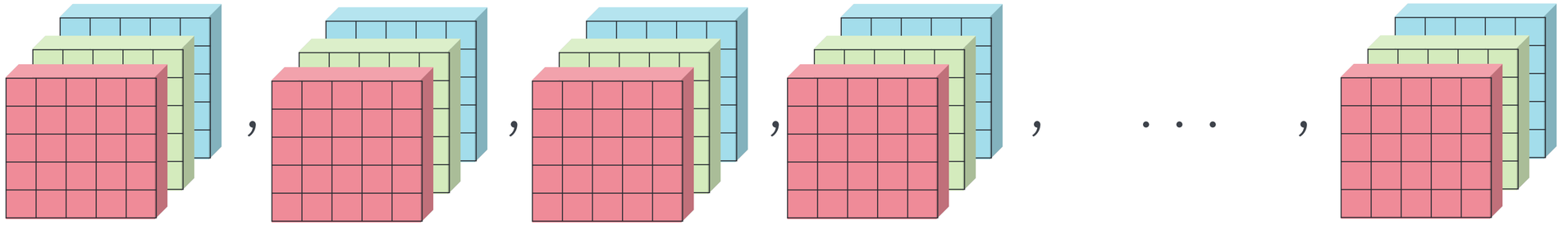
อ้างอิง https://www.reachiteasily.com/2021/06/3d-convolutional-neural-network-pytorch.html



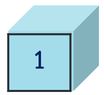
คือ โครงข่ายประสาทแบบคอนโวลูชันประเภทหนึ่งที่ได้รับการออกแบบมาเพื่อให้รองรับข้อมูลแบบ 3 มิติ โดยทั่วไปโครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ จะใช้พื้นที่ (Memory) และเวลาในการประมวลผลมากกว่าแบบ 2 มิติ แต่ค่อนข้างจะมีความแม่นยำมากกว่ามากกว่าโครงข่ายประสาทแบบคอนโวลูชันแบบ 2 มิติ สำหรับข้อมูล 3 มิติ

โครงสร้างข้อมูลวีดิโอสี

Frame ที่ 1 Frame ที่ 2 Frame ที่ 3 Frame ที่ 4 Frame สุดท้าย

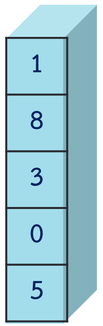
V = {  }

ซึ่งโครงสร้างข้อมูลในลักษณะข้างต้นมักนิยมเรียกว่า Tensor ที่มีดีกรี (Rank) อยู่ที่ 4 ซึ่งบางคนจะเรียกว่าข้อมูล 4 มิติ ก็ได้



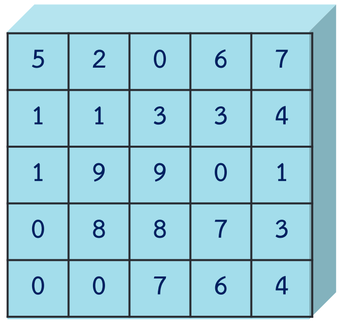
Tensor Rank 0

(Scalar)



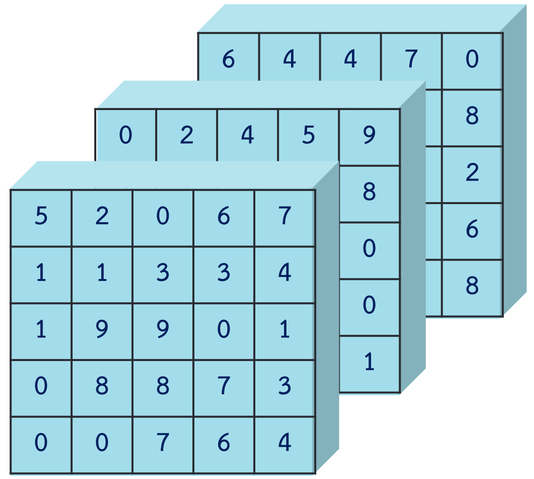
Tensor Rank 1

(Vector)



Tensor Rank 2

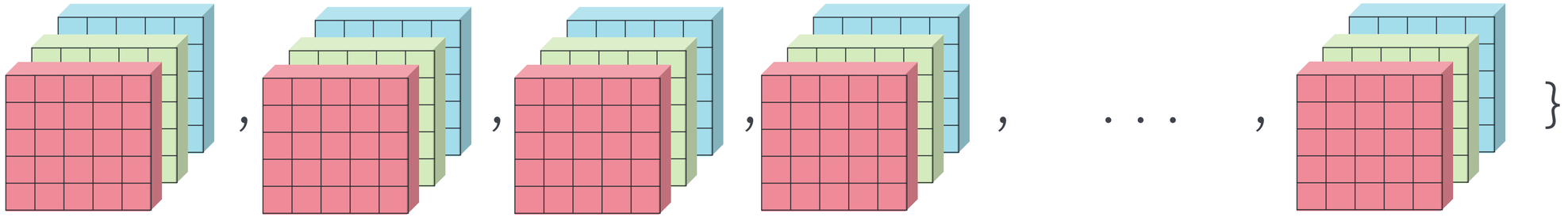
(Matrix)

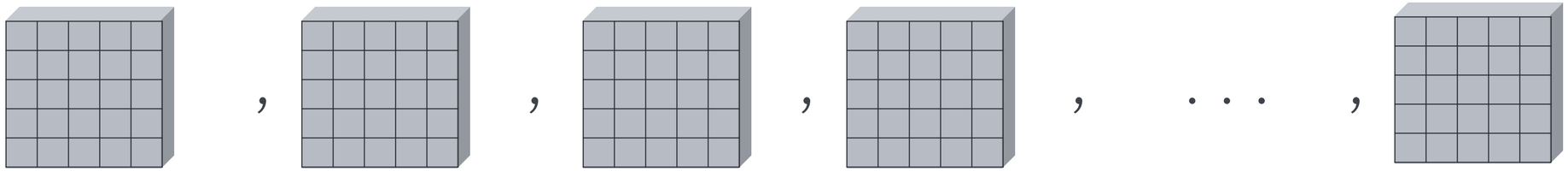


Tensor Rank 3 or higher (Tensor)

โครงสร้างข้อมูลวีดิโอสี

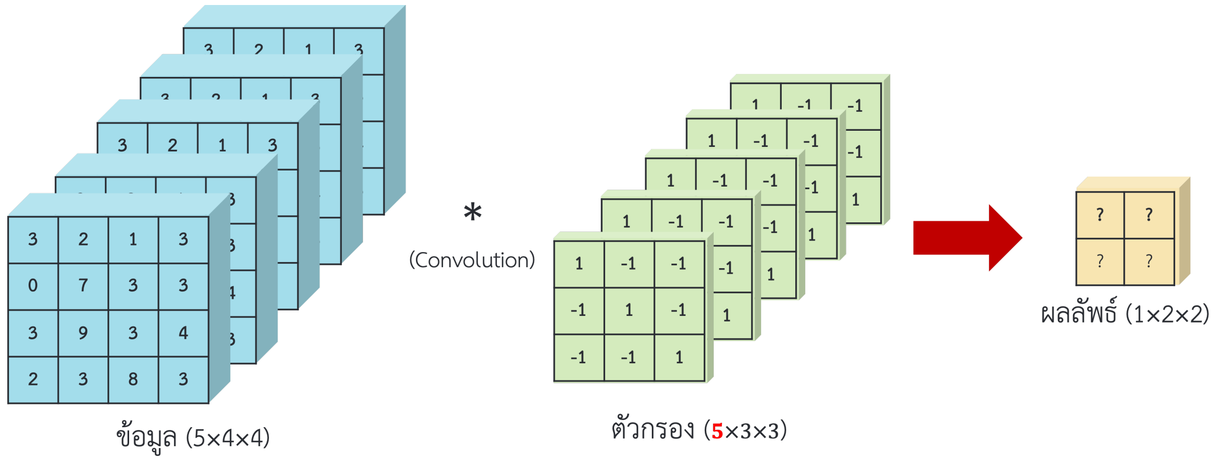
Frame ที่ 1 Frame ที่ 2 Frame ที่ 3 Frame ที่ 4 Frame สุดท้าย

V = {  }

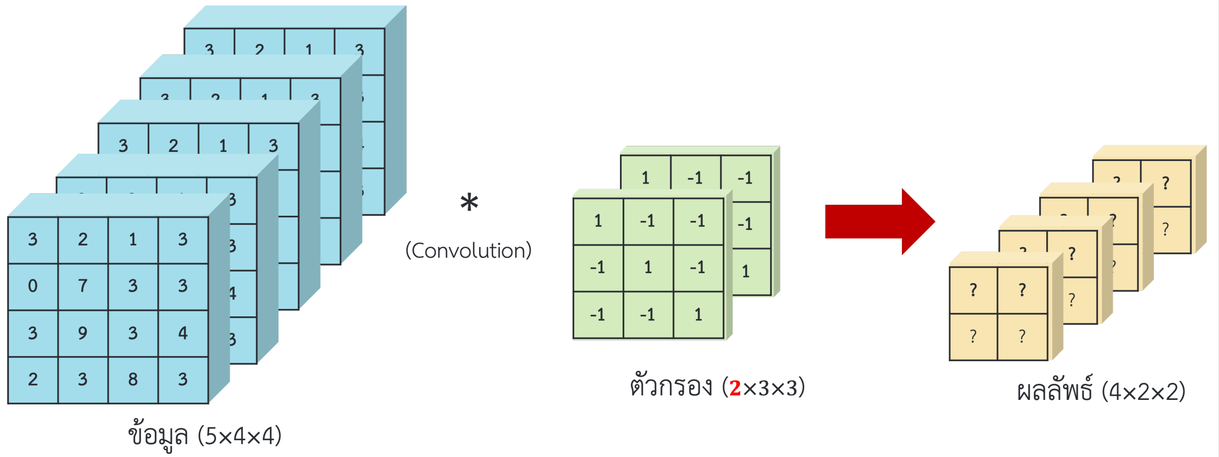
V = {  }

3D Convolution Layer

- 2D Convolution Layer



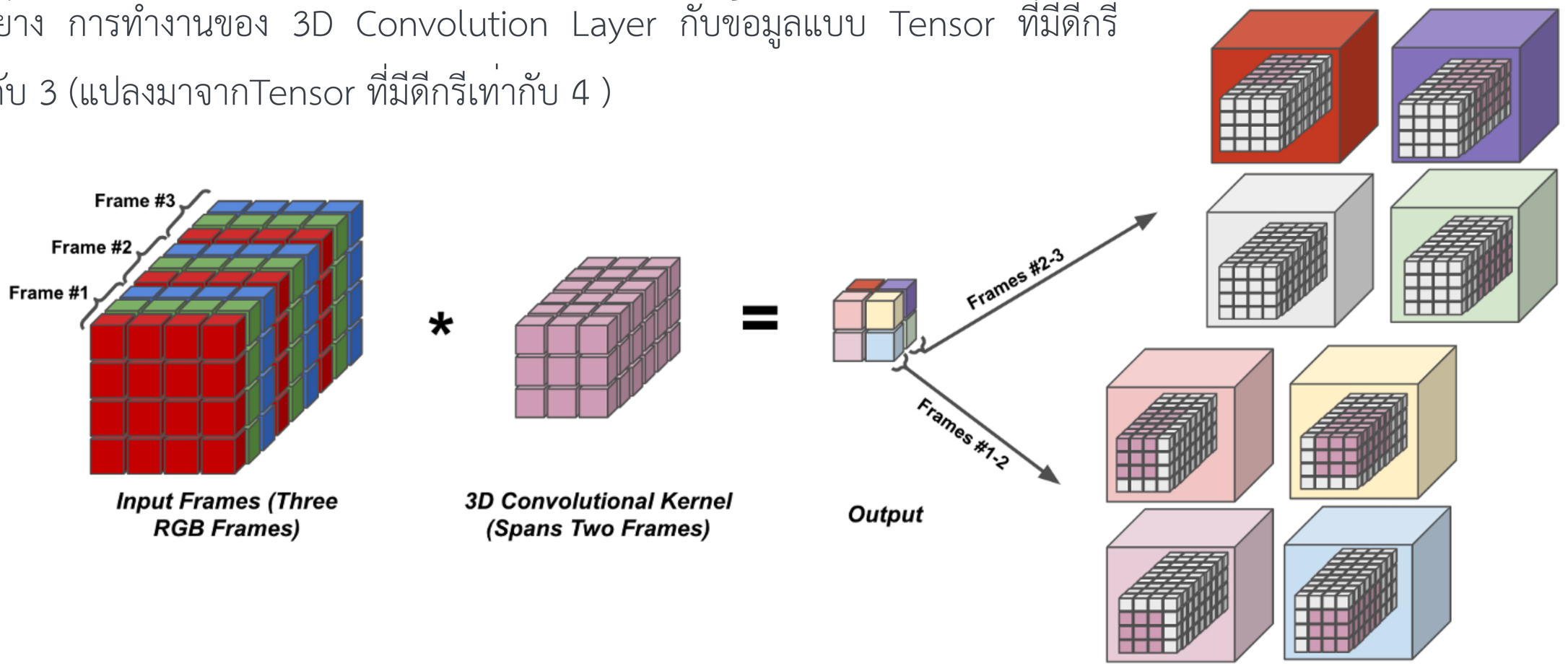
- 3D Convolution Layer



การทำงานของ Convolution Layer สำหรับข้อมูลแบบ Tensor ที่มีดีกรีเท่ากับ 3 (แปลงมาจากTensor ที่มีดีกรีเท่ากับ 4) จะนิยมเรียกว่า 3D Convolution Layer โดยทั่วไป การทำคอนโวลูชั่นแบบ 2 มิติ นักศึกษาจะ**ขยับตัวกรองไปบนข้อมูลเพียงแกน x และ y เท่านั้น** แล้วจะคำนวณจากทุก Channel (หรือแกน Z) แต่ขณะที่ การทำคอนโวลูชั่นแบบ 3 มิติ นักศึกษาจะ**ขยับตัวกรองไปบนข้อมูลทุกแกน (แกน x, y และ z)**

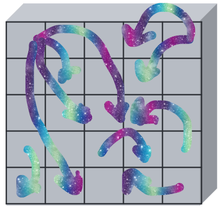
3D Convolution Layer

ตัวอย่าง การทำงานของ 3D Convolution Layer กับข้อมูลแบบ Tensor ที่มีดีกรีเท่ากับ 3 (แปลงมาจากTensor ที่มีดีกรีเท่ากับ 4)

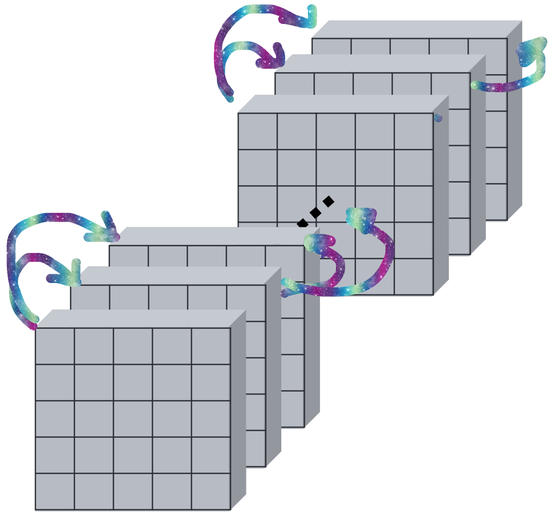


(อ้างอิง https://towardsdatascience.com/deep-learning-on-video-part-one-the-early-days-8a3632ed47d4)

3D Convolution Layer



คุณลักษณะเชิงตำแหน่ง (Spatial Feature)



ความสัมพันธ์ระหว่างเฟรมที่ต่อเนื่องกัน (Temporal Feature)

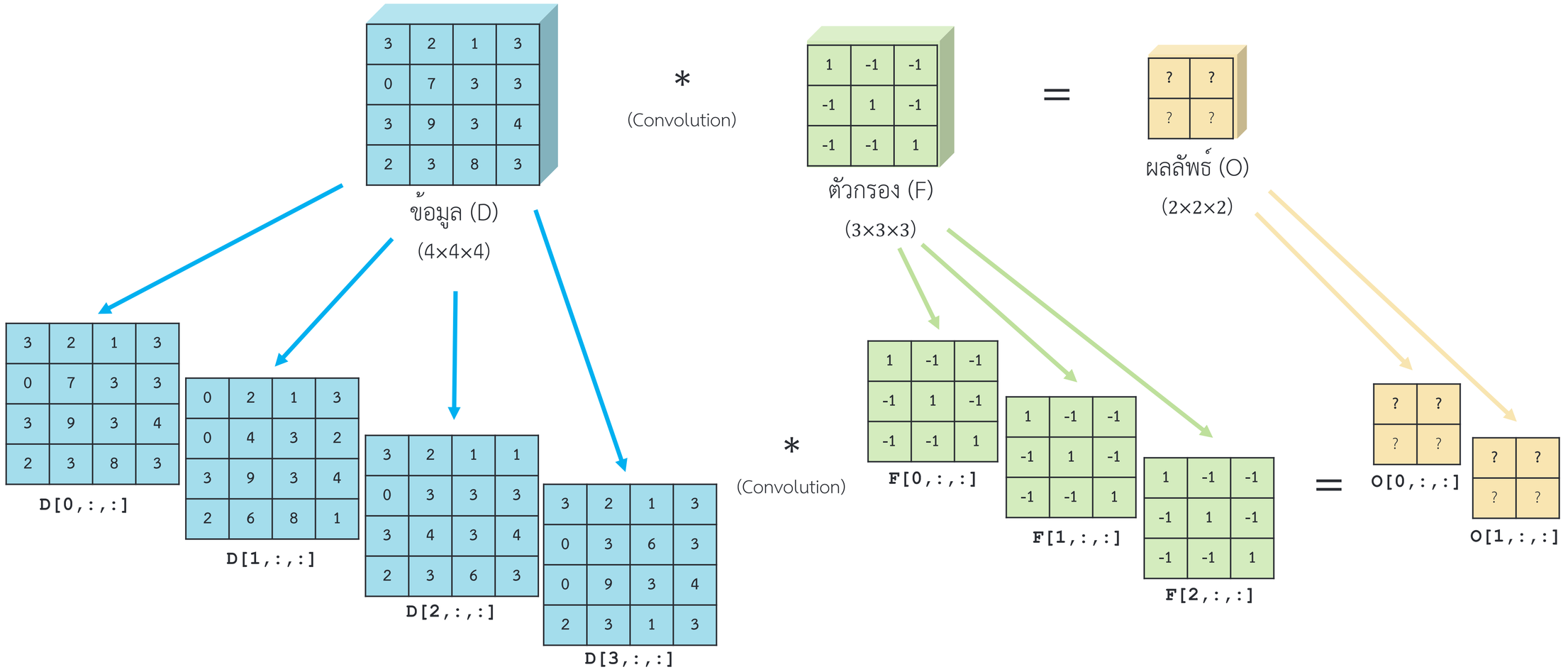
หมายความว่าการสกัดคุณลักษณะ

นอกจากนี้ ยังสามารถกล่าวได้ว่า 3D Convolution คือ ตัวดำเนินการแบบการดำเนินงานเชิงพื้นที่ชั่วคราว (Spatio-Temporal operation) ซึ่งหมายความว่า เป็นตัวดำเนินการที่

- สกัดคุณลักษณะเชิงตำแหน่ง (Spatial Feature) ของแต่ละเฟรม
- สกัดหาความสัมพันธ์ระหว่างเฟรมที่ต่อเนื่องกัน (Temporal Feature)

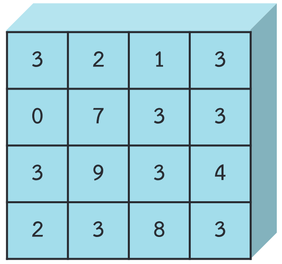
หรือสามารถกล่าวได้ว่า Spatial Feature สื่อถึงคุณลักษณะเชิงพื้นที่หรือตำแหน่ง แต่ขณะที่ Temporal Feature สื่อถึงคุณลักษณะเชิงเวลา

3D Convolution Layer



อ้างอิง https://ichi.pro/th/kar-chi-ngan-thi-la-khan-txn-3d-convolutional-neural-network-ni-keras-20820891513136

3D Convolution Layer

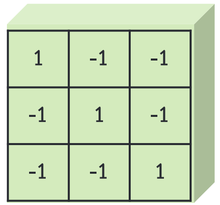


ข้อมูล (D)

(4×4×4)

*

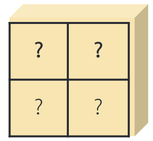
(Convolution)



ตัวกรอง (F)

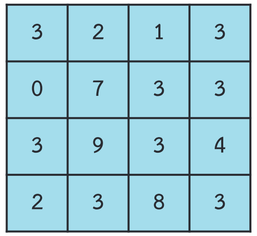
(3×3×3)

=

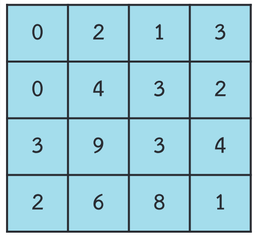


ผลลัพธ์ (O)

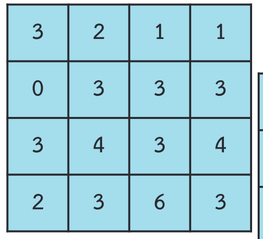
(2×2×2)



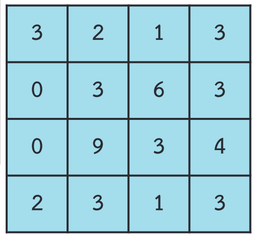
D[0,:,:]



D[1,:,:]



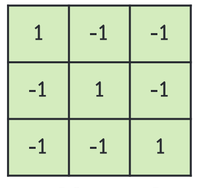
D[2,:,:]



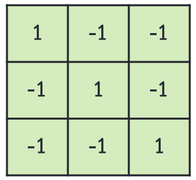
D[3,:,:]

*

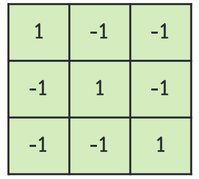
(Convolution)



F[0,:,:]

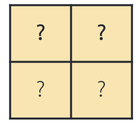


F[1,:,:]



F[2,:,:]

=



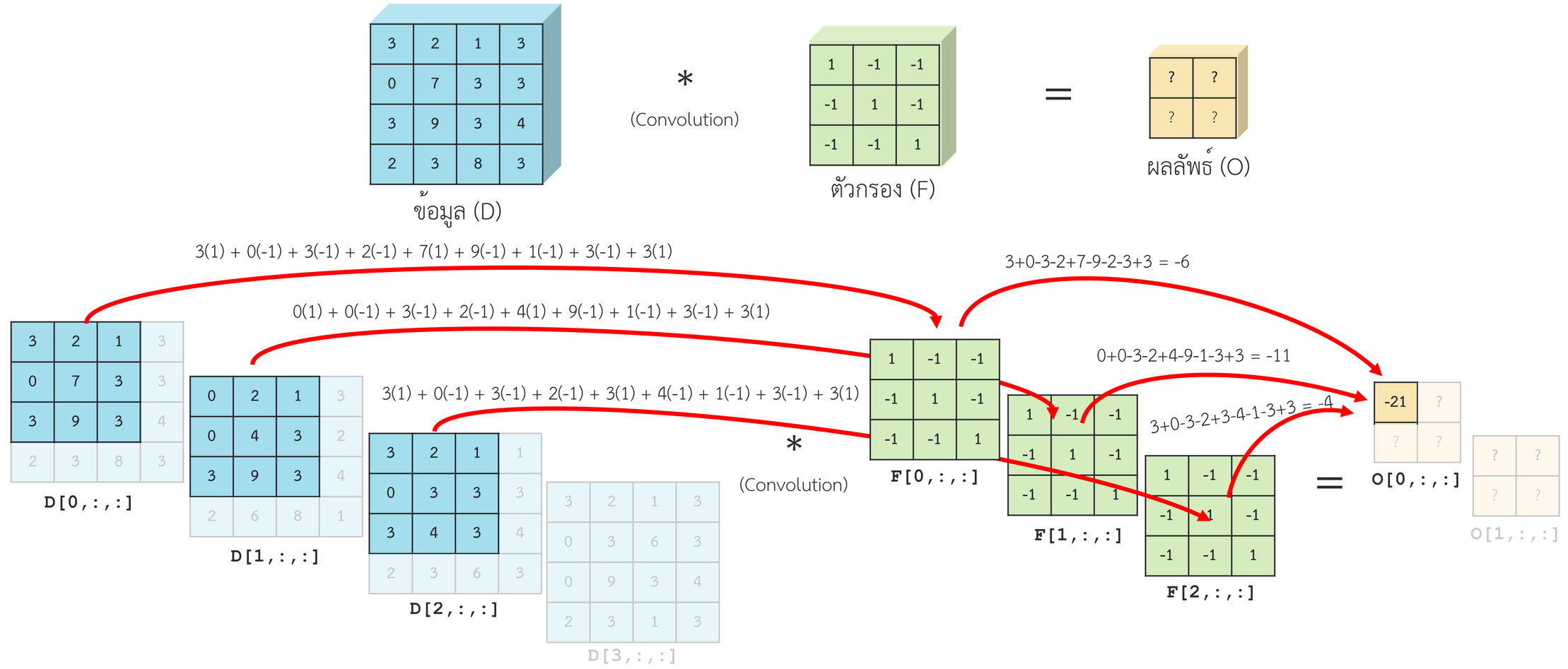
O[0,:,:]



O[1,:,:]

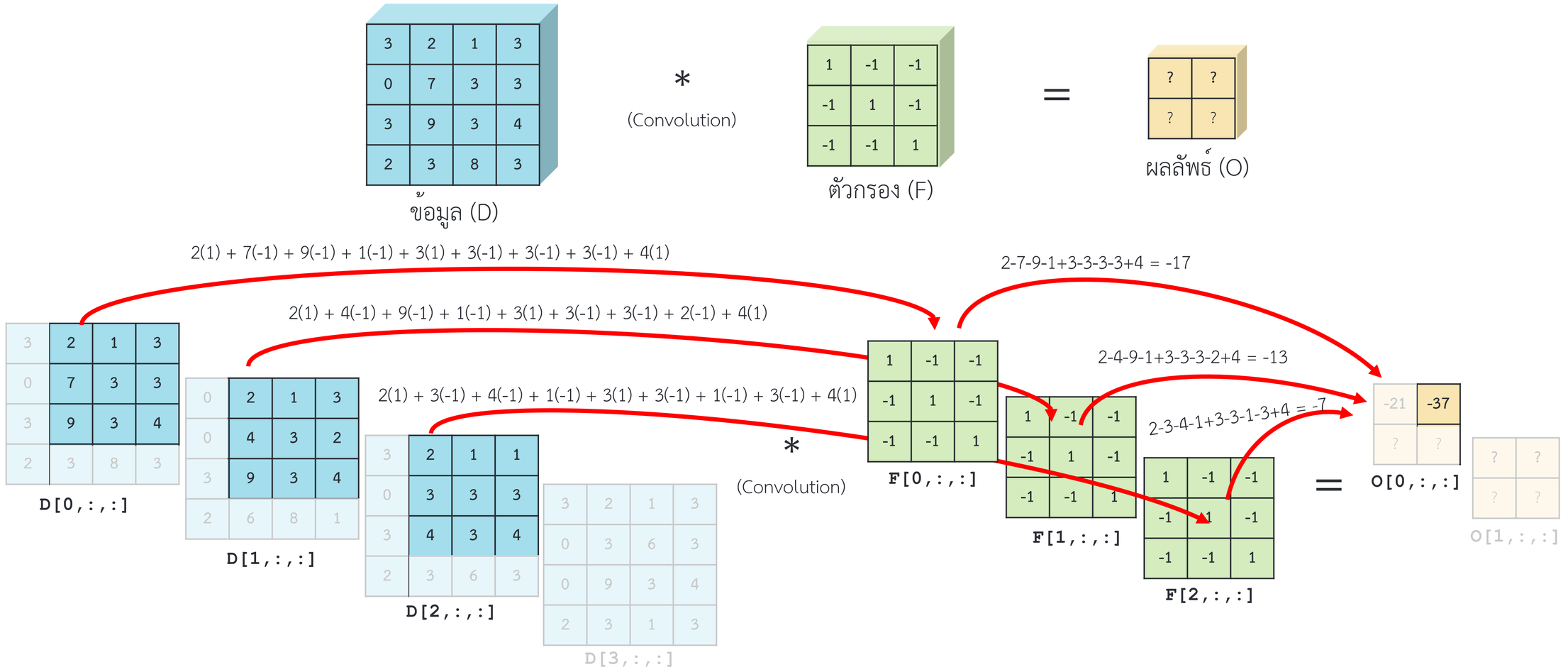
อ้างอิง https://ichi.pro/th/kar-chi-ngan-thi-la-khan-txn-3d-convolutional-neural-network-ni-keras-20820891513136

3D Convolution Layer



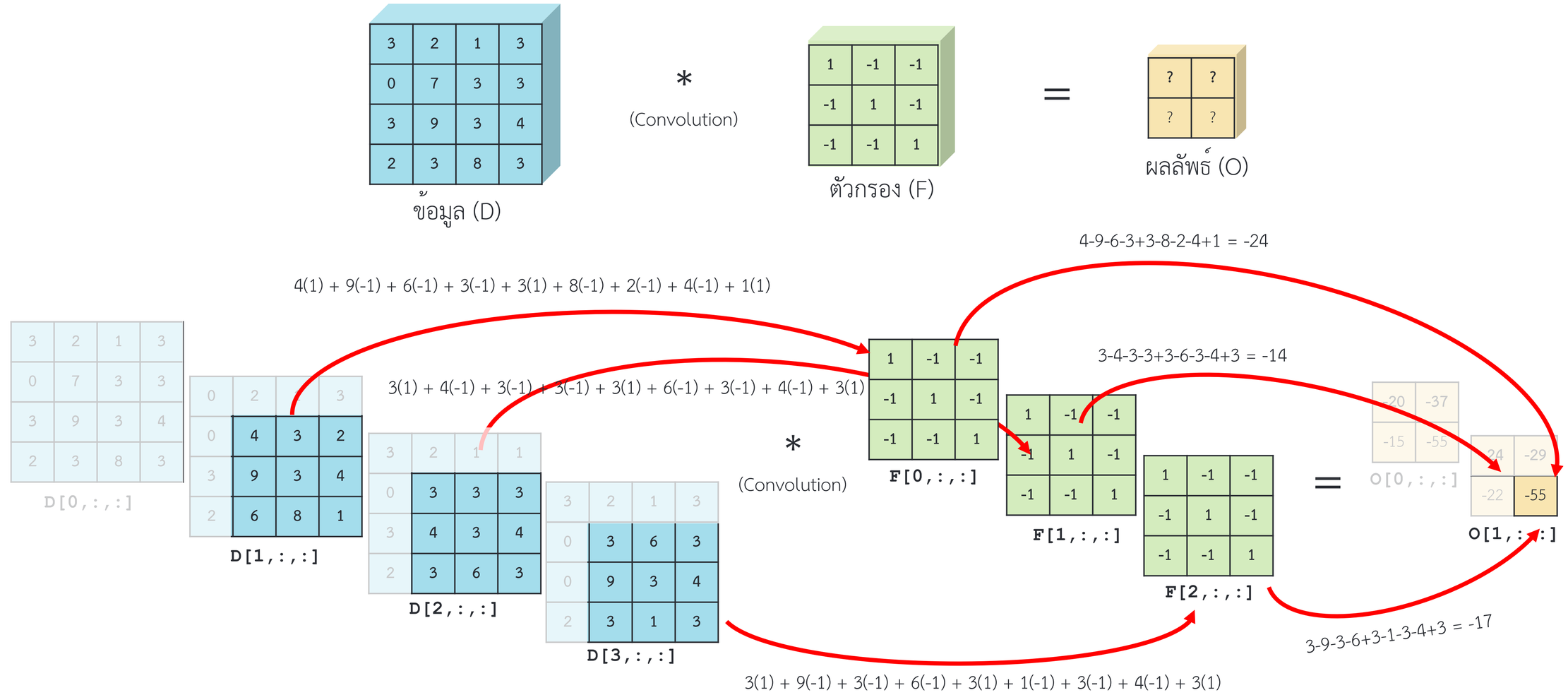
อ้างอิง https://ichi.pro/th/kar-chi-ngan-thi-la-khan-txn-3d-convolutional-neural-network-ni-keras-20820891513136

3D Convolution Layer



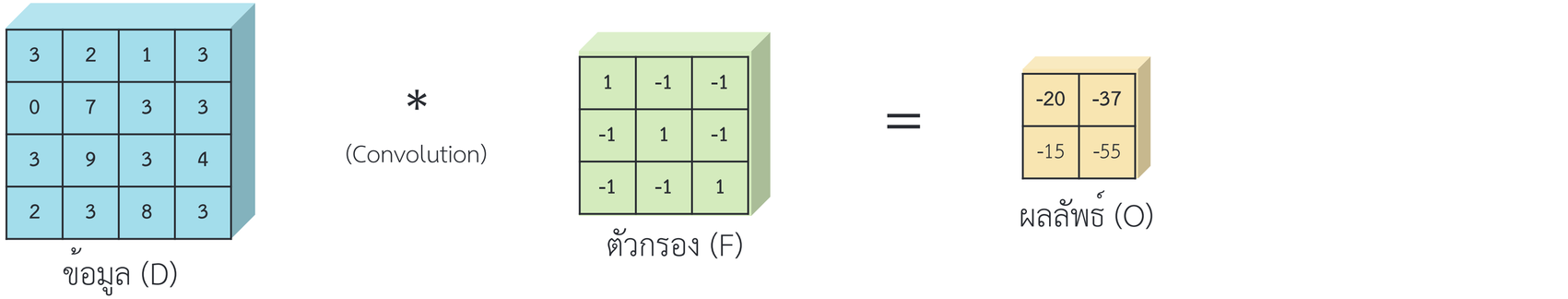
อ้างอิง https://ichi.pro/th/kar-chi-ngan-thi-la-khan-txn-3d-convolutional-neural-network-ni-keras-20820891513136

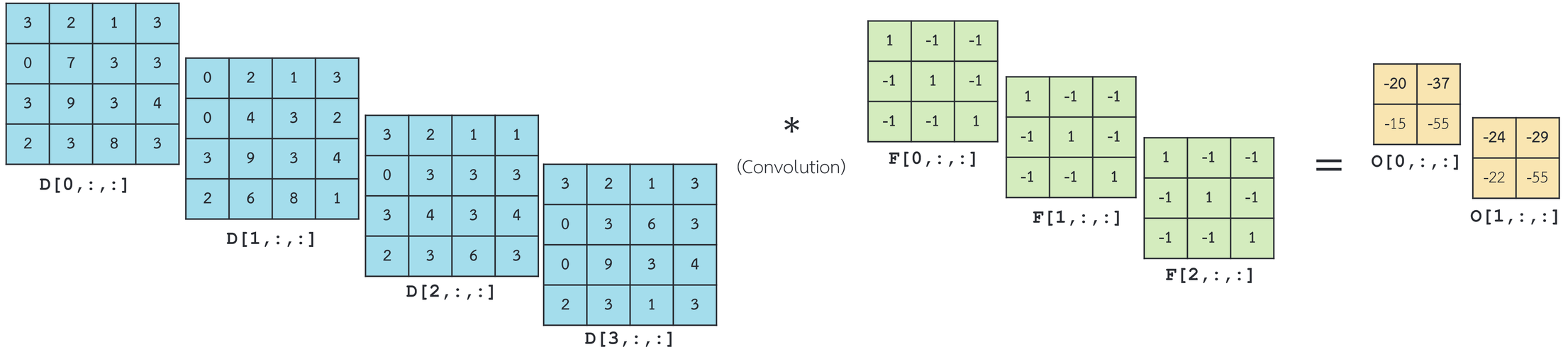
3D Convolution Layer



อ้างอิง https://ichi.pro/th/kar-chi-ngan-thi-la-khan-txn-3d-convolutional-neural-network-ni-keras-20820891513136

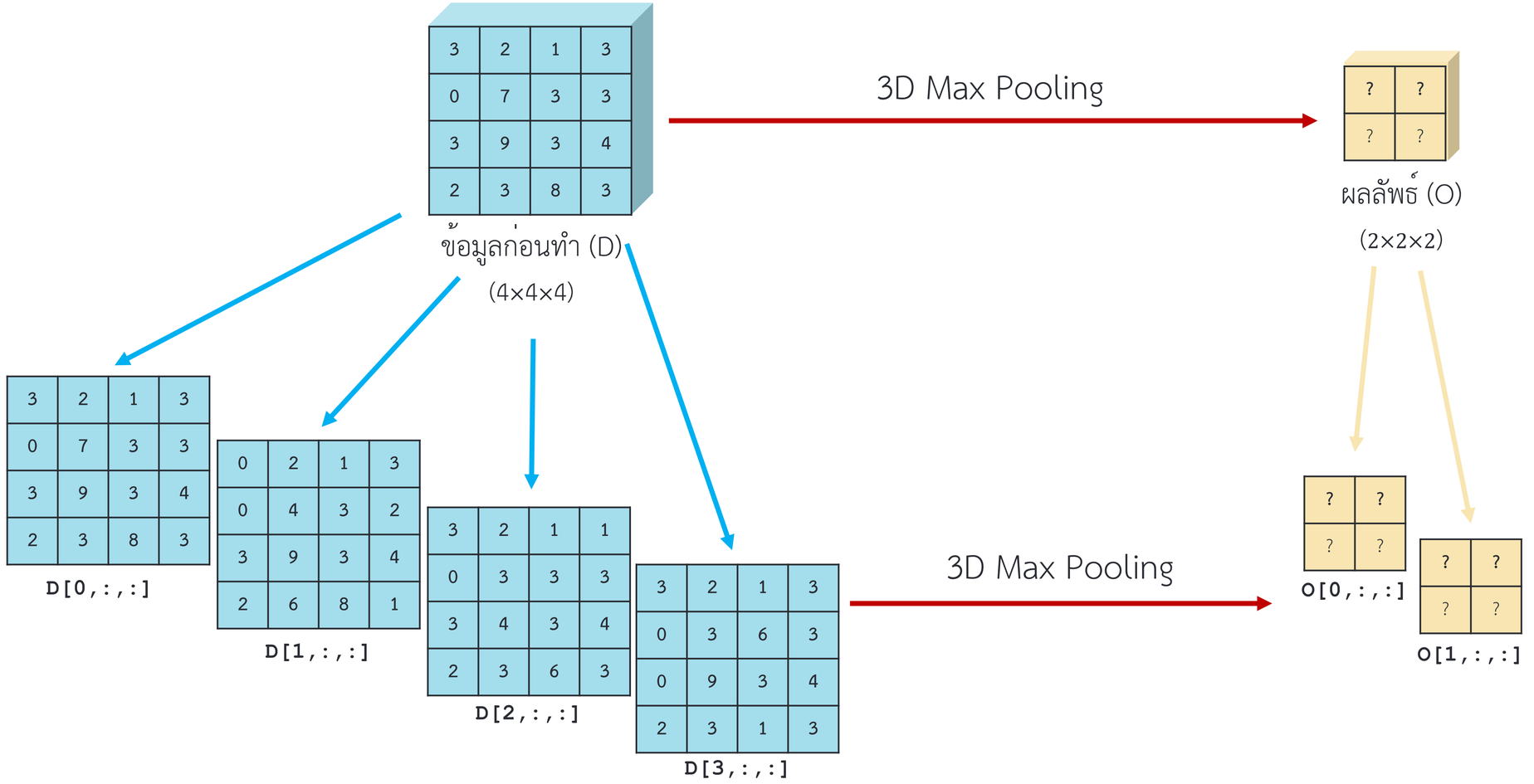
3D Convolution Layer



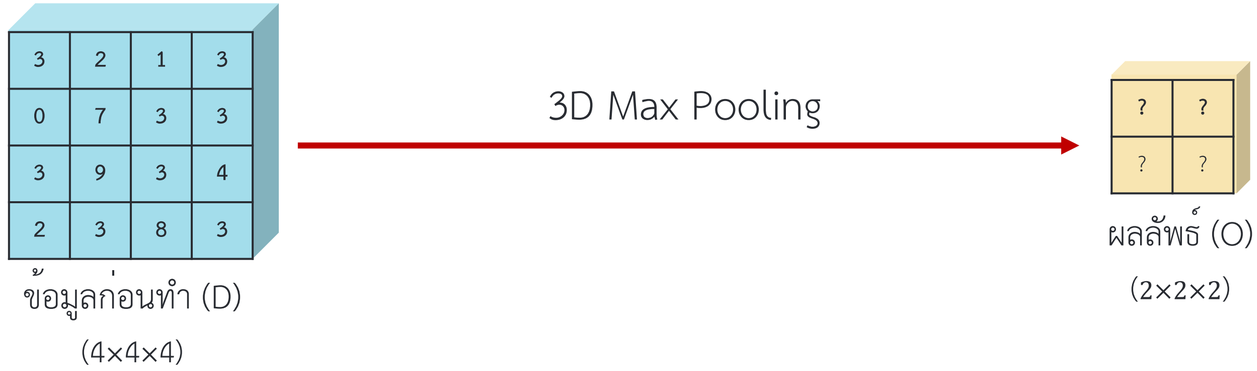


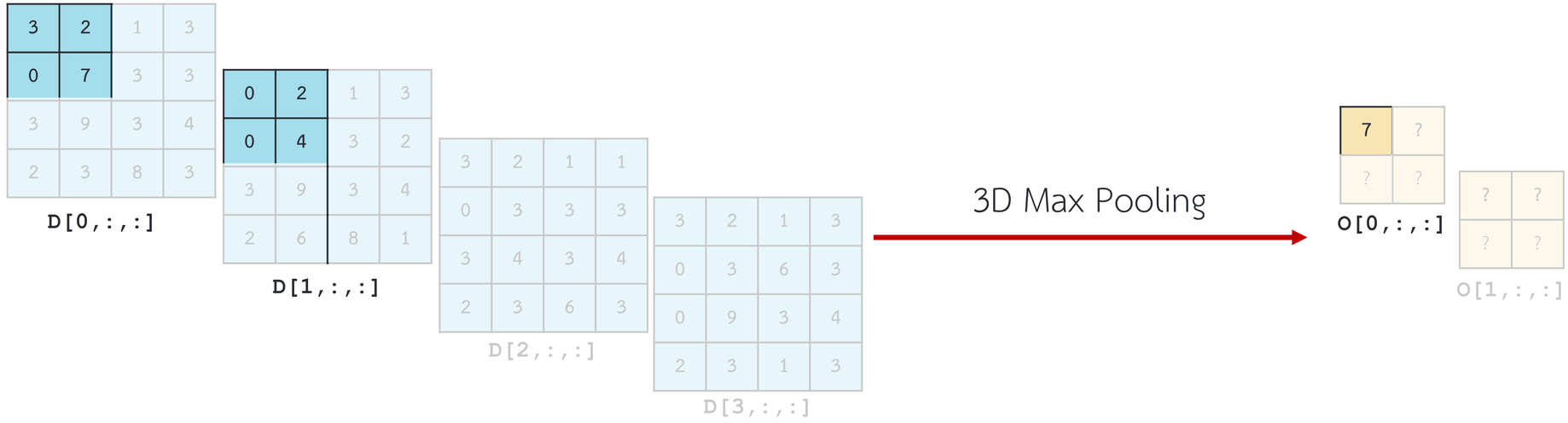
อ้างอิง https://ichi.pro/th/kar-chi-ngan-thi-la-khan-txn-3d-convolutional-neural-network-ni-keras-20820891513136

3D Max Pooling Layer

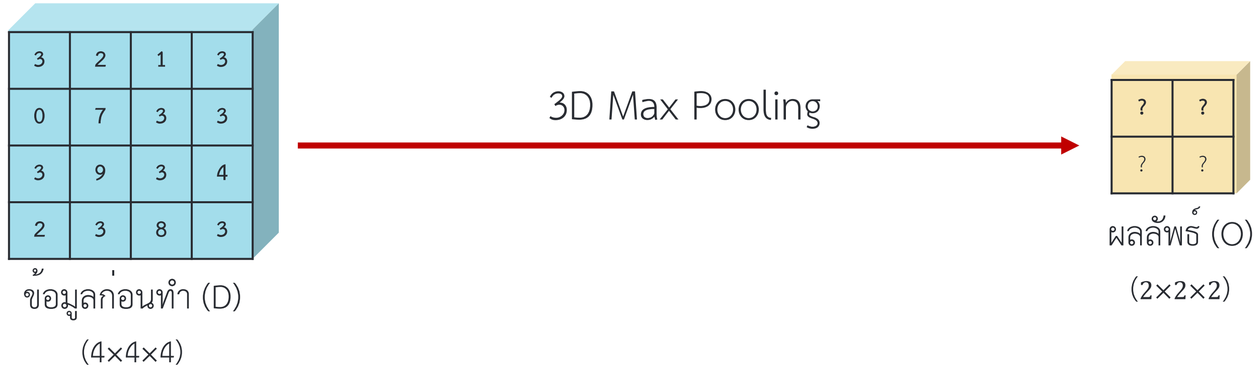


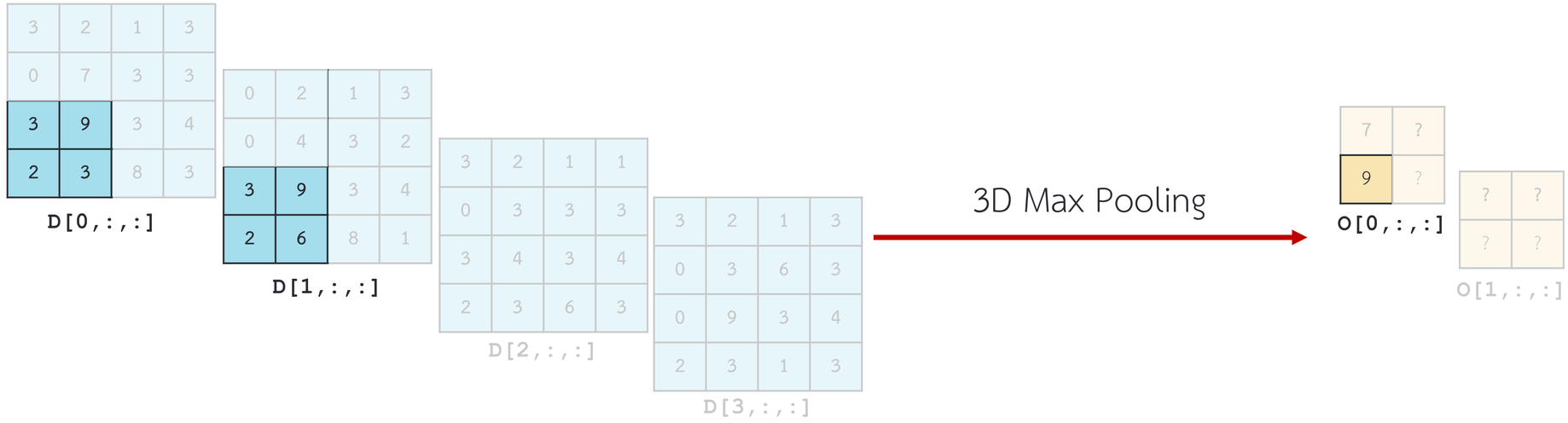
3D Max Pooling Layer



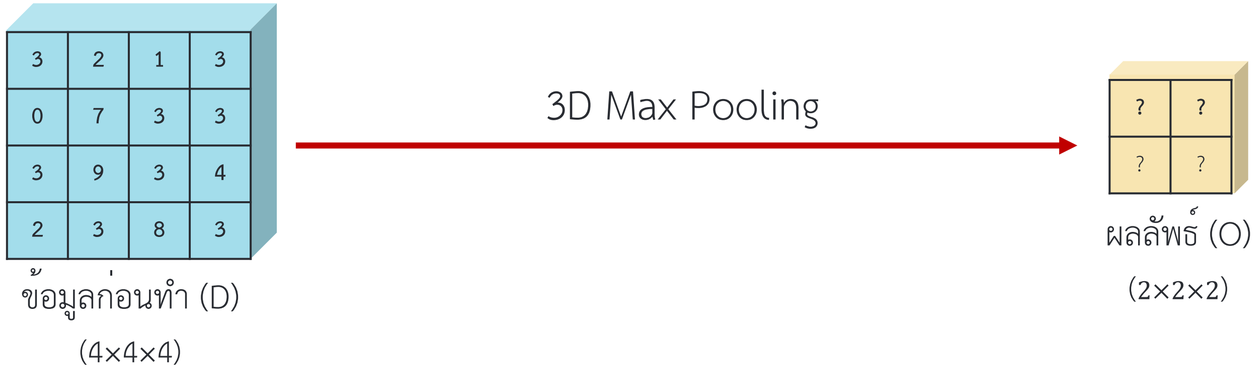


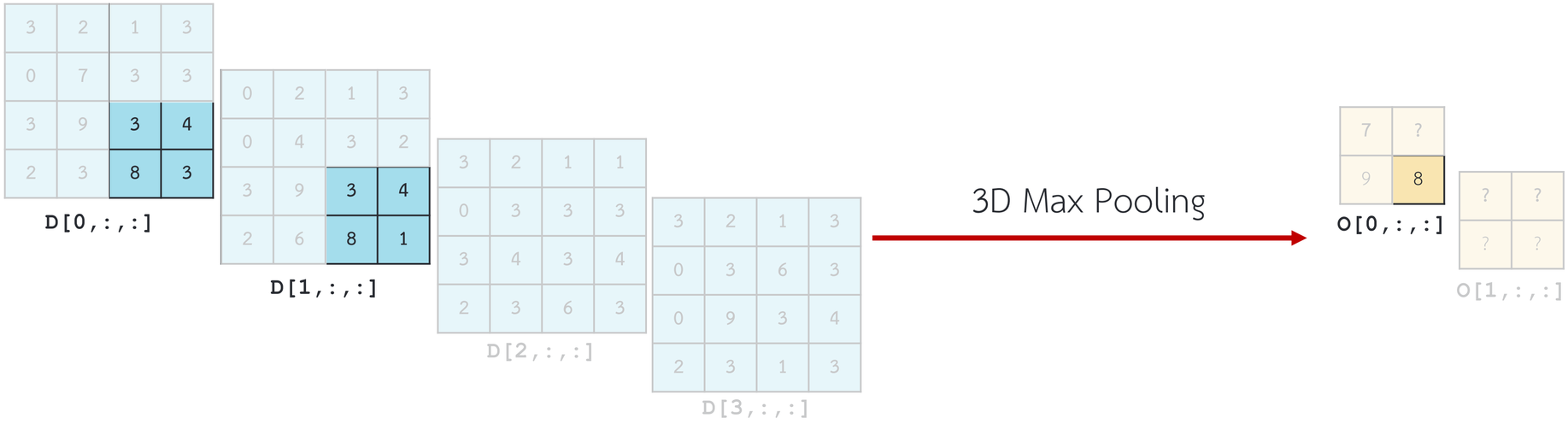
3D Max Pooling Layer



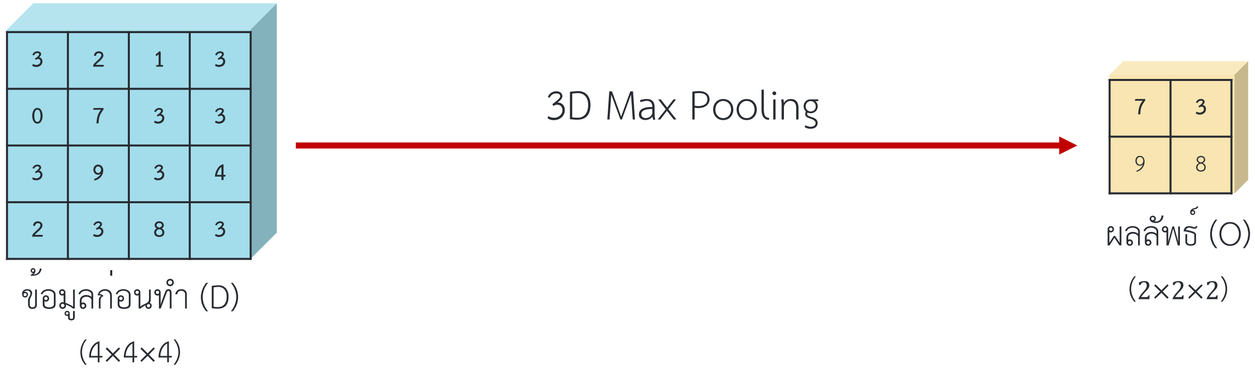


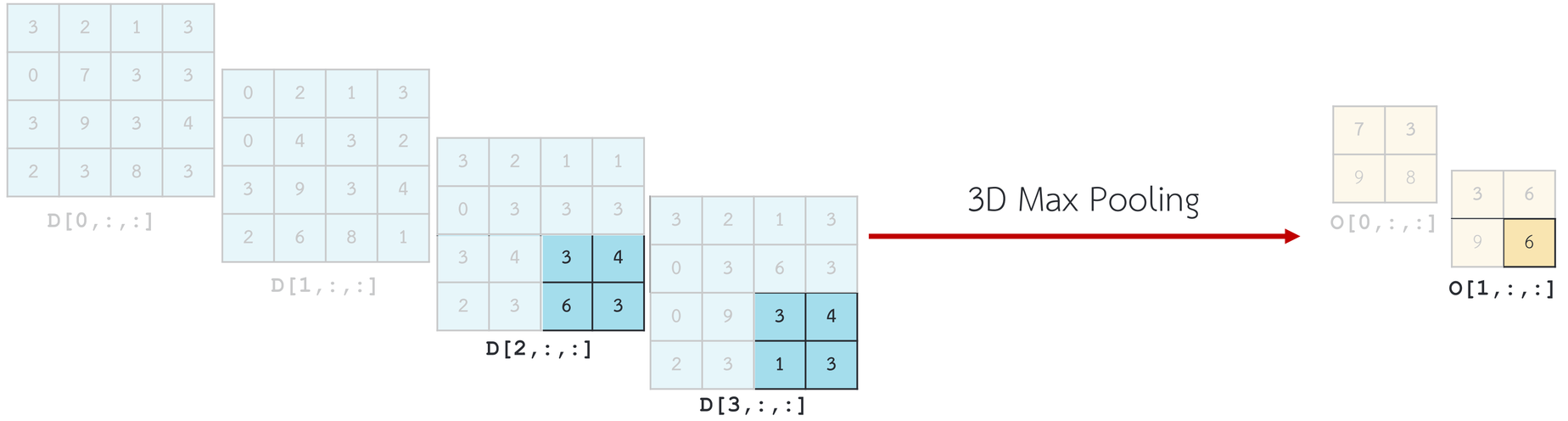
3D Max Pooling Layer



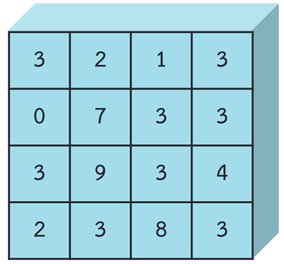


3D Max Pooling Layer





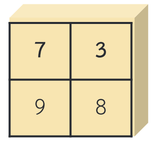
3D Max Pooling Layer



ข้อมูลก่อนทำ (D)

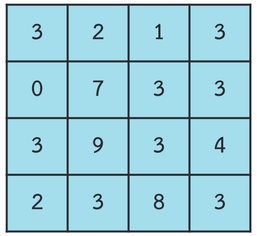
(4×4×4)

3D Max Pooling

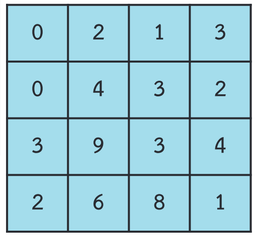


ผลลัพธ์ (O)

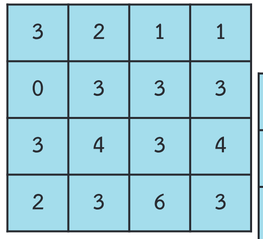
(2×2×2)



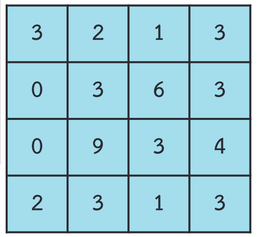
D[0,:,:]



D[1,:,:]



D[2,:,:]

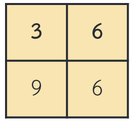


D[3,:,:]

3D Max Pooling



O[0,:,:]



O[1,:,:]

ตัวอย่างคำสั่ง Conv3d

PyTorch

Learn Ecosystem Edge Docs Blogs & News About Become a Member

2.3

Search Docs

- Community [+]
- Developer Notes [+]
- Language Bindings [+]
- Python API [-]
  - torch
  - torch.nn
  - torch.nn.functional
  - torch.Tensor
  - Tensor Attributes
  - Tensor Views
  - torch.amp
  - torch.autograd
  - torch.library
  - torch.cpu
  - torch.cuda
  - Understanding CUDA Memory Usage

Docs > torch.nn > Conv3d

Shortcuts

Conv3d

Conv3d

Conv3d

CLASS torch.nn.Conv3d(*in_channels*, *out_channels*, *kernel_size*, *stride=1*, *padding=0*, *dilation=1*, *groups=1*, *bias=True*, *padding_mode='zeros'*, *device=None*, *dtype=None*) [SOURCE]

Applies a 3D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N, C_{in}, D, H, W) and output $(N, C_{out}, D_{out}, H_{out}, W_{out})$ can be precisely described as:

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \star input(N_i, k)$$

where $\star$ is the valid 3D cross-correlation operator

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

- `stride` controls the stride for the cross-correlation.
- `padding` controls the amount of padding applied to the input. It can be either a string {'valid', 'same'} or

https://pytorch.org/docs/stable/generated/torch.nn.Conv3d.html

ตัวอย่างคำสั่ง Conv3d

ตัวอย่างการเรียกใช้งาน

```
# With square kernels and equal stride
m = nn.Conv3d(16, 33, 3, stride=2)

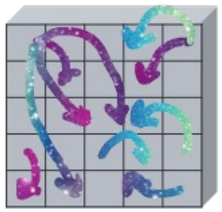
# non-square kernels and unequal stride and with padding
m = nn.Conv3d(16, 33, (3, 5, 2), stride=(2, 1, 1), padding=(4, 2, 0))
input = torch.randn(20, 16, 10, 50, 100)
output = m(input)
```

จากคำสั่ง m = nn.Conv3d(16, 33, 3, stride=2) ของ PyTorch เป็นการสร้างเลเยอร์การคอนโวลูชัน 3 มิติ (3D Convolutional Layer) ซึ่งมีพารามิเตอร์ต่างๆ ที่กำหนดลักษณะการทำงานของเลเยอร์ดังนี้

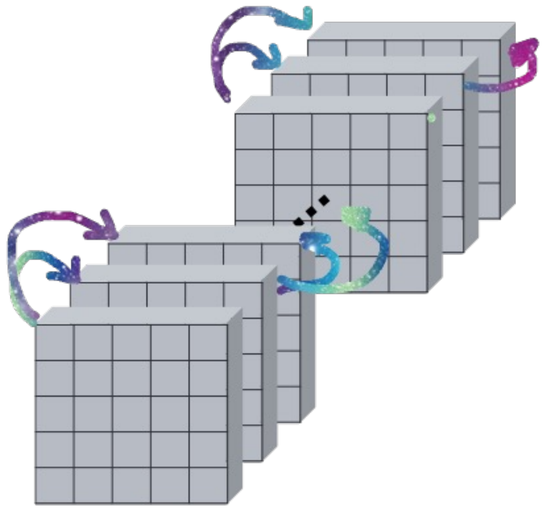
- 16 คือ จำนวนช่องอินพุต (input channels) ของข้อมูลที่เข้าสู่เลเยอร์คอนโวลูชัน
- 33 คือ จำนวนช่องเอาท์พุต (output channels) หรือจำนวนฟิลเตอร์ที่เลเยอร์คอนโวลูชันจะสร้างขึ้น
- 3 คือ ขนาดของเคอร์เนล (kernel size) หรือฟิลเตอร์ที่ใช้ในคอนโวลูชัน ซึ่งในที่นี้คือ 3 × 3 × 3
- stride=2 คือ กำหนดขนาดของการก้าวข้าม (stride) ของฟิลเตอร์ในทุกมิติ

https://pytorch.org/docs/stable/generated/torch.nn.Conv3d.html

โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ



คุณลักษณะเชิงตำแหน่ง (Spatial Feature)



ความสัมพันธ์ระหว่างเฟรมที่ต่อเนื่องกัน (Temporal Feature)

ถึงแม้ว่าโครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติจะค่อนข้างมีความแม่นยำมากกว่ามากกว่าโครงข่ายประสาทแบบคอนโวลูชันแบบ 2 มิติสำหรับข้อมูล 3 มิติ อีกทั้ง ยังสามารถสกัดคุณลักษณะเชิงตำแหน่ง (Spatial Feature) ของแต่ละเฟรมพร้อมกับสกัดหาความสัมพันธ์ระหว่างเฟรมที่ต่อเนื่องกัน (Temporal Feature) ได้

แต่อย่างไรก็ตาม โครงข่ายประสาทแบบคอนโวลูชันแบบ 3 มิติ ก็มีจำนวนพารามิเตอร์ที่ใช้ค่อนข้างมาก สิ่งนี้ทำให้มีความเสี่ยงในการเกิด Overfitting ค่อนข้างมาก และยังมี Pre-train ค่อนข้างน้อย

การเรียนรู้เชิงลึกสำหรับข้อมูลประเภทวีดิโอ

❑ สถาปัตยกรรม Slowfast (2019)

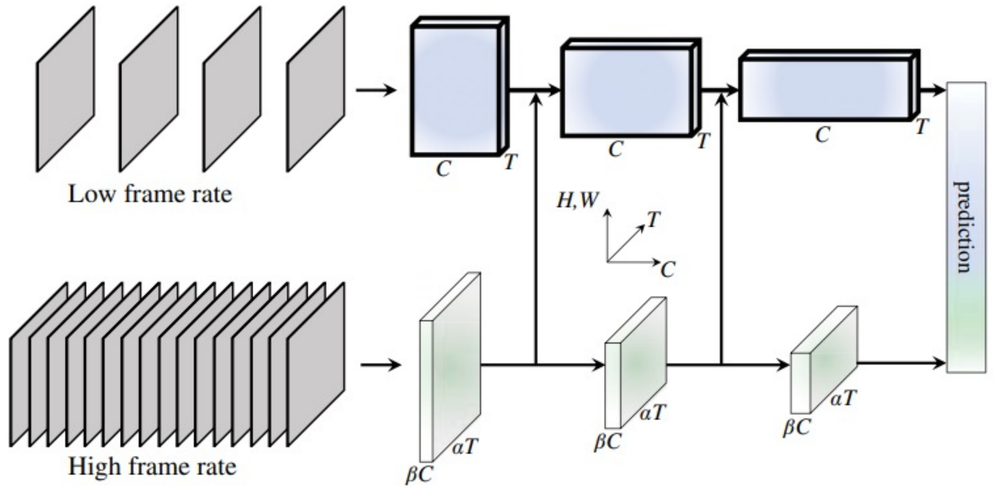


Figure 1. **A SlowFast network** has a low frame rate, low temporal resolution *Slow* pathway and a high frame rate, $\alpha\times$ higher temporal resolution *Fast* pathway. The Fast pathway is lightweight by using a fraction (β, *e.g.*, 1/8) of channels. Lateral connections fuse them.

อ้างอิง C. Feichtenhofer, H. Fan, J. Malik and K. He, "SlowFast Networks for Video Recognition," 2019 IEEE/CVF International Conference on Computer Vision (ICCV), Seoul, Korea (South), 2019, pp. 6201-6210, doi: 10.1109/ICCV.2019.00630.

❑ สถาปัตยกรรม MViTv2 (2021)

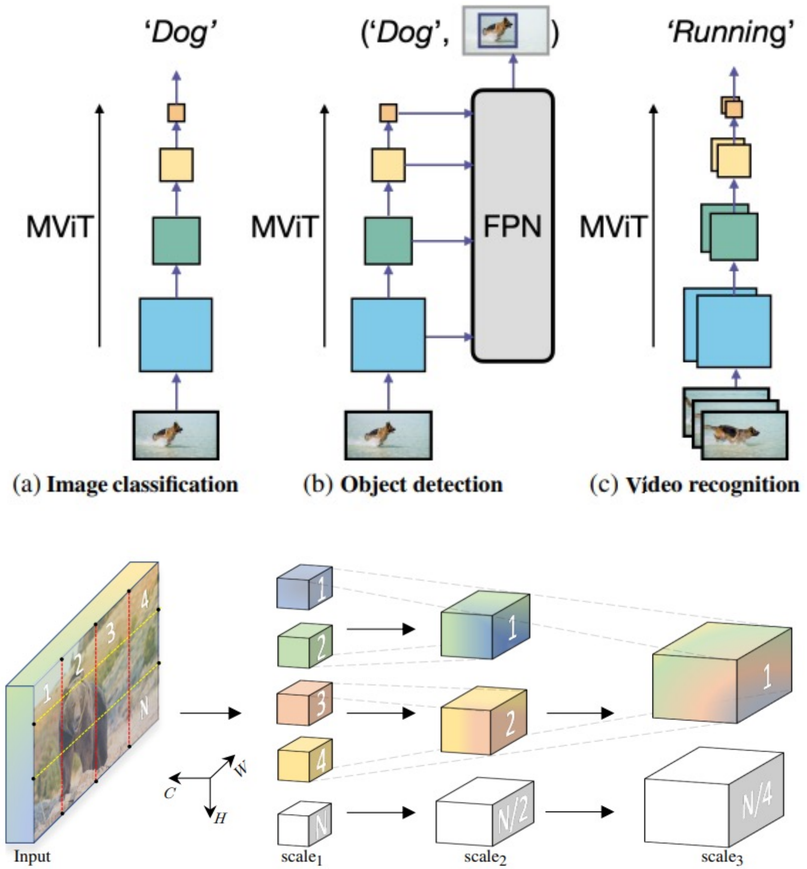


Figure 1. **Multiscale Vision Transformers** learn a hierarchy from

อ้างอิง https://arxiv.org/abs/2112.01526 และ https://arxiv.org/pdf/2104.11227

การเรียนรู้เชิงลึกสำหรับข้อมูลประเภทวีดิโอ

❑ สถาปัตยกรรม X3D (2020)

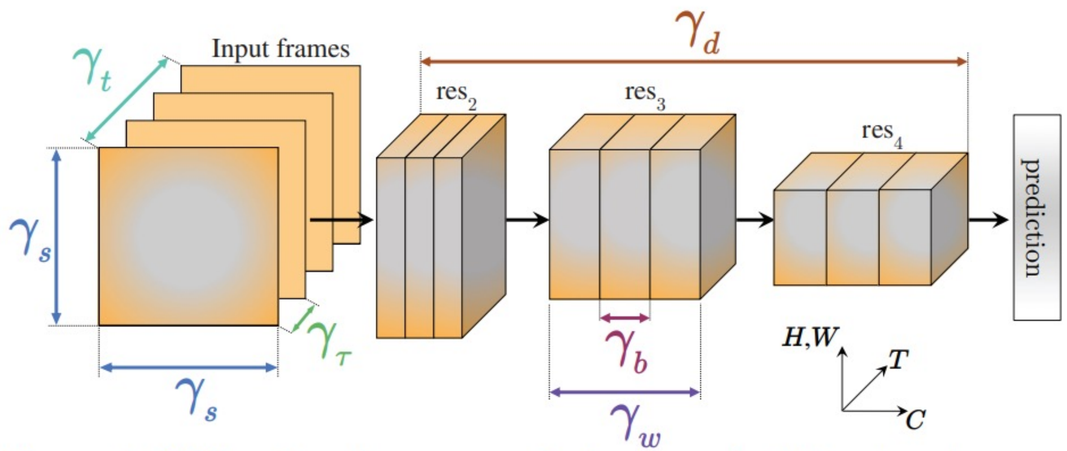


Figure 1. **X3D** networks progressively expand a 2D network across the following axes: Temporal duration γ_t, frame rate γ_τ, spatial resolution γ_s, width γ_w, bottleneck width γ_b, and depth γ_d.

อ้างอิง https://arxiv.org/pdf/2004.04730

❑ สถาปัตยกรรม Temporal Pyramid Network (2020)

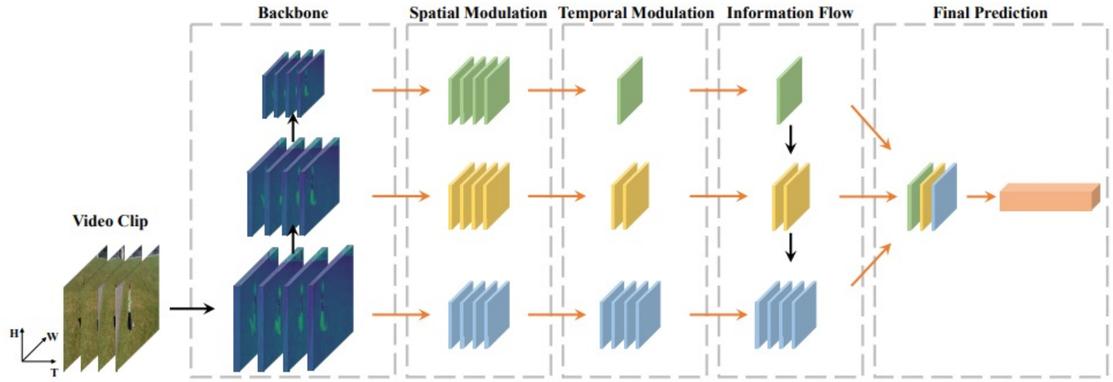


Figure 2. **Framework of TPN:** *Backbone Network* to extract multiple level features. *Spatial Semantic Modulation* spatially downsamples features to align semantics. *Temporal Rate Modulation* temporally downsamples features to adjust relative tempo among levels. *Information Flow* aggregates features in various directions to enhance and enrich level-wise representations. *Final Prediction* rescales and concatenates all levels of pyramid along channel dimension. Note that the channel dimensions in *Final Prediction* and corresponding operations are omitted for brevity.

อ้างอิง https://arxiv.org/pdf/2004.03548

การเรียนรู้เชิงลึกสำหรับข้อมูลประเภทวีดิโอ

❑ สถาปัตยกรรม UniFormer (2023)

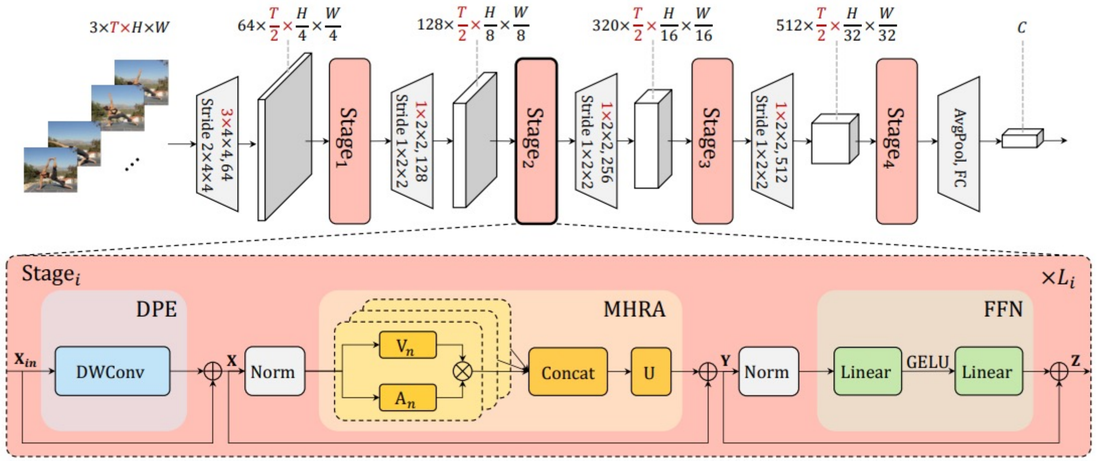


Fig. 3. Unified transFormer (UniFormer). A UniFormer block consists of three key modules, i.e., Dynamic Position Embedding (DPE), Multi-Head Relation Aggregrator (MHRA), and Feed Forward Network (FFN). The dimensions highlighted in red only exist for the video input, while all of them are equal to one for image input. More detailed explanations can be found in Section 3.

อ้างอิง https://arxiv.org/pdf/2201.09450

❑ สถาปัตยกรรม Video Swin Transformer (2021)

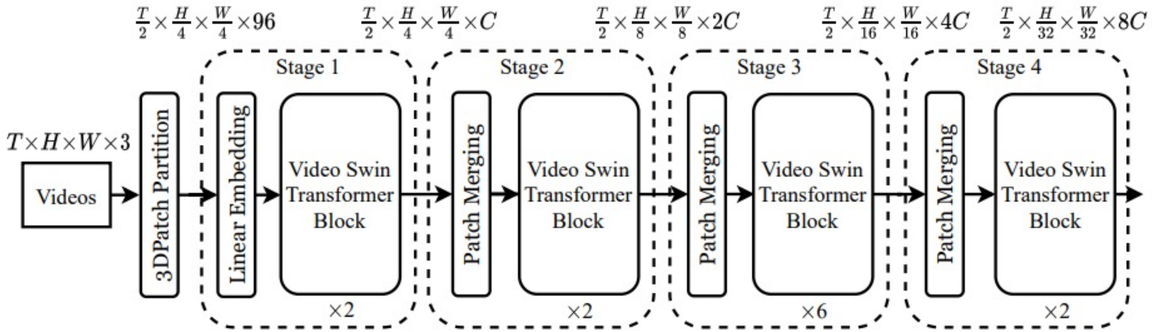


Figure 1: Overall architecture of Video Swin Transformer (tiny version, referred to as Swin-T).

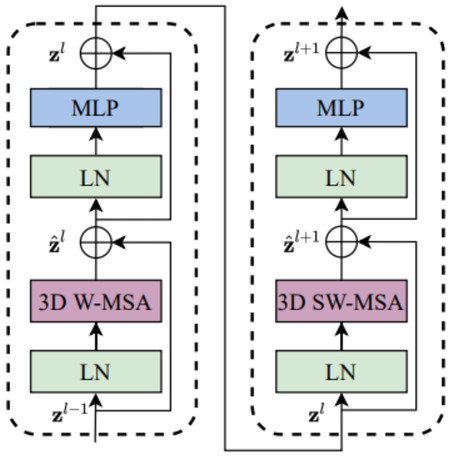
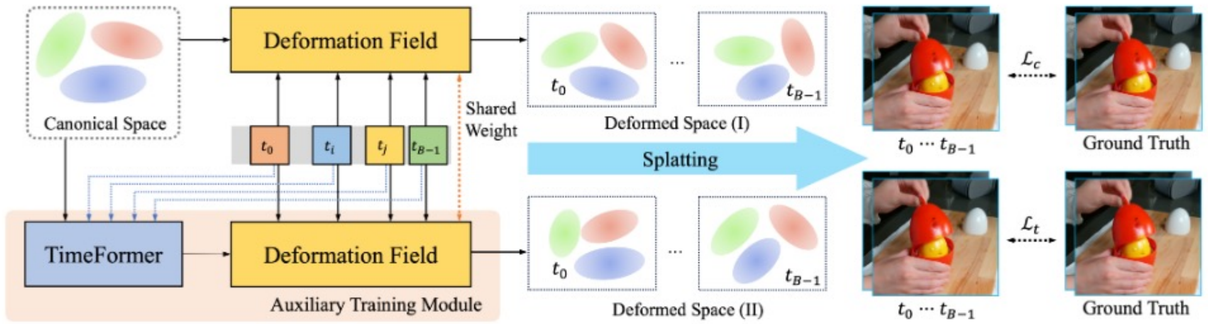


Figure 2: An illustration of two successive Video Swin Transformer blocks.

อ้างอิง https://arxiv.org/pdf/2106.13230

การเรียนรู้เชิงลึกสำหรับข้อมูลประเภทวีดิโอ

❑ สถาปัตยกรรม TimeFormer (2025)



The Framework of Deformable 3D Gaussians Reconstruction with TimeFormer. Existing deformable 3D Gaussians framework usually includes the canonical space and the deformation field (first row), we incorporate TimeFormer to capture cross-time relationships and explore motion patterns implicitly (second row). We share weights of two deformation fields to transfer the learned motion knowledge. This allows us to exclude this Auxiliary Training Module during inference.

อ้างอิง https://patrickddj.github.io/TimeFormer/

❑ สถาปัตยกรรม VideoMAE (2022)

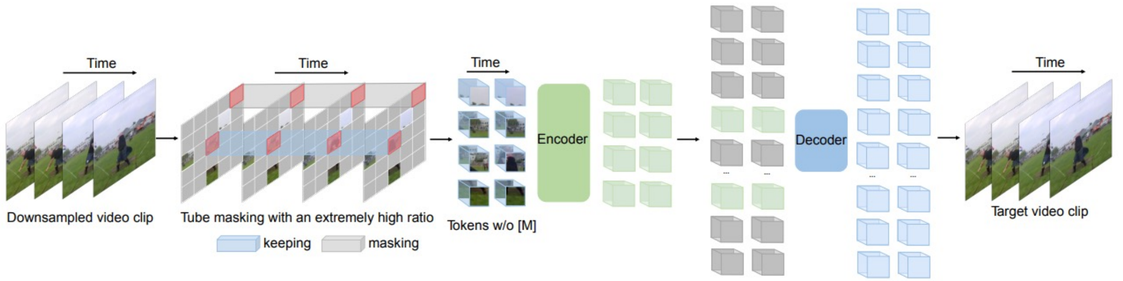
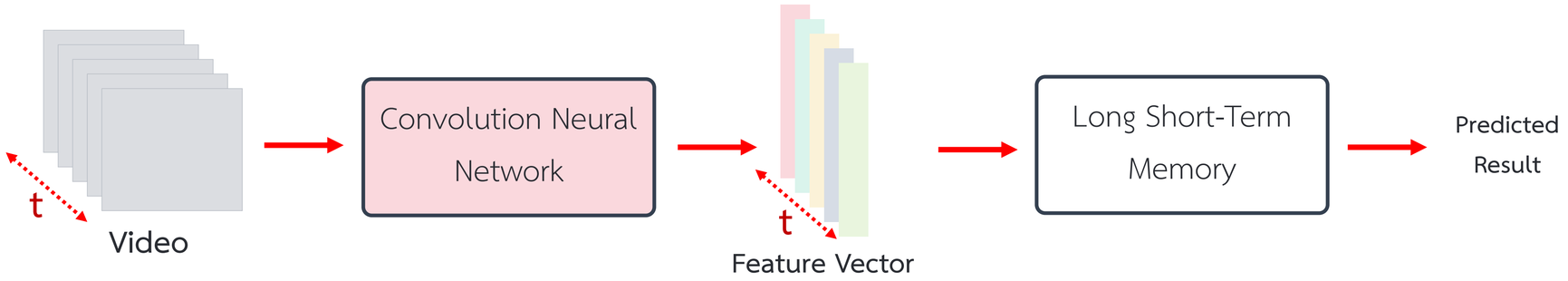


Figure 1: **VideoMAE** performs the task of masking random cubes and reconstructing the missing ones with an asymmetric encoder-decoder architecture. Due to high redundancy and temporal correlation in videos, we present the customized design of tube masking with an extremely high ratio (90% to 95%). This simple design enables us to create a more challenging and meaningful self-supervised task to make the learned representations capture more useful spatiotemporal structures.

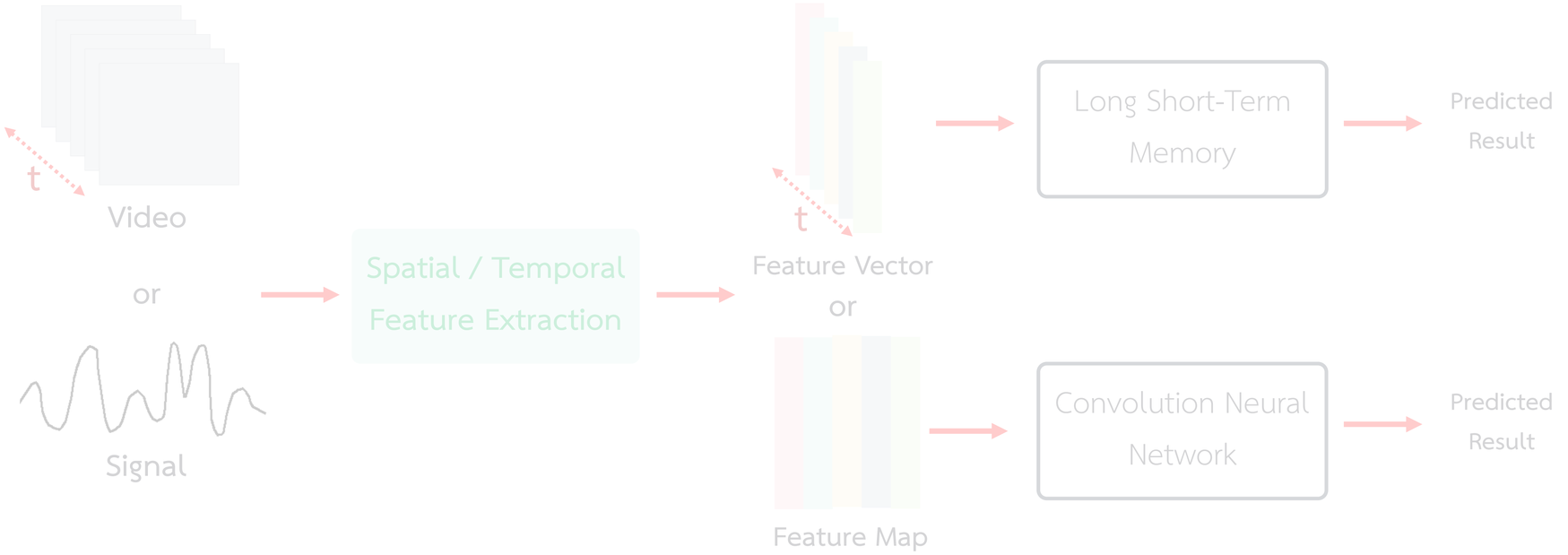
อ้างอิง https://arxiv.org/pdf/2203.12602

การใช้งาน Deep Learning สำหรับวีดิโอ

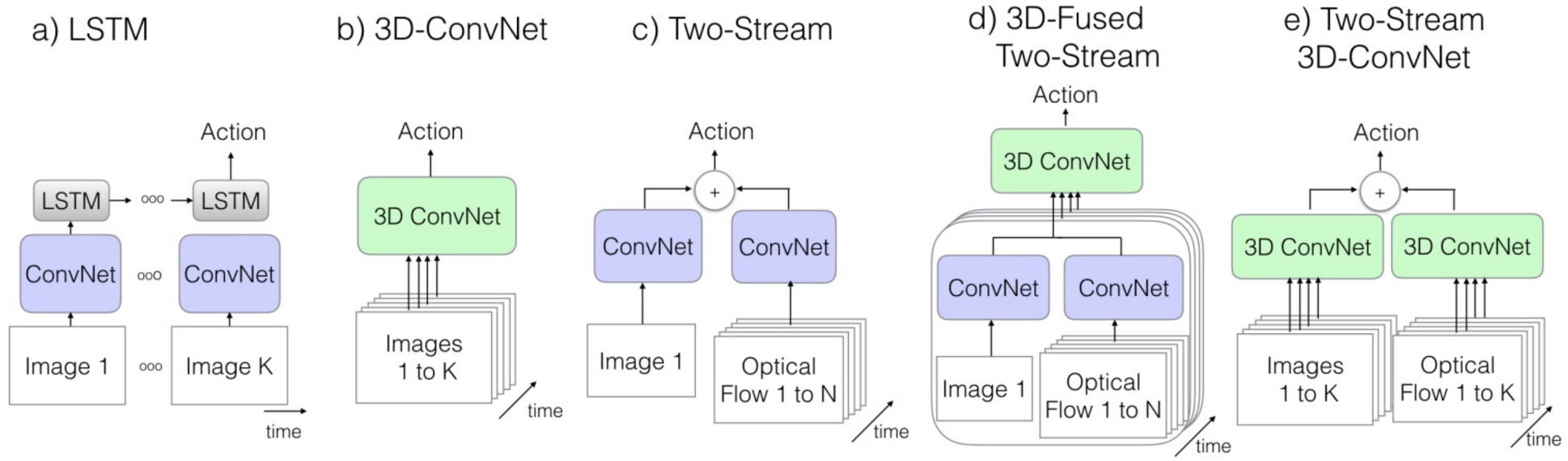
1



2

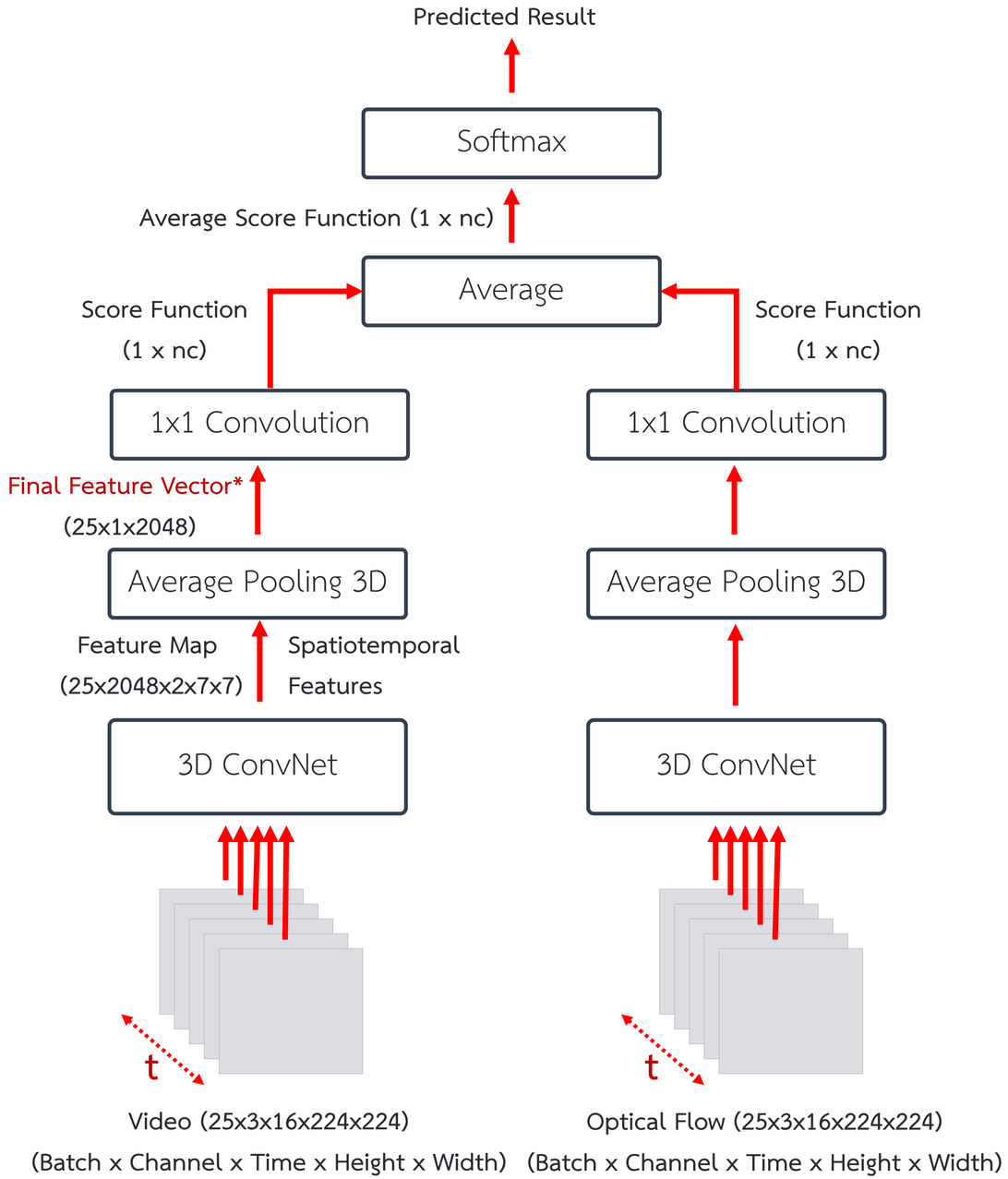


ปัญหาหลักของงาน Action Recognition



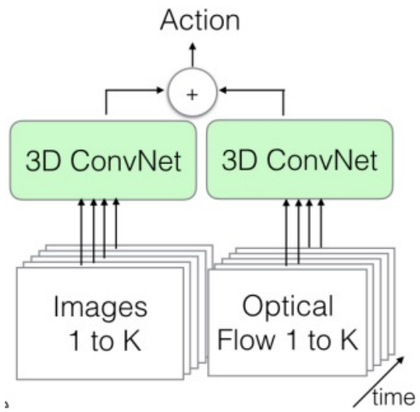
- ชุดข้อมูลวิดีโอที่ใช้กันทั่วไป เช่น UCF-101, HMDB-51 มีขนาดค่อนข้างเล็ก ทำให้โมเดลที่ซับซ้อนมีโอกาสเกิดปัญหา Overfit
- สถาปัตยกรรมที่ใช้ในงานภาพนิ่ง (ConvNet) นั้นพัฒนาไปมาก แต่เมื่อนำมาใช้งานบนวิดีโอที่ข้อมูลมีมิติเวลาร่วมด้วยค่อนข้างยาก
- สถาปัตยกรรมที่ใช้ภาพนิ่งและคำนึงถึงมิติเวลาร่วมด้วย (3D ConvNet) มักมีพารามิเตอร์มาก ทำให้กระบวนการฝึกสอนทำได้ยาก
- เพื่อแก้ไขปัญหาข้างต้น จึงได้นำแนวคิด *Two-Stream* ที่แยกการประมวลผลระหว่าง “ภาพ RGB” และ “อัตราการเคลื่อนที่ (Optical Flow)” มาใช้งาน แต่ส่วนใหญ่ยังอาศัย ConvNet แบบ 2D จากนั้น จึงนำผลลัพธ์ของทั้งสอง Stream มารวมผล (fusion) ในภายหลัง

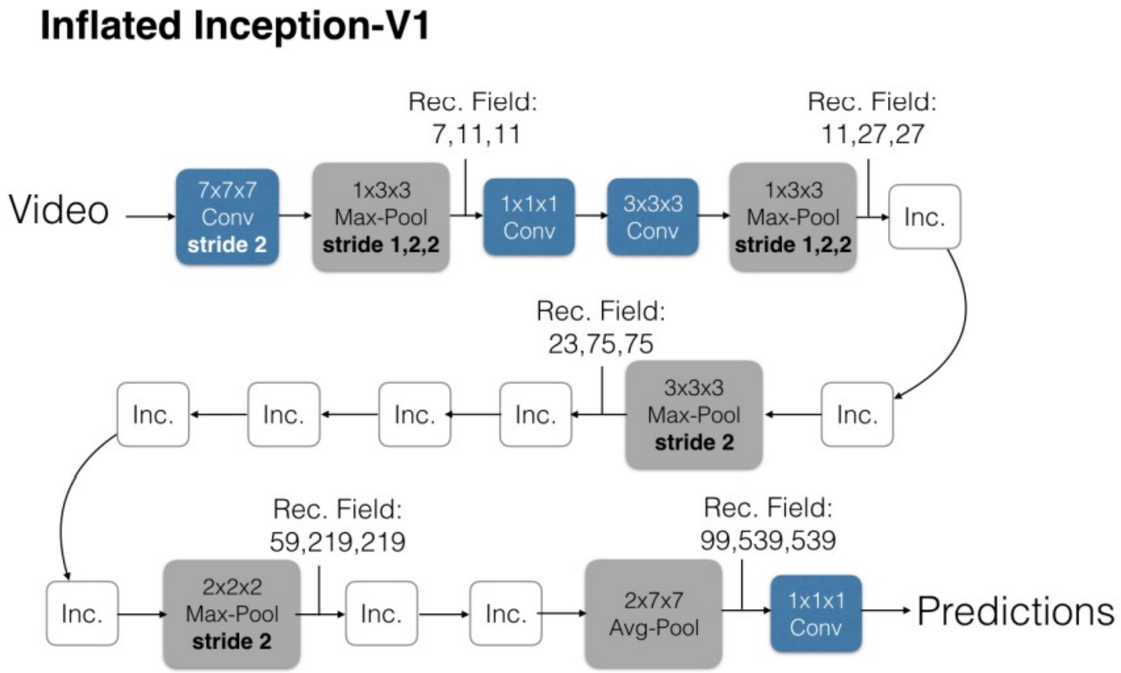
อ้างอิง https://arxiv.org/pdf/1705.07750



สถาปัตยกรรม “Inflated 3D ConvNet” หรือเรียกว่า I3D คือ การเพิ่ม “มิติเวลา (inflate) ลงในโมเดล 2D ให้กลายเป็น 3D” โดย I3D นำเสนอวิธีที่ผสานจุดเด่นของโมเดล 2D ที่ได้รับการออกแบบดีแล้ว (เช่น Inception) เข้ากับแนวคิด 3D เพื่อให้โมเดล “มองเห็น” ทั้ง spatial และ temporal ได้พร้อมกัน ซึ่งมีแนวคิดที่สำคัญดังนี้

สถาปัตยกรรม Inflated 3D ConvNet





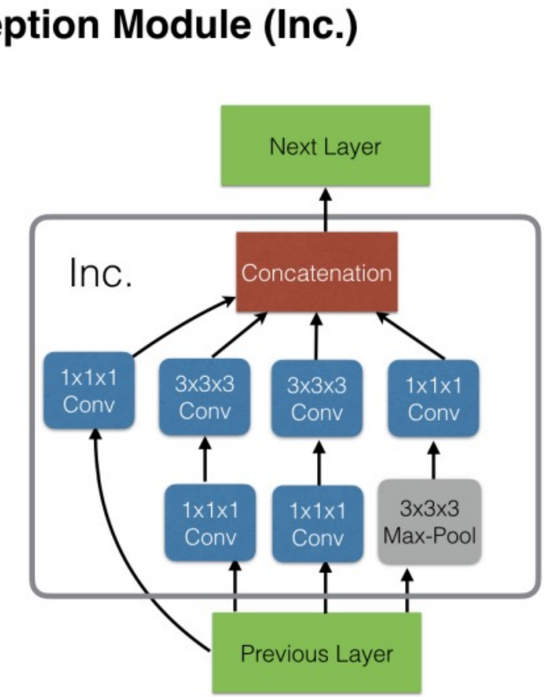
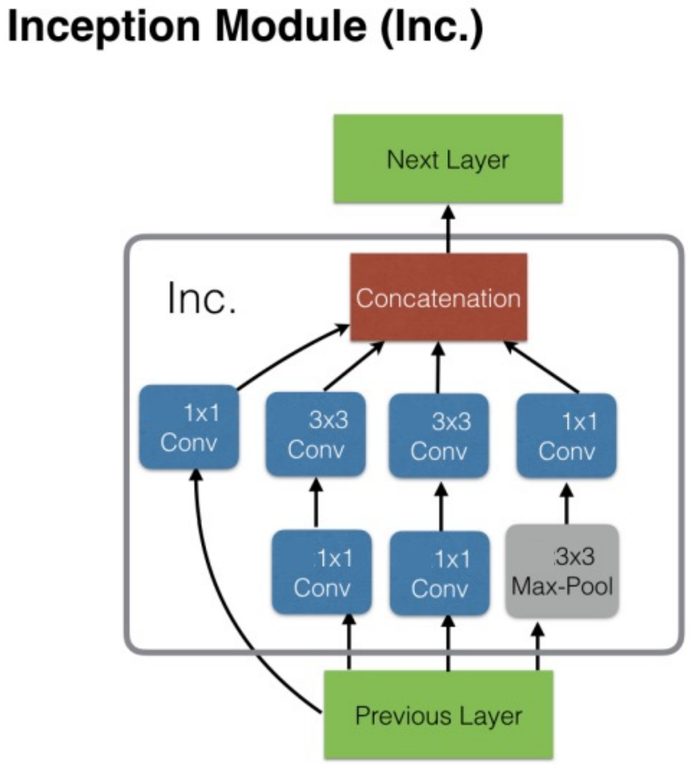
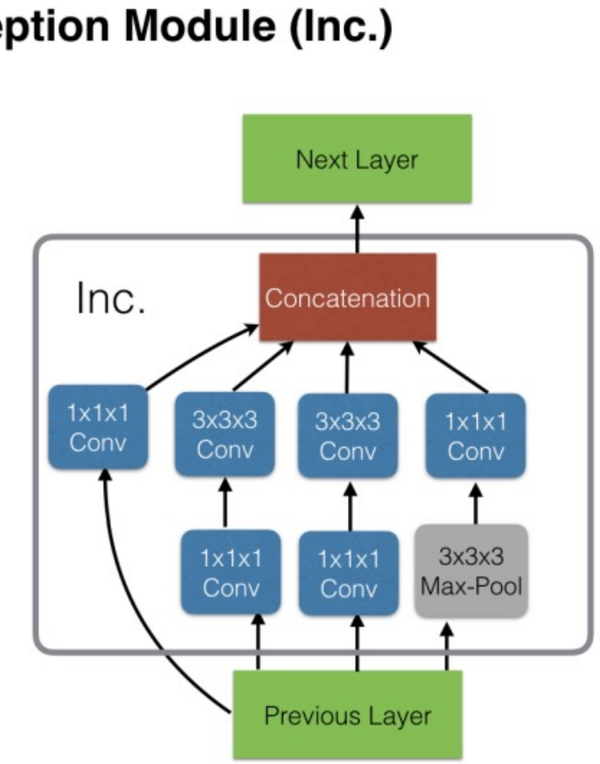


Figure 3. The Inflated Inception-V1 architecture (left) and its detailed inception submodule (right). The strides of convolution and pooling operators are 1 where not specified, and batch normalization layers, ReLu's and the softmax at the end are not shown. The theoretical sizes of receptive field sizes for a few layers in the network are provided in the format "time,x,y" – the units are frames and pixels. The predictions are obtained convolutionally in time and averaged.

อ้างอิง https://arxiv.org/pdf/1705.07750

สถาปัตยกรรม Inflated 3D ConvNet





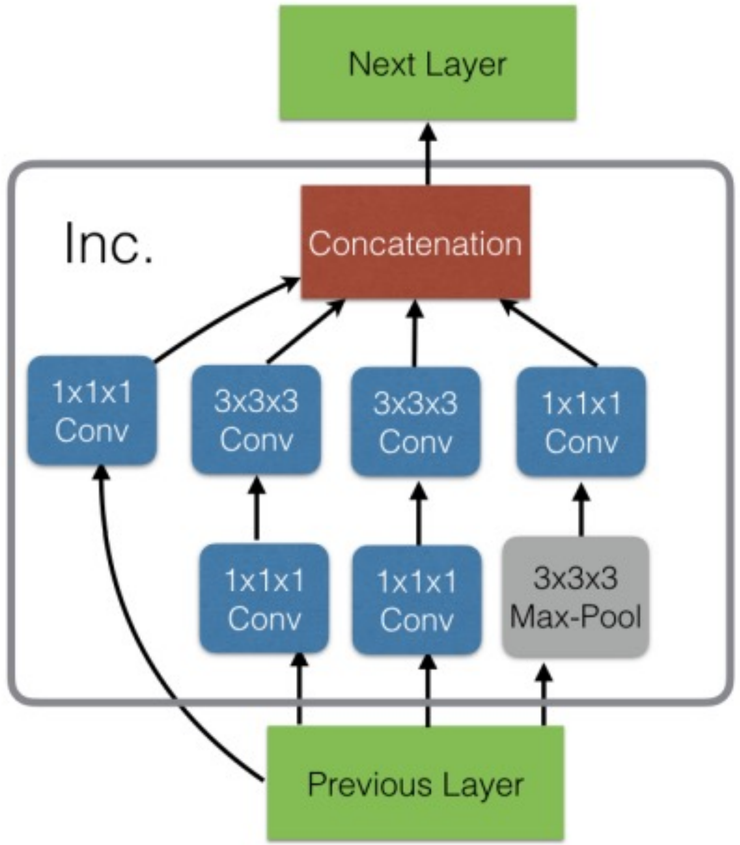
❑ Inflation (ขยายเป็น 3D)

- เริ่มจากโครงสร้าง ConvNet 2D ที่ออกแบบดีแล้ว (Inception-v1)
- แปลงตัวคอนโวลูชันฟิลเตอร์และการ pooling ที่เดิมเป็น 2D (ขนาด $N \times N$) ให้กลายเป็น 3D ที่มีขนาด $T \times N \times N$ โดย T คือความยาวในมิติเวลาหนึ่ง
- จุดสำคัญคือ การเลือกว่า temporal stride / temporal pooling จะเป็นอย่างไร เพื่อให้ receptive field ทางเวลาเหมาะสม

อ้างอิง https://arxiv.org/pdf/1705.07750

สถาปัตยกรรม Inflated 3D ConvNet

Inception Module (Inc.)



❑ตัวดำเนินการ 1×1×1 convolution ใน I3D

ใช้ 3D convolution ซึ่ง filter มี 3 มิติ ได้แก่

- T = temporal (เวลา, กี่ frame ต่อ kernel)
- H, W = spatial (ความสูง × ความกว้างของภาพ)

ดังนั้น 1×1×1 convolution หมายถึง kernel ขนาด

- Temporal size = 1 (ไม่ครอบเวลา, ใช้ frame เดียวในเชิงเวลา)
- Spatial size = 1×1 (ไม่ครอบพื้นที่, ใช้ pixel ตรงตำแหน่งเดียว)

สรุป มันไม่ “มองพื้นที่” และไม่ “มองเวลา” แต่ทำการ linear combination ของ channels เท่านั้น

❑บทบาทของตัวดำเนินการ 1×1×1 convolution ใน I3D

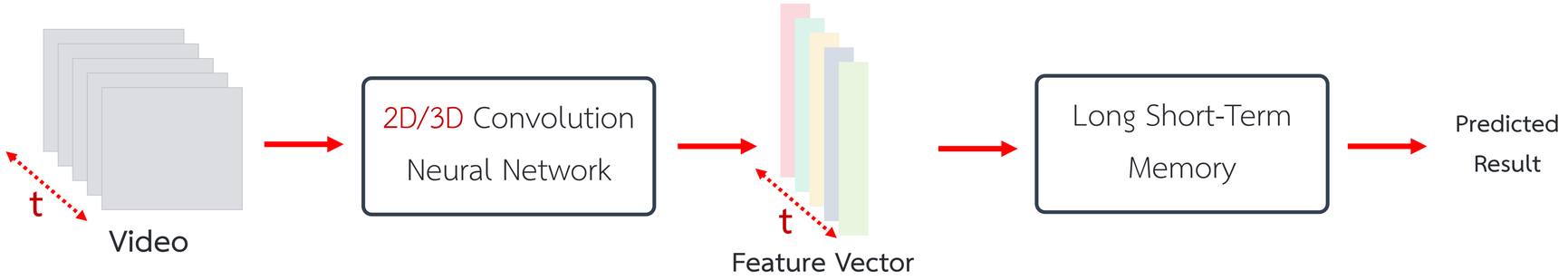
ทำหน้าที่คล้าย 1×1 conv ในโมเดล 2D แต่ขยายสู่ 3D เพื่อ

- Channel reduction ทำให้ลด computation, memory
- Channel mixing รวมข้อมูลจากหลาย channel feature ให้เกิดการผสม (linear combination) ทำให้โมเดลเรียน feature representation ที่มีความหมายใหม่ ๆ ได้
- Non-linearity เมื่อทำงานร่วมกับ ReLU และ BatchNorm จะเพิ่มความซับซ้อนของฟังก์ชันที่เรียนรู้
- Flexibility ในการออกแบบ Inception branch เพื่อให้ได้ feature “สั้นและเบา”

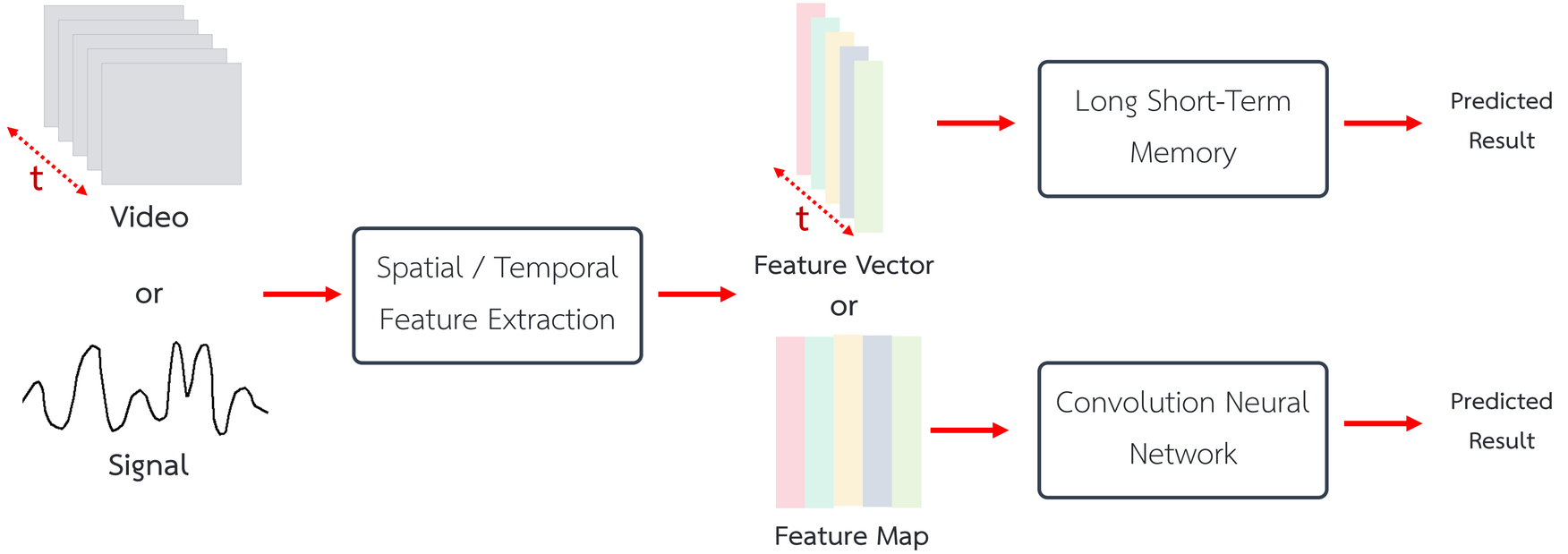
อางอิง https://arxiv.org/pdf/1705.07750

การใช้งาน Deep Learning สำหรับวีดิโอ

1

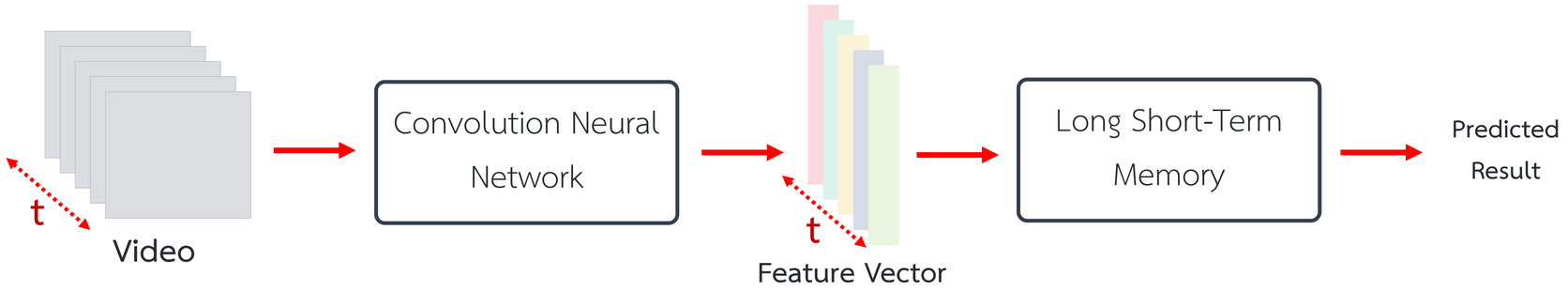


2



การใช้งาน Deep Learning สำหรับวีดิโอ

1



3



4

